UNIVERSITÀ DEGLI STUDI DI MILANO BICOCCA

Università della Svizzera italiana — Software Institute

On Using Wave Function Collapse for Depicting Software Systems

Author:

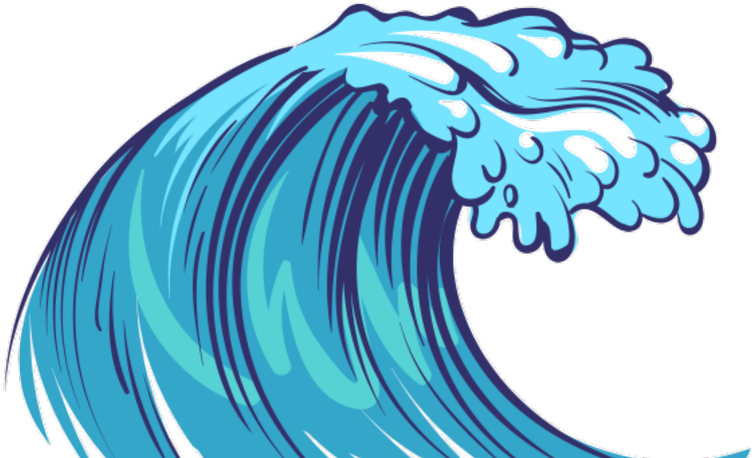
Simone Lesinigo

Supervised by:

Prof. Dr. Michele Lanza

Prof. Dr. Claudia Raibulet

Dr. Marco Raglianti



Supervisors



Prof. Dr. Michele Lanza

USI advisor



Dr. Marco Raglianti

USI co-advisor



Prof. Dr. Claudia Raibulet

UniMiB co-advisor

Software Comprehension

Software Comprehension

70%

of a Developer's Time

is spent on program comprehension
rather than writing new code.

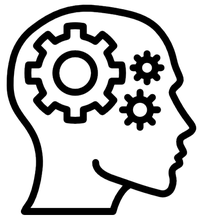
[Minelli et al., IEEE International Conference on Program Comprehension 2015]

Software Comprehension

70%

of a Developer’s Time

is spent on program comprehension rather than writing new code.



Building Mental Model

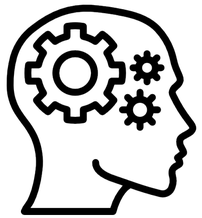
[Minelli et al., IEEE International Conference on Program Comprehension 2015]

Software Comprehension

70%

of a Developer's Time

is spent on program comprehension rather than writing new code.



Building Mental Model

- Mapping low-level code to high-level software architecture

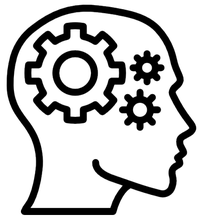
[Minelli et al., IEEE International Conference on Program Comprehension 2015]

Software Comprehension

70%

of a Developer's Time

is spent on program comprehension rather than writing new code.



Building Mental Model

- Mapping low-level code to high-level software architecture
- A demanding cognitive process

[Minelli et al., IEEE International Conference on Program Comprehension 2015]

Ball's Dilemma

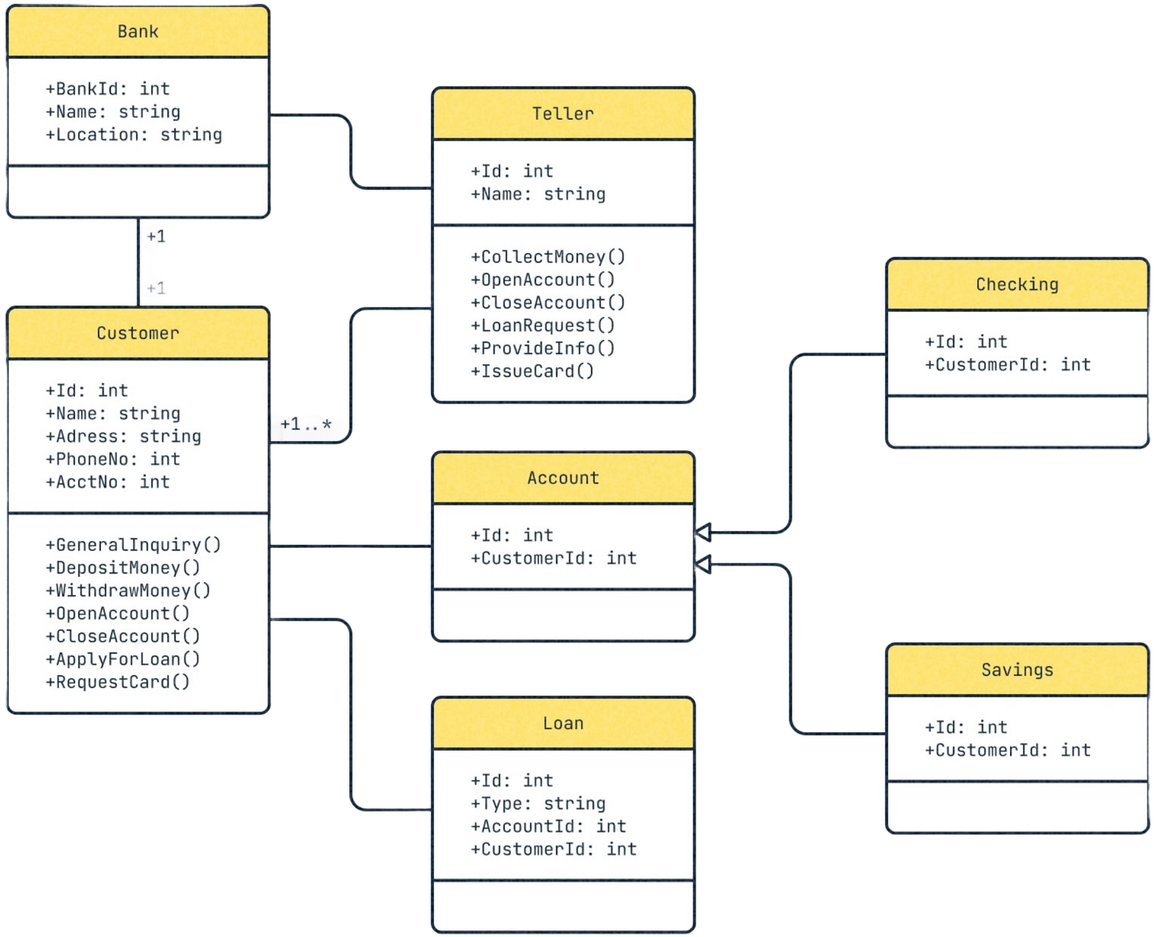
"Software is intangible, having no physical shape or size"

Thomas Ball



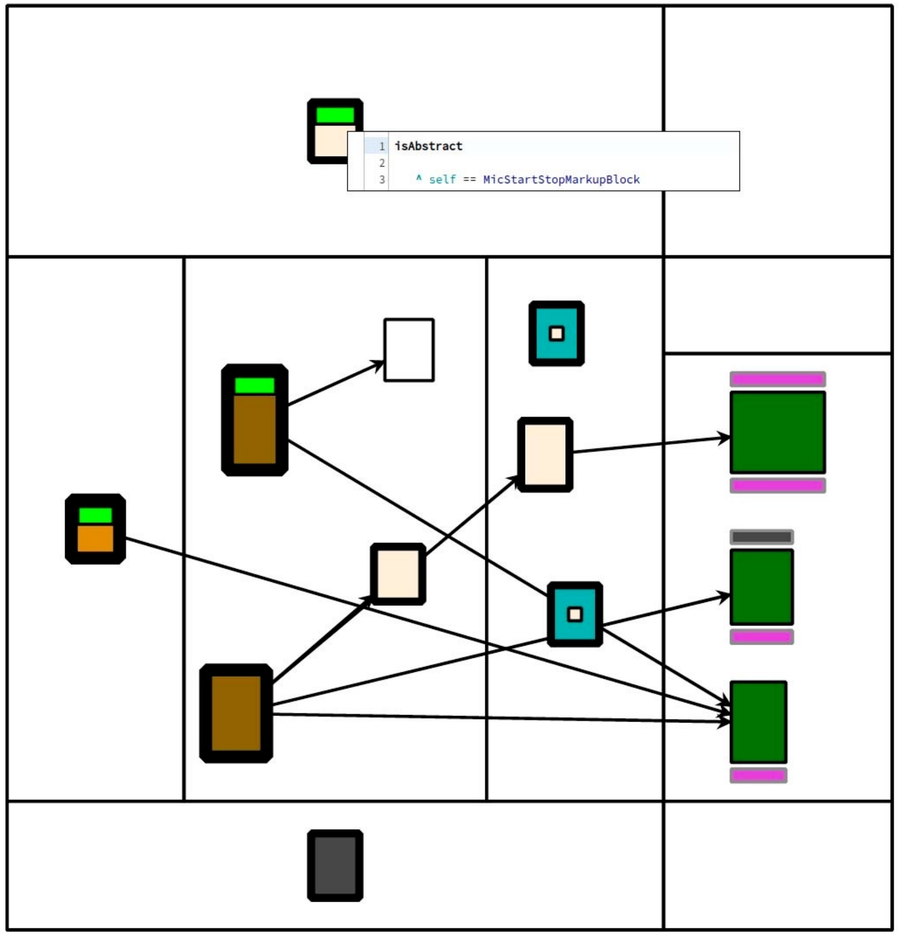
[Ball and Eick, IEEE Computer 1996]

Software Representations



UML Diagram

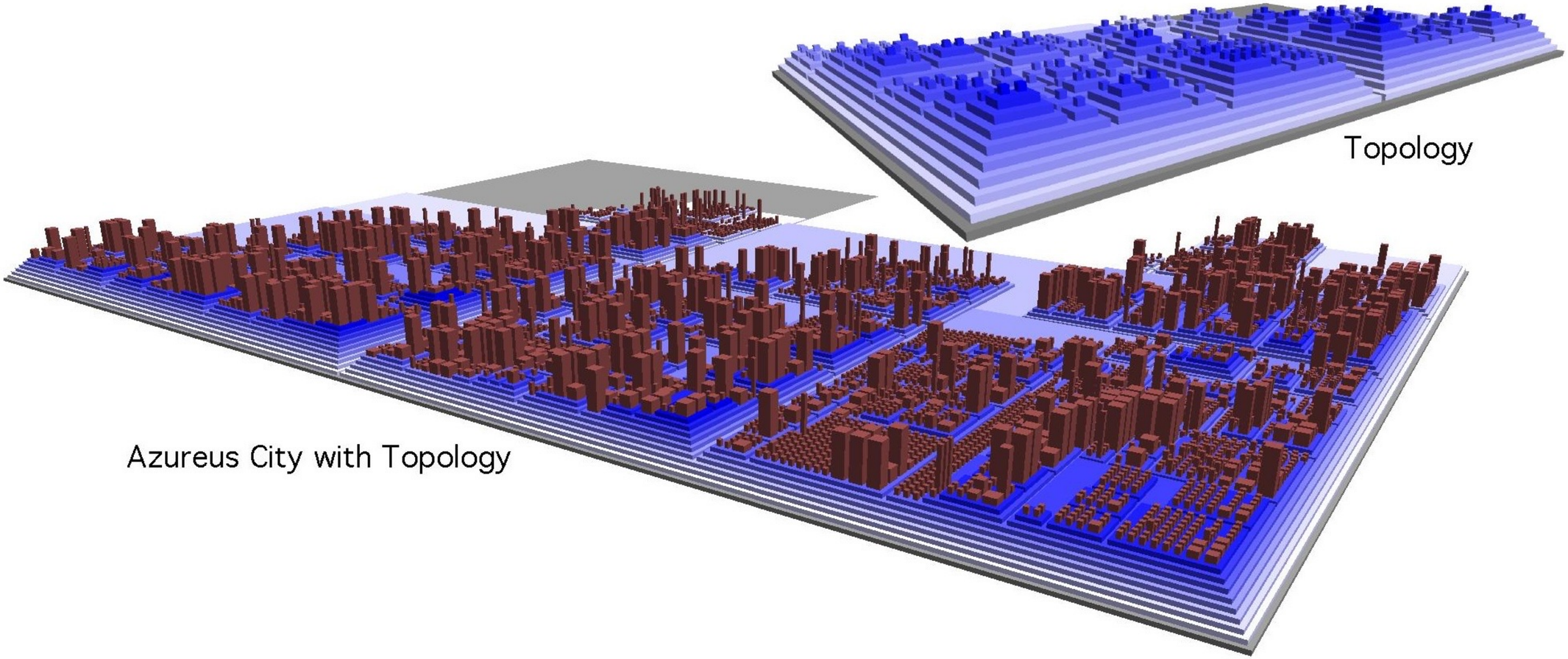
Software Representations



Class BLUEPRINT V2

[Agouf et al., IEEE Working Conference on Software Visualization 2022]

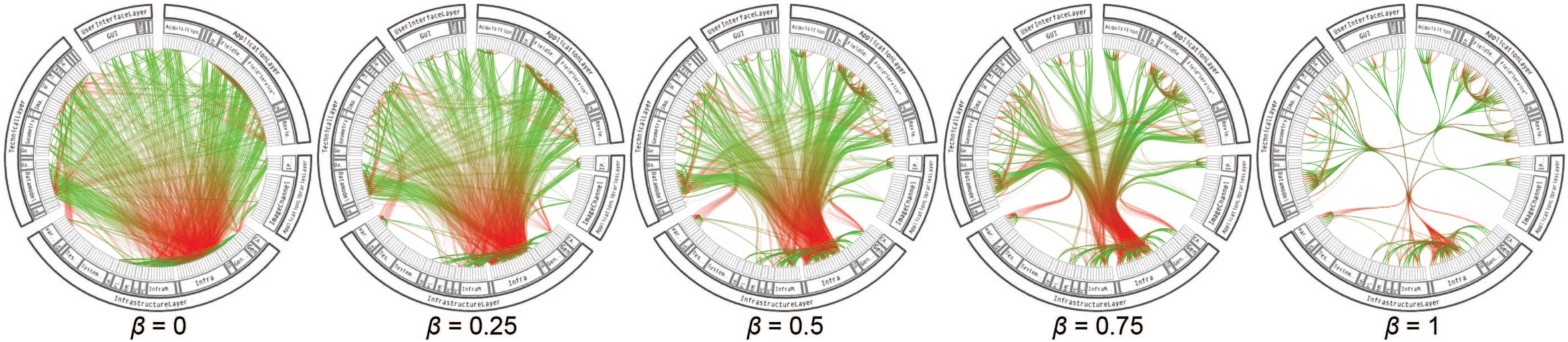
Software Representations



CodeCity

[Wettel and Lanza, IEEE International Workshop on Visualizing Software for Understanding and Analysis 2007]

Software Representations



Edge Bundling

[Holten, IEEE Transactions on Visualization and Computer Graphics 2006]

Problem Statement

Problem Statement

1. Layout Problem

How to place software entities?



Problem Statement

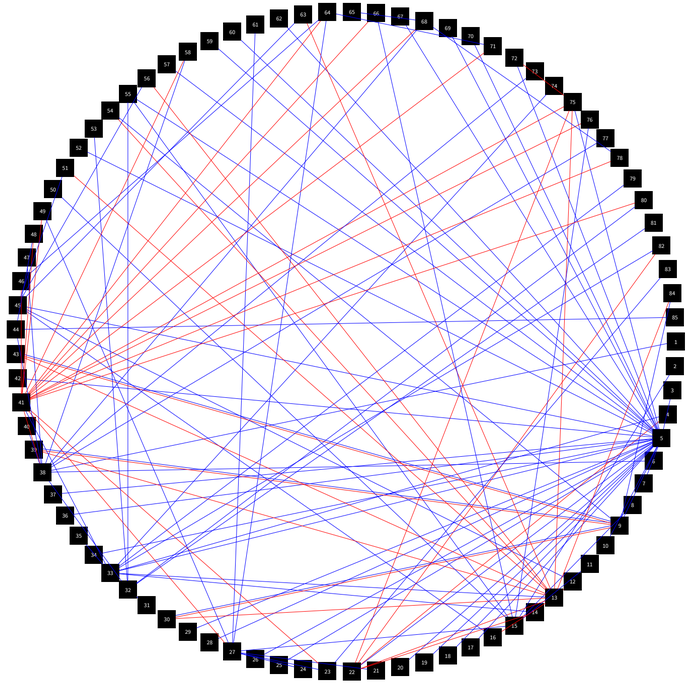
1. Layout Problem

How to place software entities?



2. Connectivity Problem

How to draw software relationships?



Problem Statement

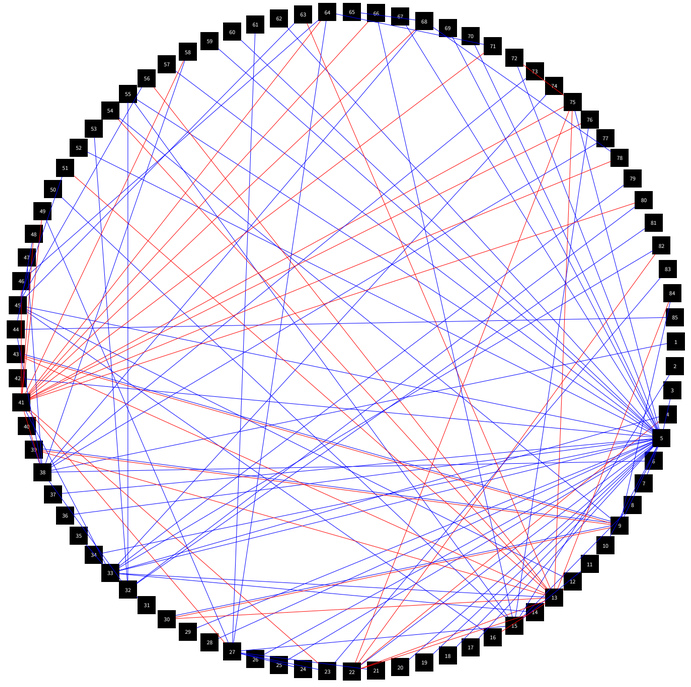
1. Layout Problem

How to place software entities?



2. Connectivity Problem

How to draw software relationships?



Our approach: **Multidimensional Scaling** (MDS)

[Torgerson and Warren, Psychometrika 1952]

Problem Statement

1. Layout Problem

How to place software entities?

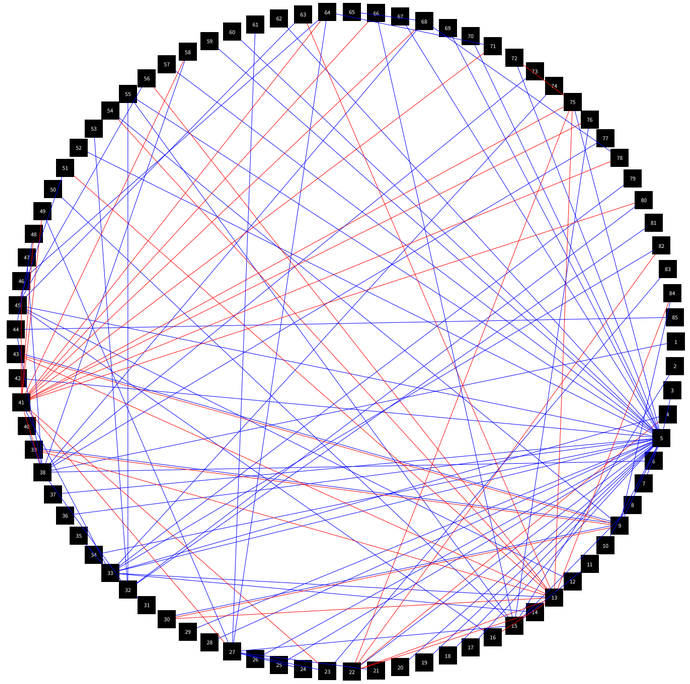


Our approach: **Multidimensional Scaling** (MDS)

[Torgerson and Warren, Psychometrika 1952]

2. Connectivity Problem

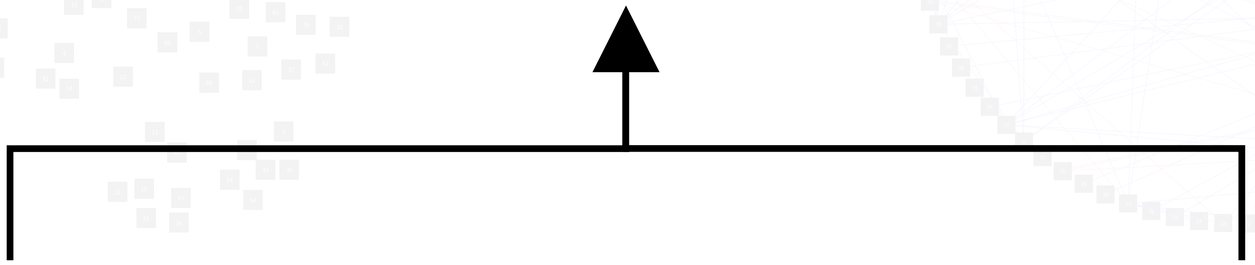
How to draw software relationships?



Our approach: **Wave Function Collapse** (WFC)

[Kart and Smith, IEEE Transactions on Games 2022]

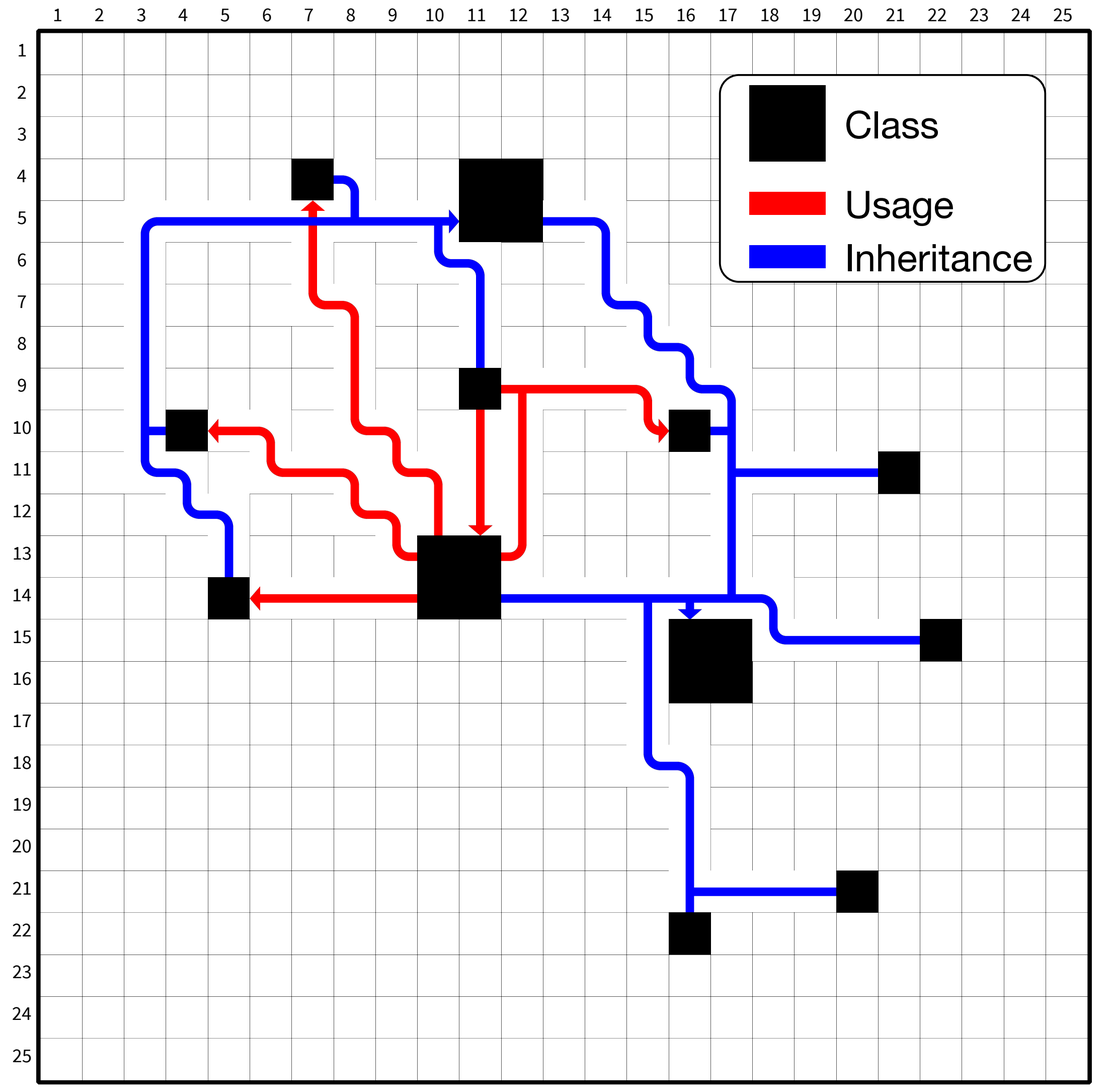
Wassis



Our approach: **Multidimensional Scaling** (MDS)

Our approach: **Wave Function Collapse** (WFC)

Wassis Example

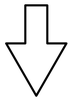


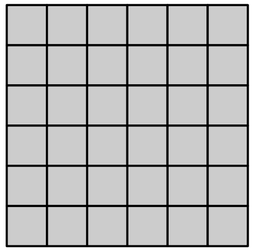
Pipeline

1. Initialization



Parsing





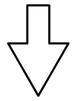
Empty Grid

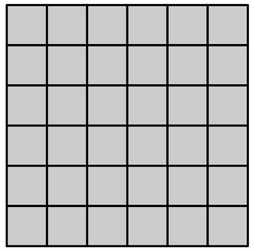
Pipeline

1. Initialization



Parsing



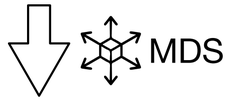


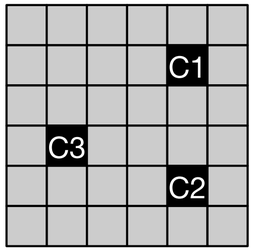
Empty Grid

2. Layout

	C1	C2	C3
C1	0	1	2
C2	1	0	1
C3	2	1	0

Distance Matrix





Class Placement

Pipeline

1. Initialization



Parsing

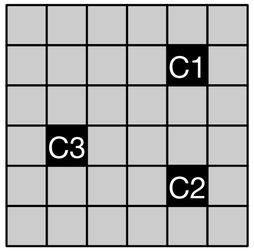
Empty Grid

2. Layout

	C1	C2	C3
C1	0	1	2
C2	1	0	1
C3	2	1	0

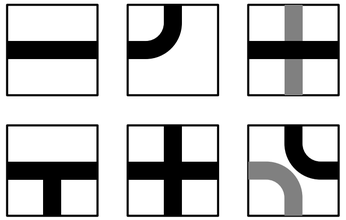
Distance Matrix

MDS

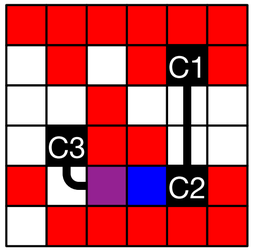


Class Placement

3. Path Generation



Tiles Creation



WFC Execution

Pipeline

1. Initialization



Parsing

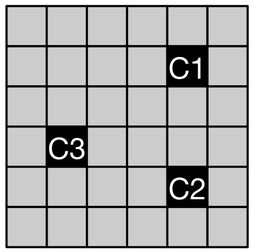
Empty Grid

2. Layout

	C1	C2	C3
C1	0	1	2
C2	1	0	1
C3	2	1	0

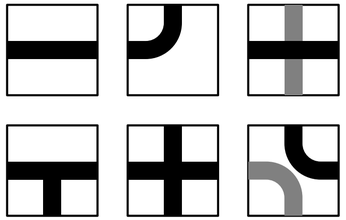
Distance Matrix

MDS

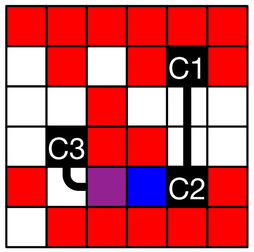


Class Placement

3. Path Generation



Tiles Creation

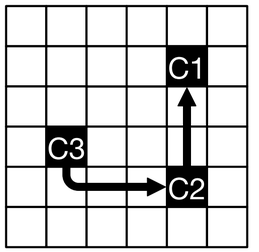


WFC Execution

4. Finalization



Metrics Calculation



Visualization

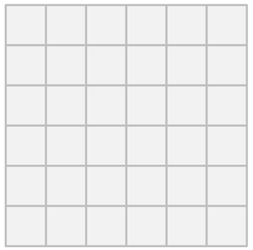
Pipeline

1. Initialization



Parsing



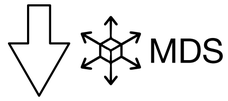


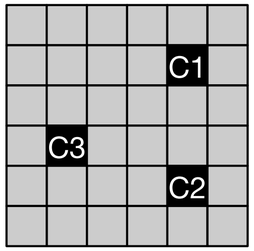
Empty Grid

2. Layout

	C1	C2	C3
C1	0	1	2
C2	1	0	1
C3	2	1	0

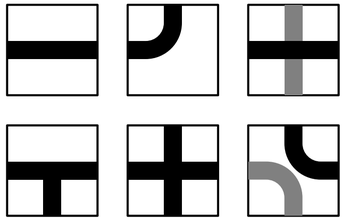
Distance Matrix



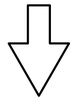


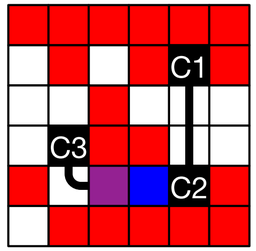
Class Placement

3. Path Generation



Tiles Creation





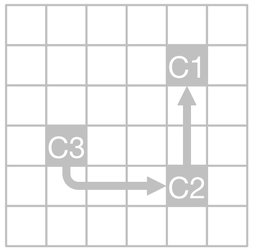
WFC Execution

4. Finalization



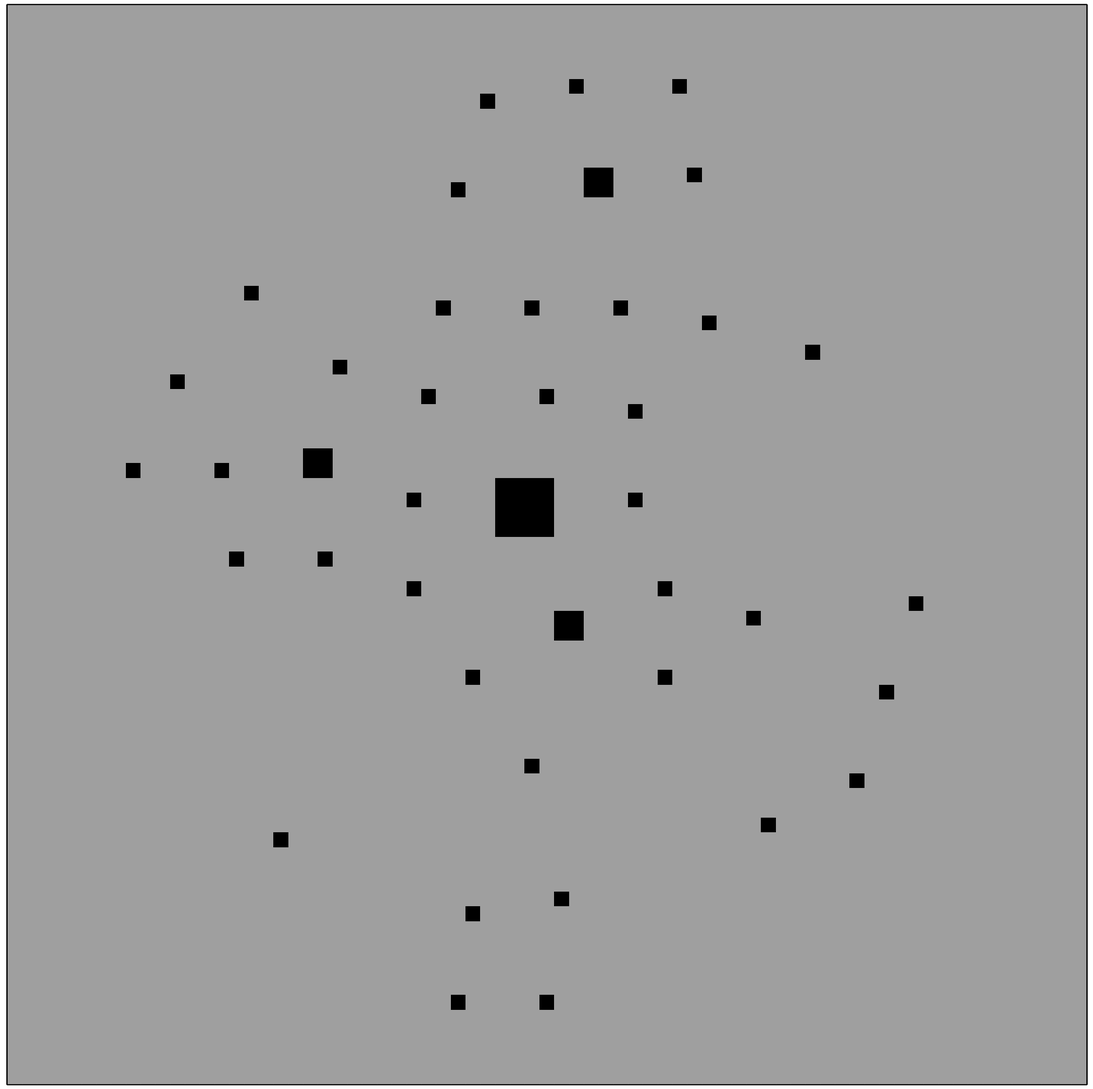
Metrics Calculation





Visualization

Layout



What is Wave Function Collapse?

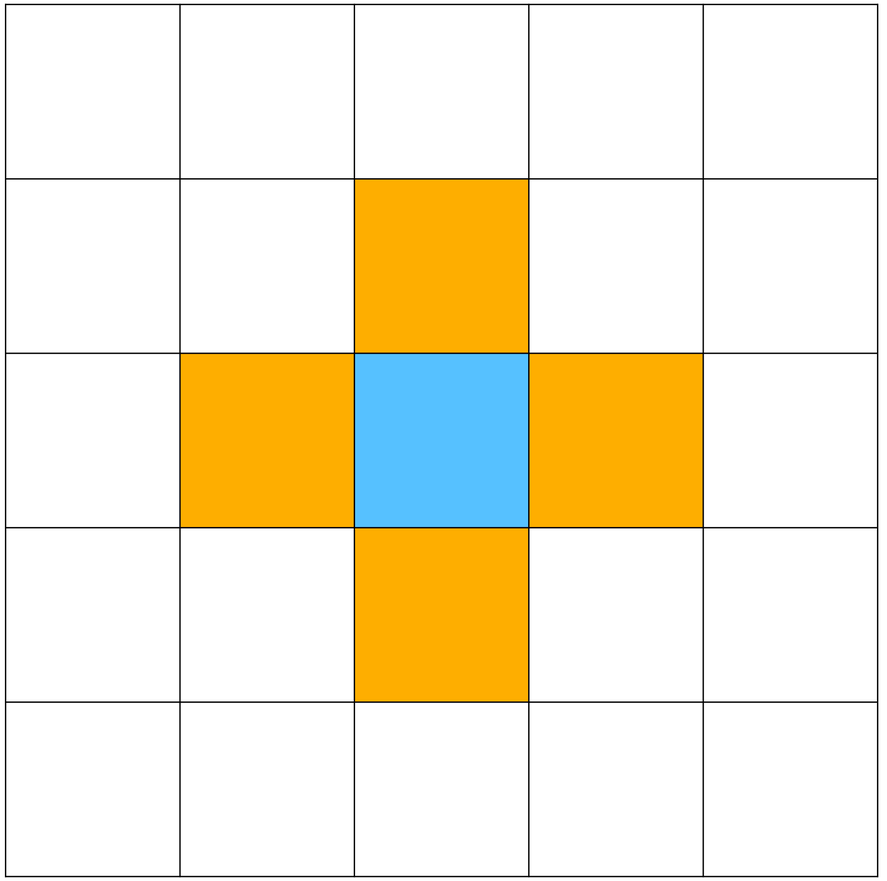
Wave Function Collapse

- PCG algorithm created by Maxim Gumin



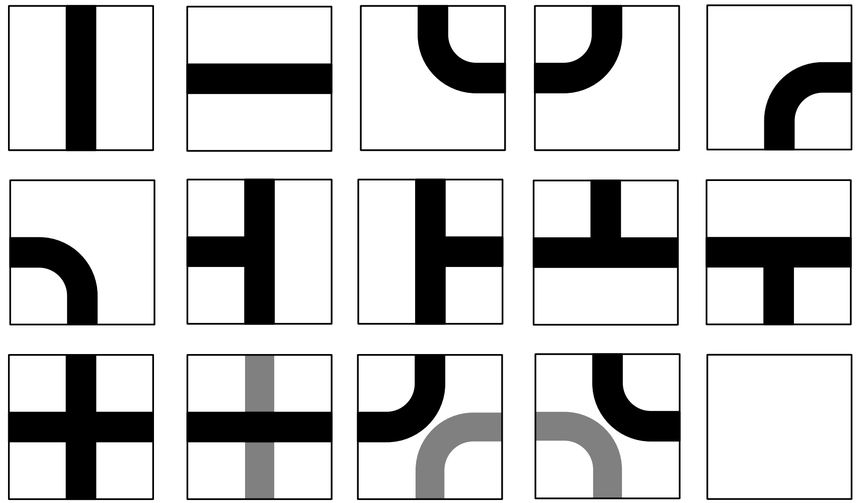
Wave Function Collapse

- PCG algorithm created by Maxim Gumin
- Operates on a well-defined neighborhood structure



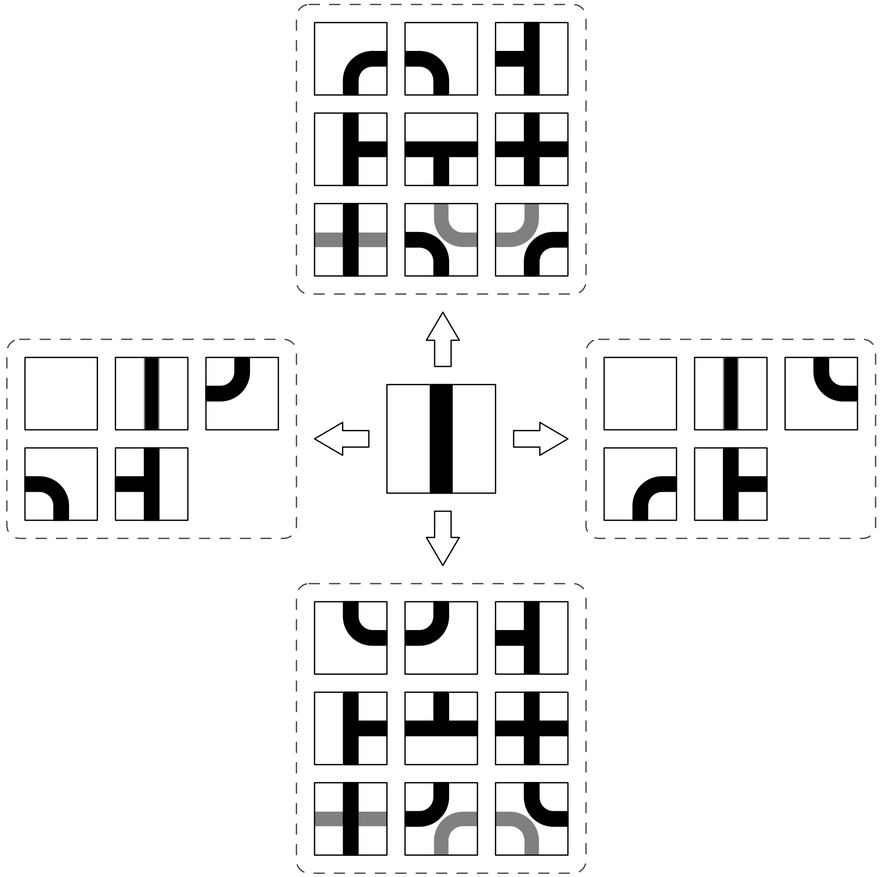
Wave Function Collapse

- PCG algorithm created by Maxim Gumin
- Operates on a well-defined neighborhood structure
- A tileset defines the possible pieces



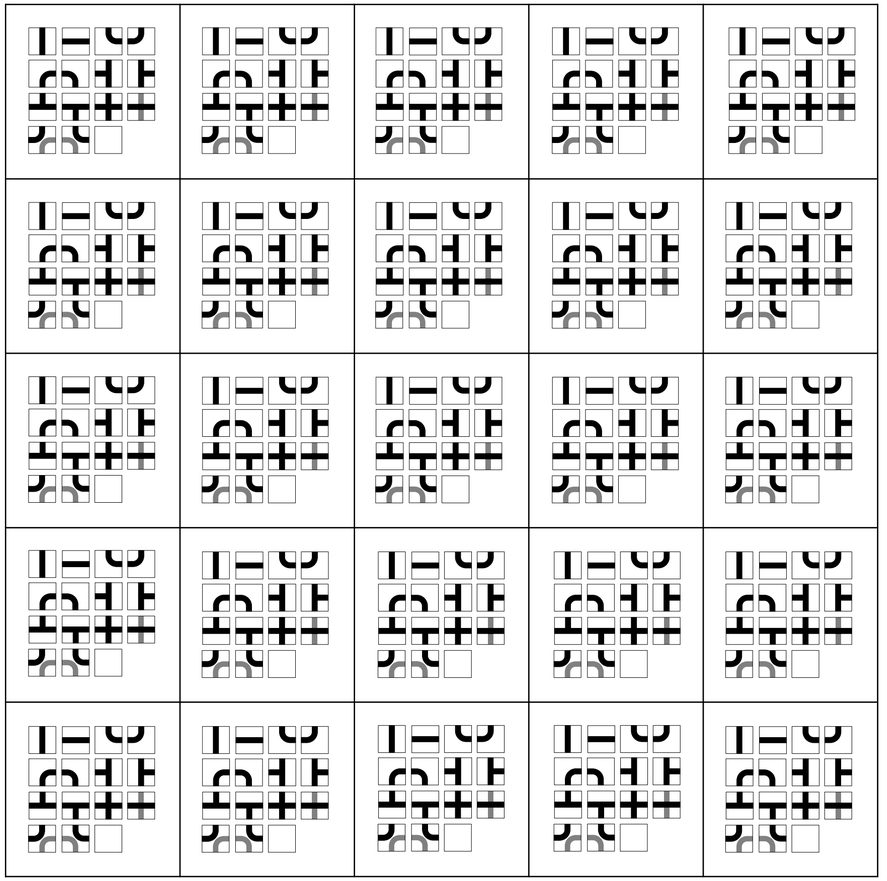
Wave Function Collapse

- PCG algorithm created by Maxim Gumin
- Operates on a well-defined neighborhood structure
- A tileset defines the possible pieces
- Local adjacency rules define compatible neighbors



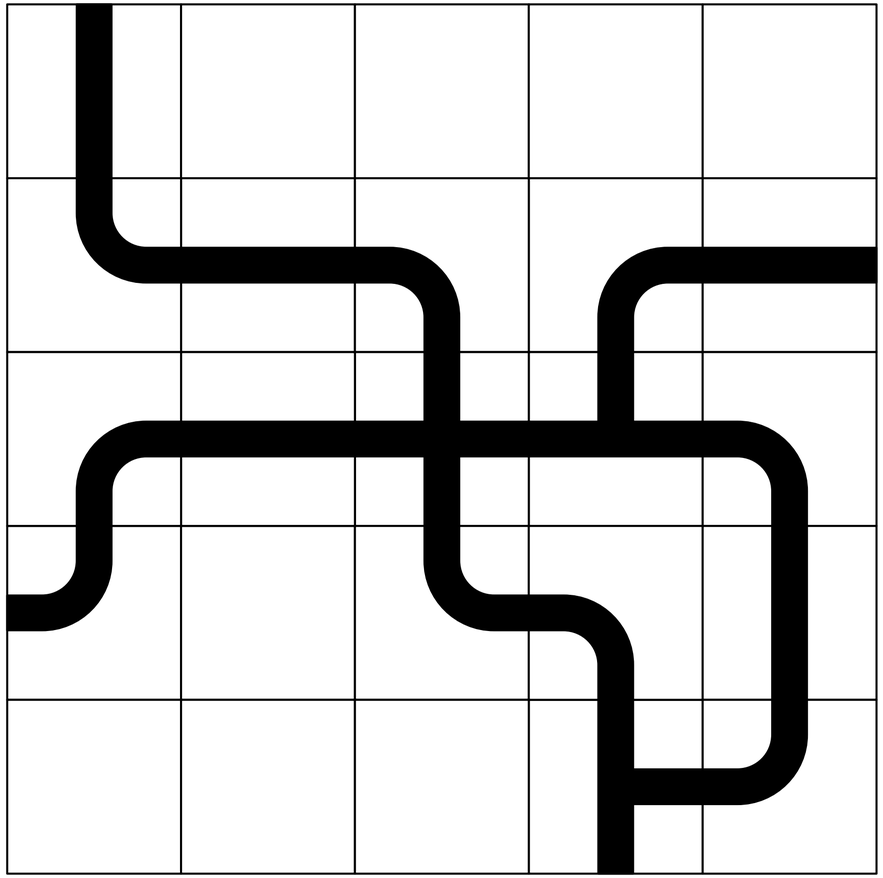
Wave Function Collapse

- PCG algorithm created by Maxim Gumin
- Operates on a well-defined neighborhood structure
- A tileset defines the possible pieces
- Local adjacency rules define compatible neighbors
- Initially, cells are in a superposition state

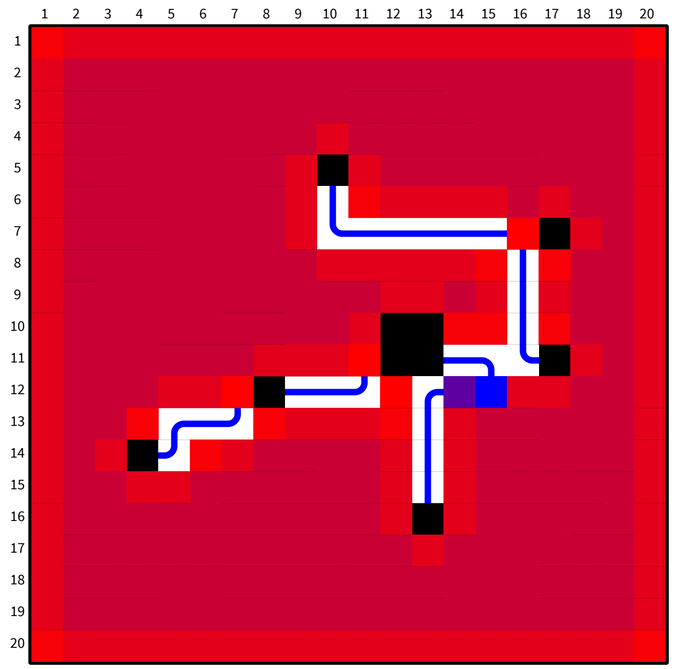


Wave Function Collapse

- PCG algorithm created by Maxim Gumin
- Operates on a well-defined neighborhood structure
- A tileset defines the possible pieces
- Local adjacency rules define compatible neighbors
- Initially, cells are in a superposition state
- Goal: one tile per cell, all local rules satisfied

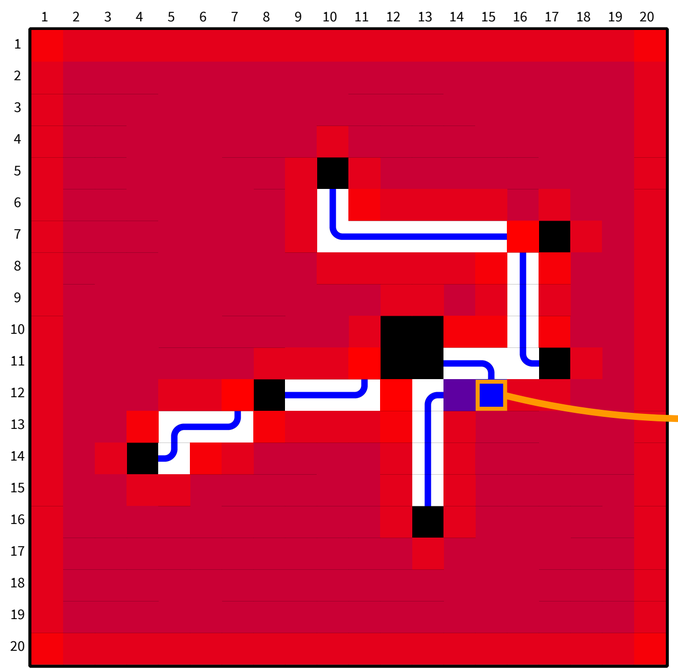


WFC Step



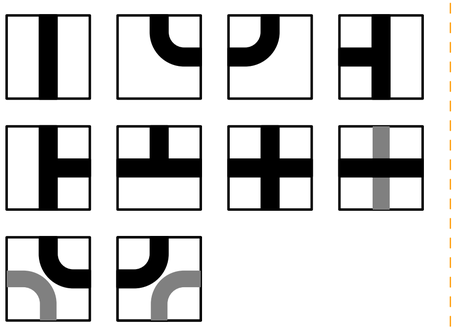
low entropy cell — high entropy cell

WFC Step



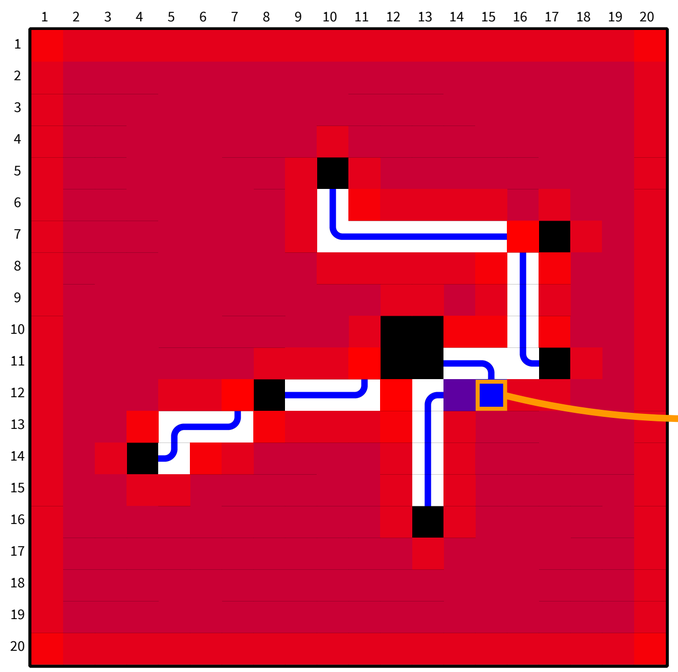
Observation - 1

Possible tiles of cell (12, 15)



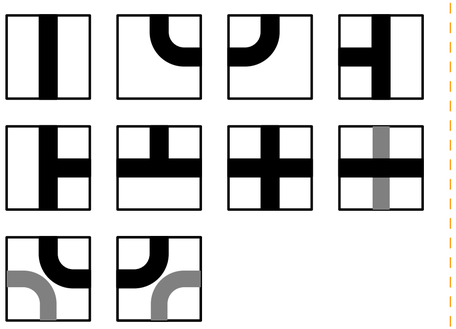


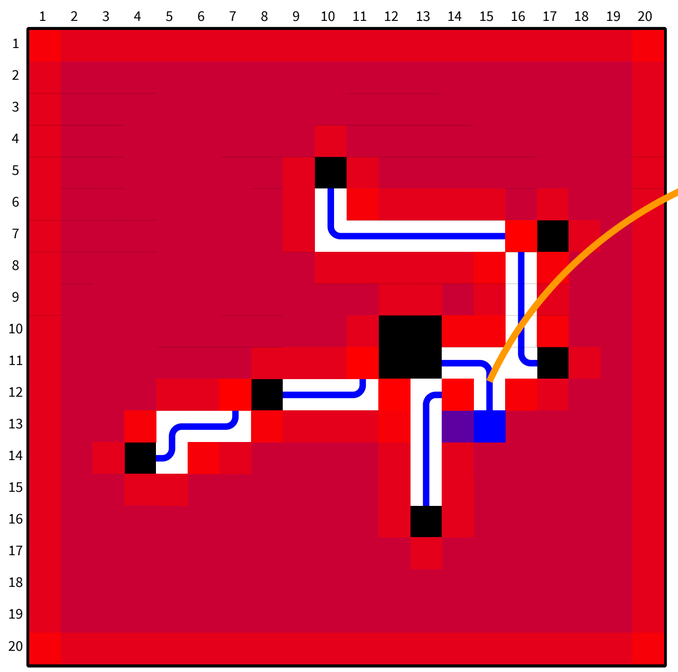
WFC Step



Observation - 1

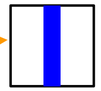
Possible tiles of cell (12, 15)





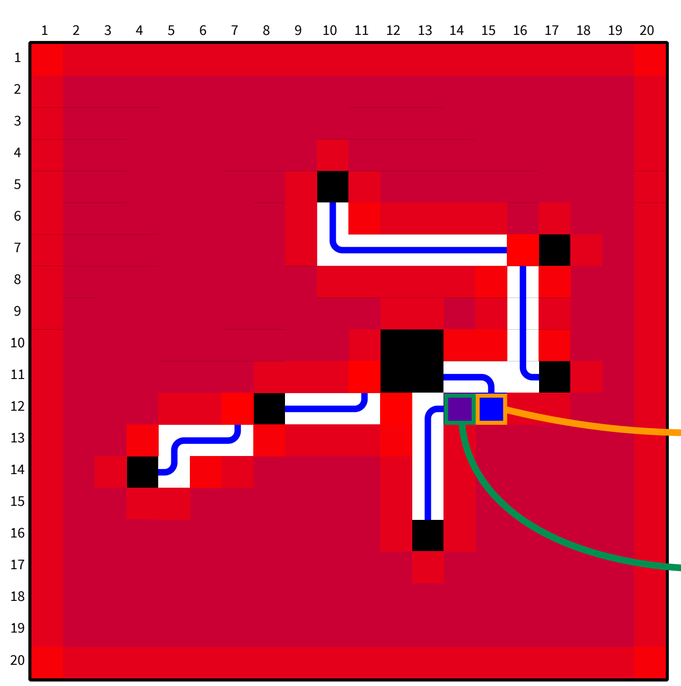
Observation - 2

Cell (12, 15) collapsed



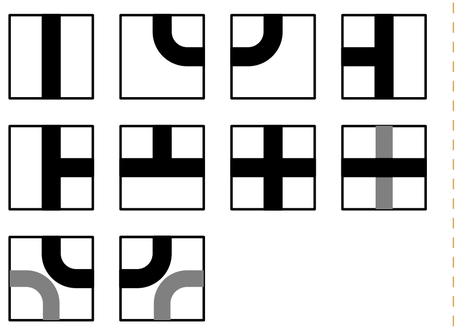
low entropy cell — high entropy cell

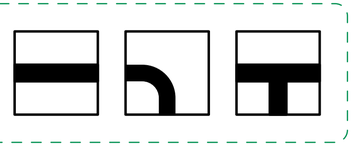
WFC Step



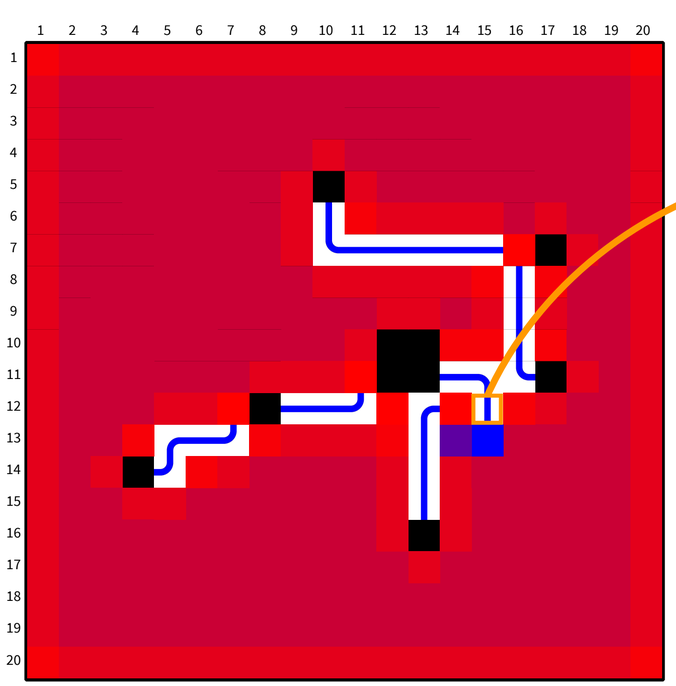
Observation - 1

Possible tiles of cell (12, 15)



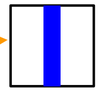


Possible tiles of cell (12, 14)



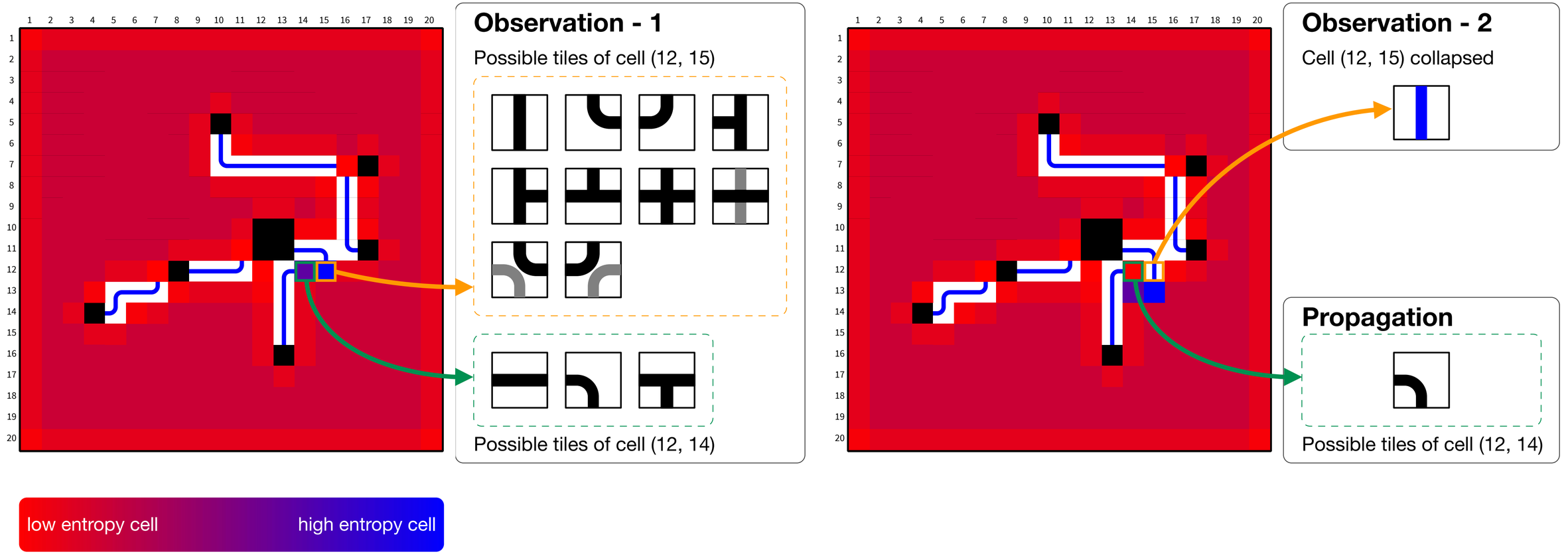
Observation - 2

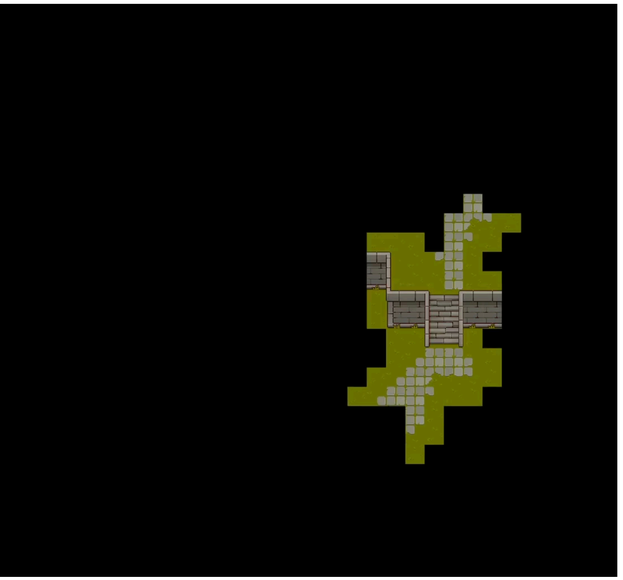
Cell (12, 15) collapsed

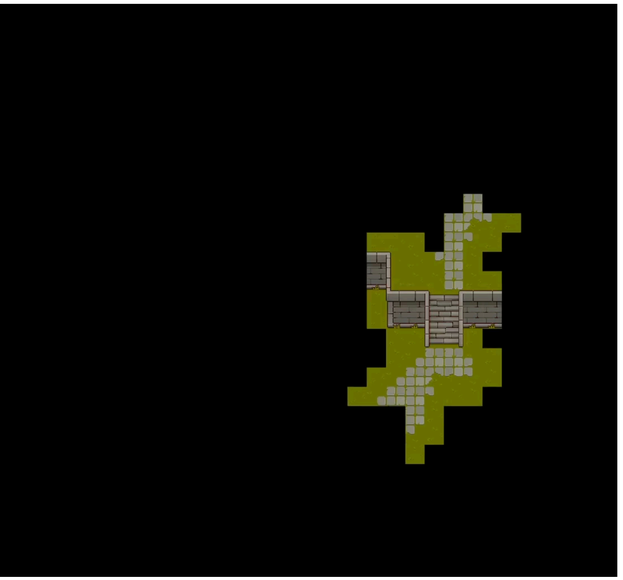


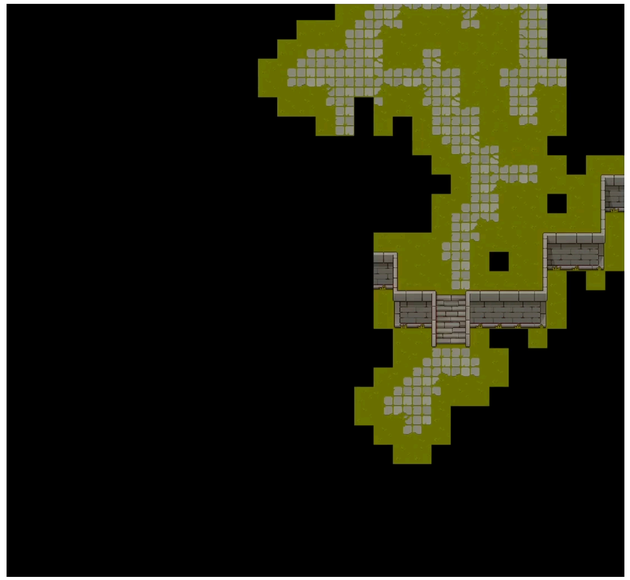
low entropy cell — high entropy cell

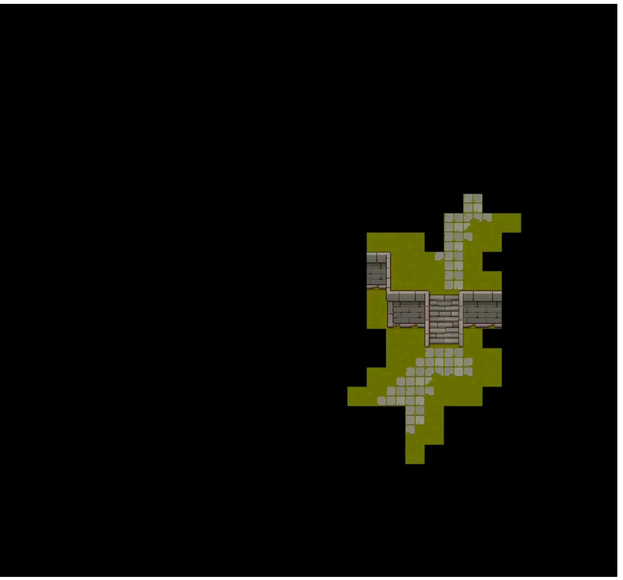
WFC Step

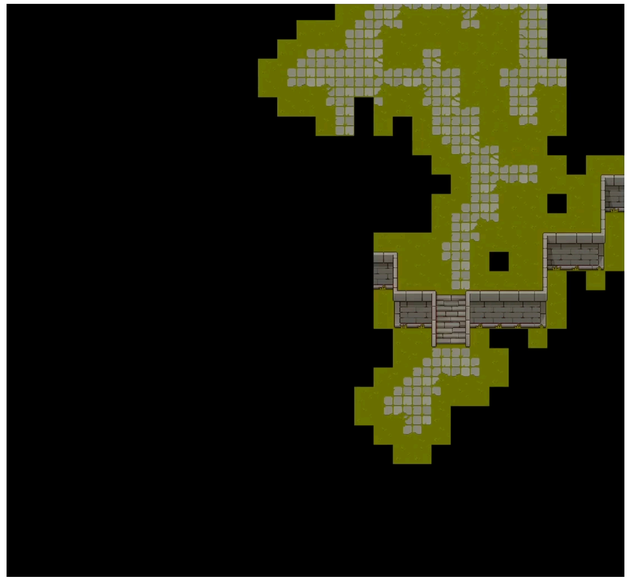

Observation - 1
Possible tiles of cell (12, 15)
Possible tiles of cell (12, 14)
Observation - 2
Cell (12, 15) collapsed
Propagation
Possible tiles of cell (12, 14)
low entropy cell
high entropy cell



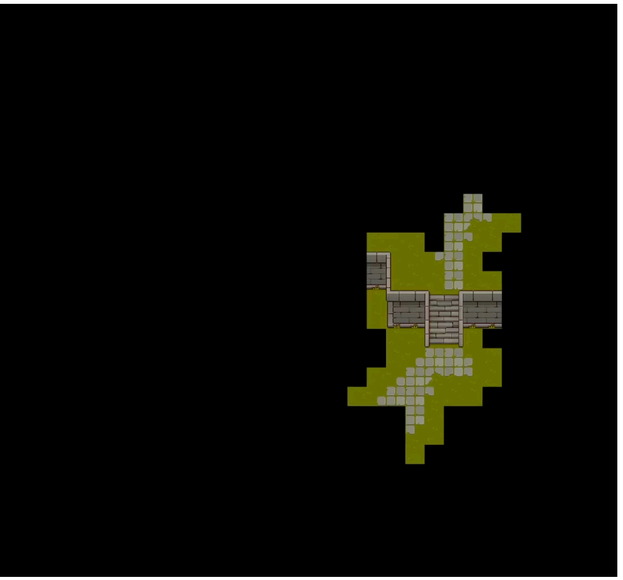
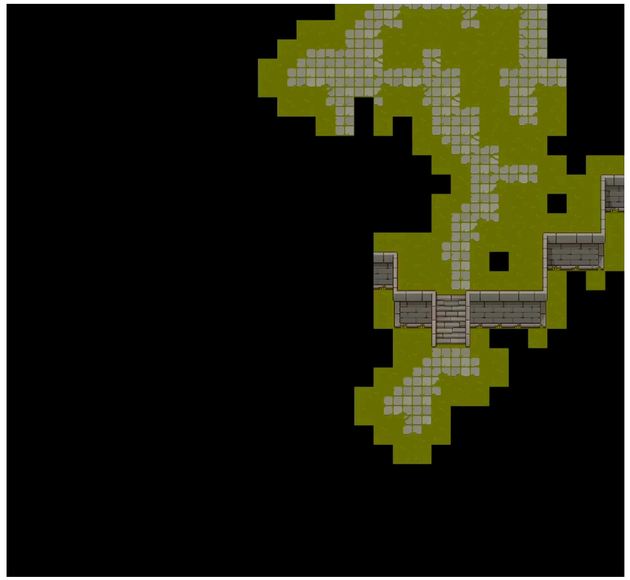




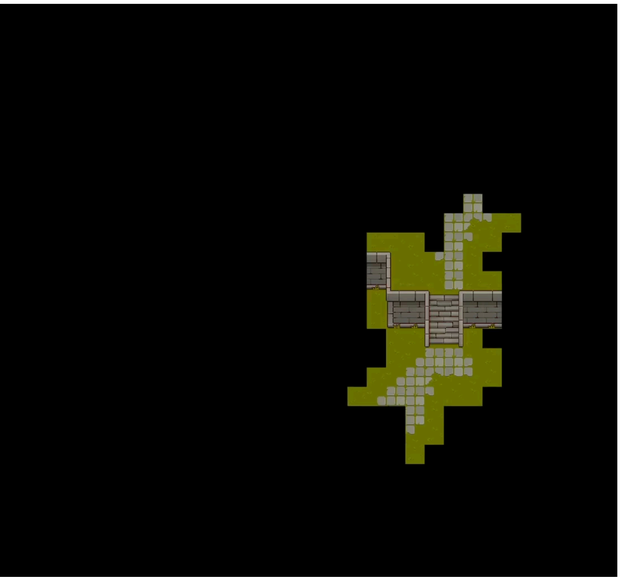
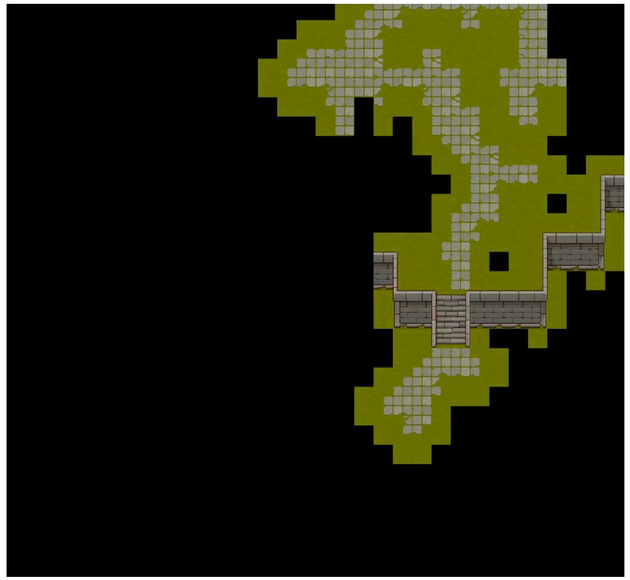


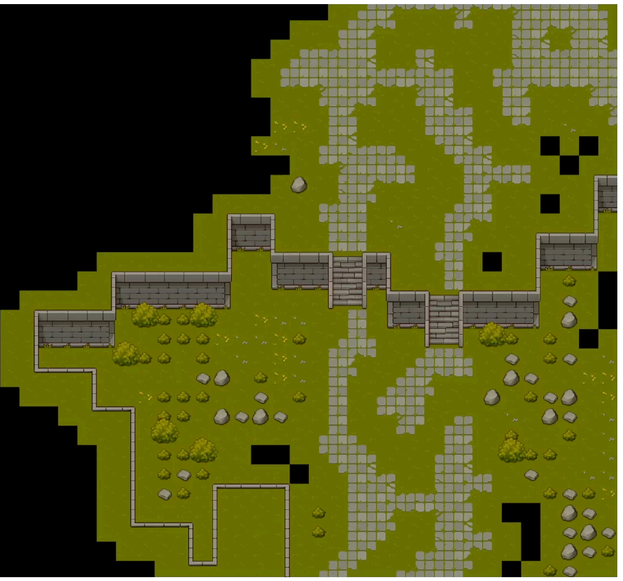


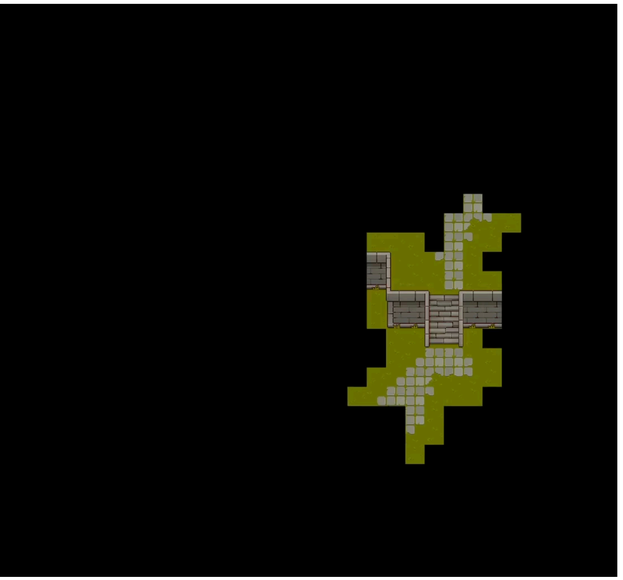
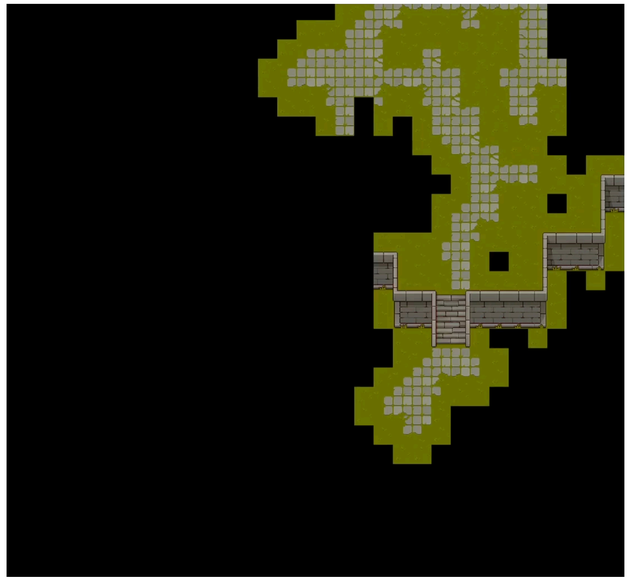


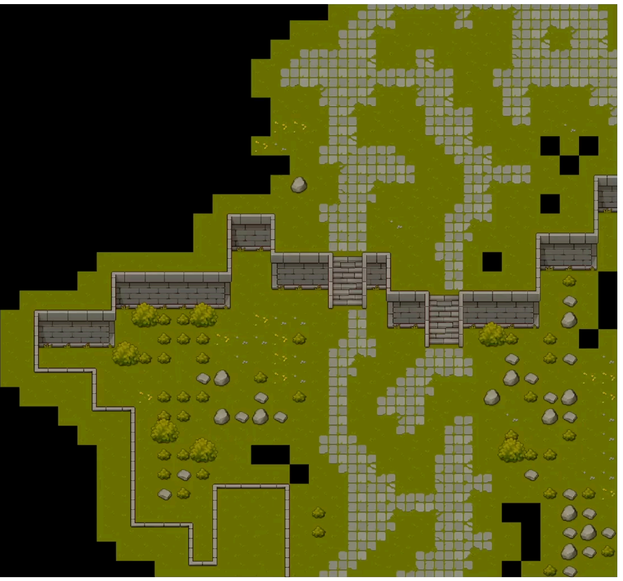
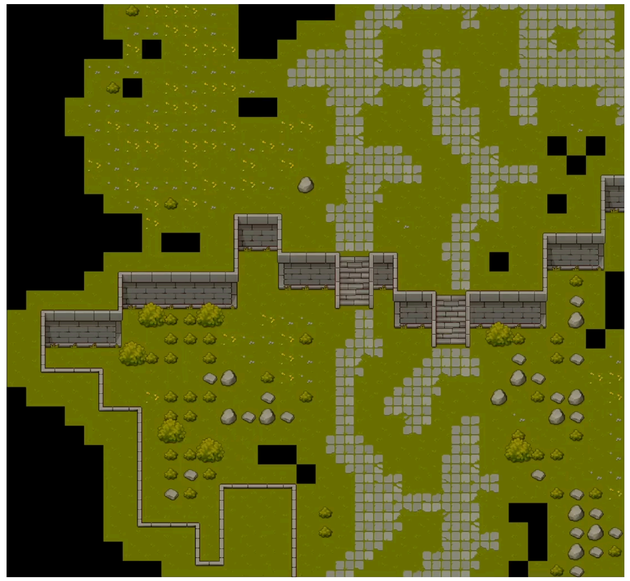



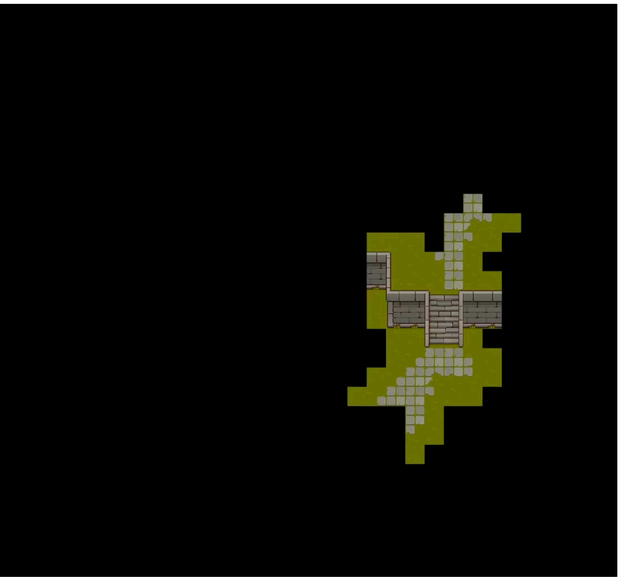
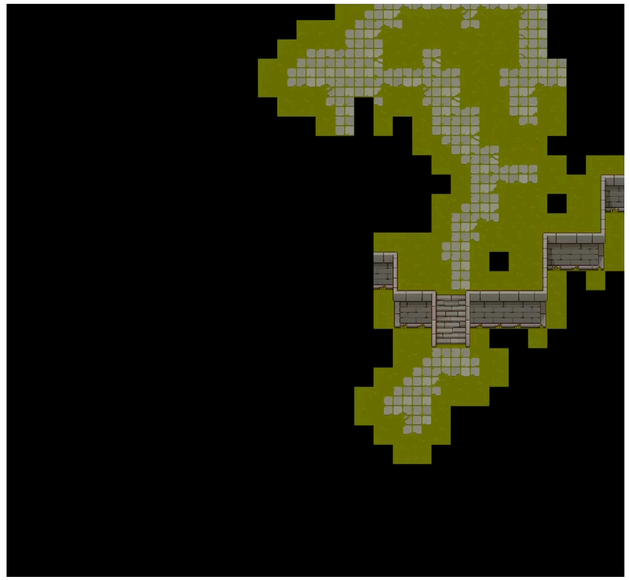


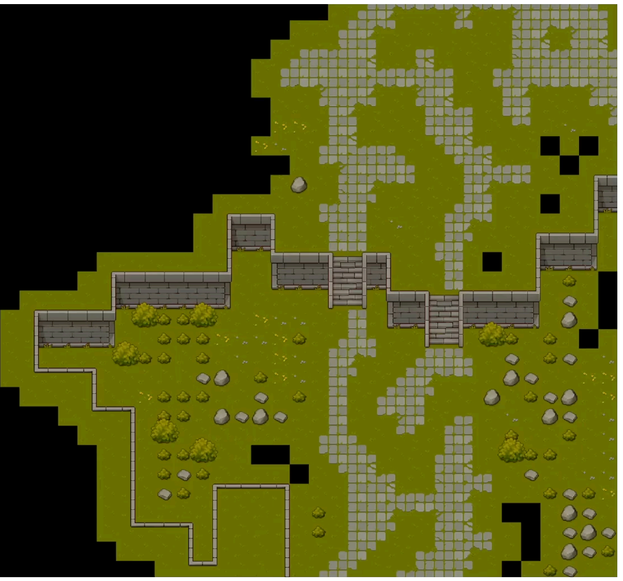
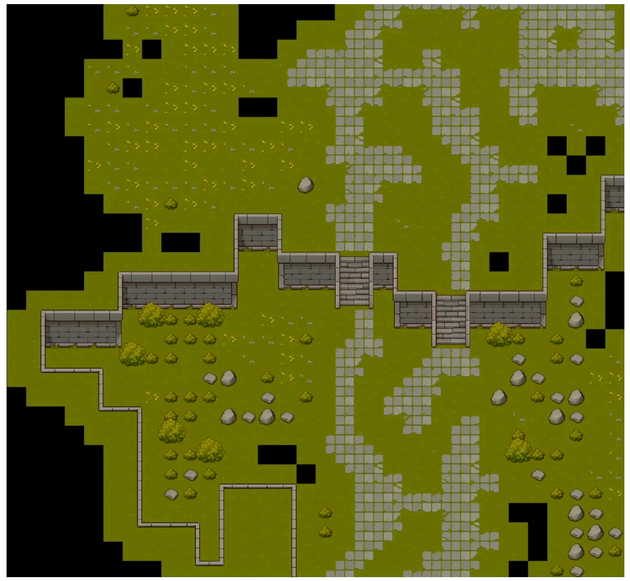



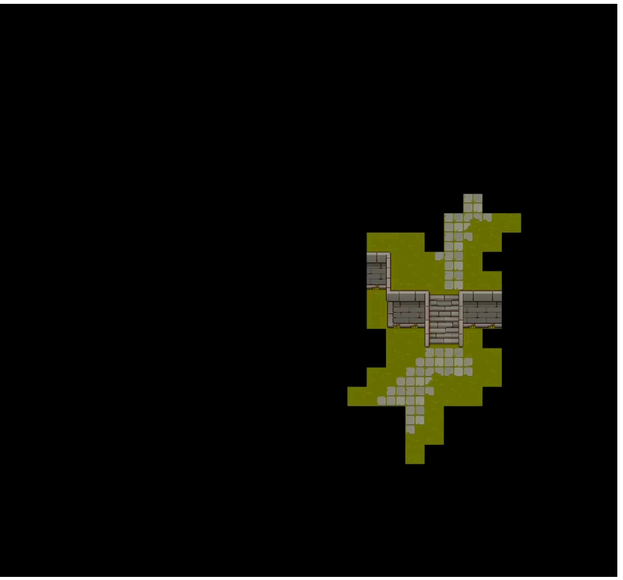
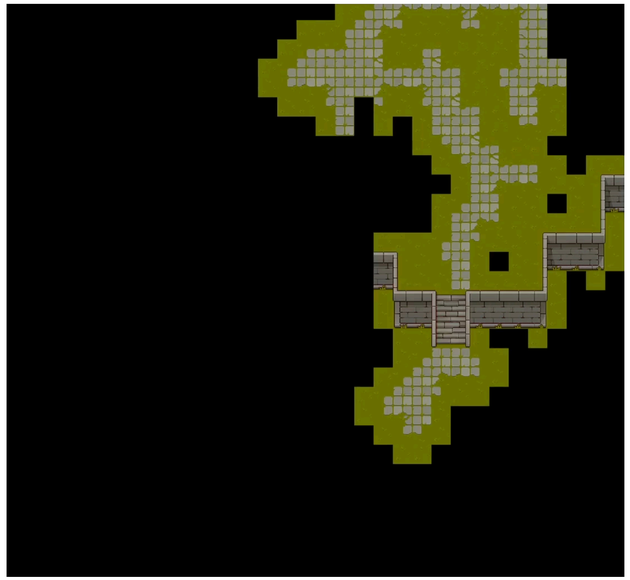



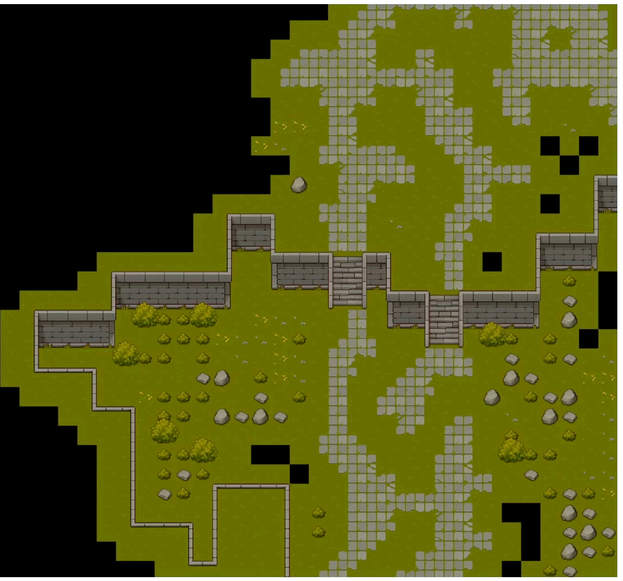
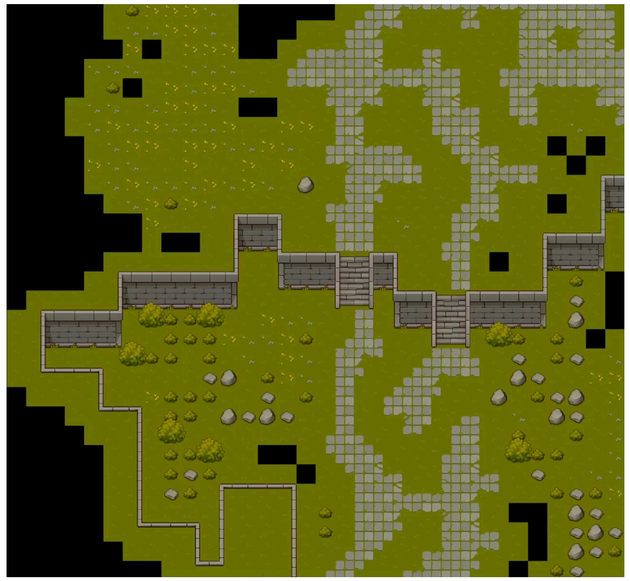



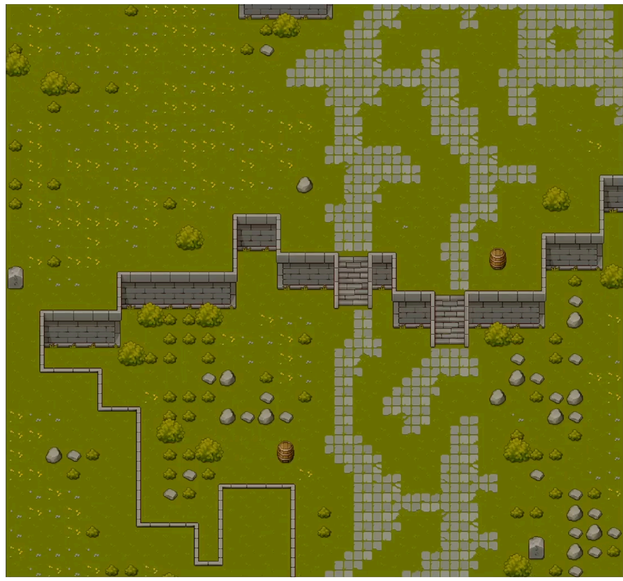






Toward Using Wave Function Collapse to Depict Software Systems

Simone Lesinigo, Marco Raglianti, Michele Lanza
REVEAL @ Software Institute — USI, Lugano, Switzerland

***Abstract*—Grasping the overall architecture of large software systems is a challenging task. A plethora of software visualization approaches has been proposed to ease comprehension of the relationships among components and classes in such systems. A core problem is deciding how to place elements and how to connect them meaningfully, avoiding excessive overlaps and intersections that make visualizations cluttered and unintelligible.**

We present an approach based on Wave Function Collapse (WFC), a procedural generation algorithm inspired by quantum mechanics, which works on a grid of locally constrained cells, to connect (software) entities while discouraging overlaps and preserving the connectivity imposed by global constraints. We show how WFC can be tuned to reduce the amount of path crossings and its scalability challenges towards representing real-world software systems. We present two key examples (single- and multi-relationship) with WASSIS, our prototype implementation of wave function collapse, illustrating the characteristics that produce early-stage, yet promising, "organic" visualizations.

***Index Terms*—wave function collapse, procedural content generation, system comprehension, software visualization, multidimensional scaling.**

I. INTRODUCTION

Program comprehension is as old as software engineering itself. As code evolves and grows in complexity, developers struggle to build and maintain a mental model of the system, which increases the difficulty of tasks such as debugging and refactoring [1]–[3]. To deal with this problem, visual modeling languages (*e.g.,* UML [4]) and software visualizations [5]–[7] have been widely researched to tame complexity and provide intelligible views depicting classes and their relationships.

Two problems arise when visualizing large software systems. The first is the **classic layout** problem: Given a set of entities one must decide how to place them on a canvas. Depending on what needs to be conveyed, classes can be grouped according to semantic or structural similarity as, for example, inheritance hierarchies, by package containment, proximity by coupling or usage relationships. The second is the **connectivity** problem: Displaying relationships between entities can enrich the visualization with crucial information, but systems can be huge in terms of number of entities, and even more so in terms of connections between those entities.

We explore a novel approach to tackle these two problems. We use MultiDimensional Scaling (MDS) [8], [9], a dimensionality reduction technique to lay entities out on the canvas in a way that reflects their relationships. We then leverage Wave Function Collapse (WFC) [10] to solve the connectivity problem by generating paths between the entities, according to their relationships, minimizing the overlaps.

WFC produces coherent tile-based structures by propagating *local constraints* across a *grid* of *cells*. Since constraints are defined at the *tile* level (*i.e.,* the possible elements in which each cell can be collapsed), WFC allows fine-grained control over which connections are generated. This allows us to minimize visual clutter, such as overlaps, by restricting the set of possible tiles and adjusting their probabilities to appear (*i.e.,* be collapsed to that specific tile, such as a crossing) in the final solution. WFC is intrinsically and heavily non-deterministic, generating potentially very different layouts for the same system. Discarding the invalid ones and selecting the "best" ones gives developers the opportunity to explore different spatial perspectives. However, dependency relationships in software systems are inherently global constraints, while WFC operates locally. We devised our approach to generate valid and consistent paths (also good but not necessarily optimal without a cleanup step), as exemplified in Figure 1, and ensure that the generated connections reflect the actual relations in the system. Yet, the initial placement of the entities remains one of the main challenges that most influences the final outcome.

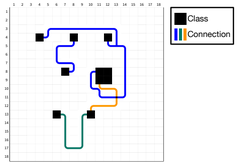


Fig. 1. WFC-pure, 7 classes, 3 relationship types, no path optimization.

II. RELATED WORK

Wave Function Collapse was introduced by Gumin in 2016 [10] as a local constraint solver for procedural content generation. Originally designed for image synthesis, since its release, it has been widely adopted across different domains. Stålberg used it for game map generation in Bad North [11] and Townscaper [12]. Kleineberg [13] used it to procedurally generate an infinite city. O'Leary [14] created a poetry generator that enforces rhyme and meter constraints by treating syllables as tiles. Kim *et al.* [15] used WFC to generate Strimko and Flow puzzles, building on their earlier graph-based WFC variant for Sudoku and the four-color problem on Voronoi diagrams [16].

Title	Toward Using Wave Function Collapse To Depict Software Systems
Authors	Simone Lesinigo, Marco Raglianti, Michele Lanza
Conference	14th Working Conference On Software Visualization
Publisher	IEEE
Year	2026
Pages	111-115

VISSOFT 2026

Core Adaptations



Core Adaptations

Global Constraints

Path Merging

Core Adaptations

Global Constraints

Path Merging

Dynamic Placeholder

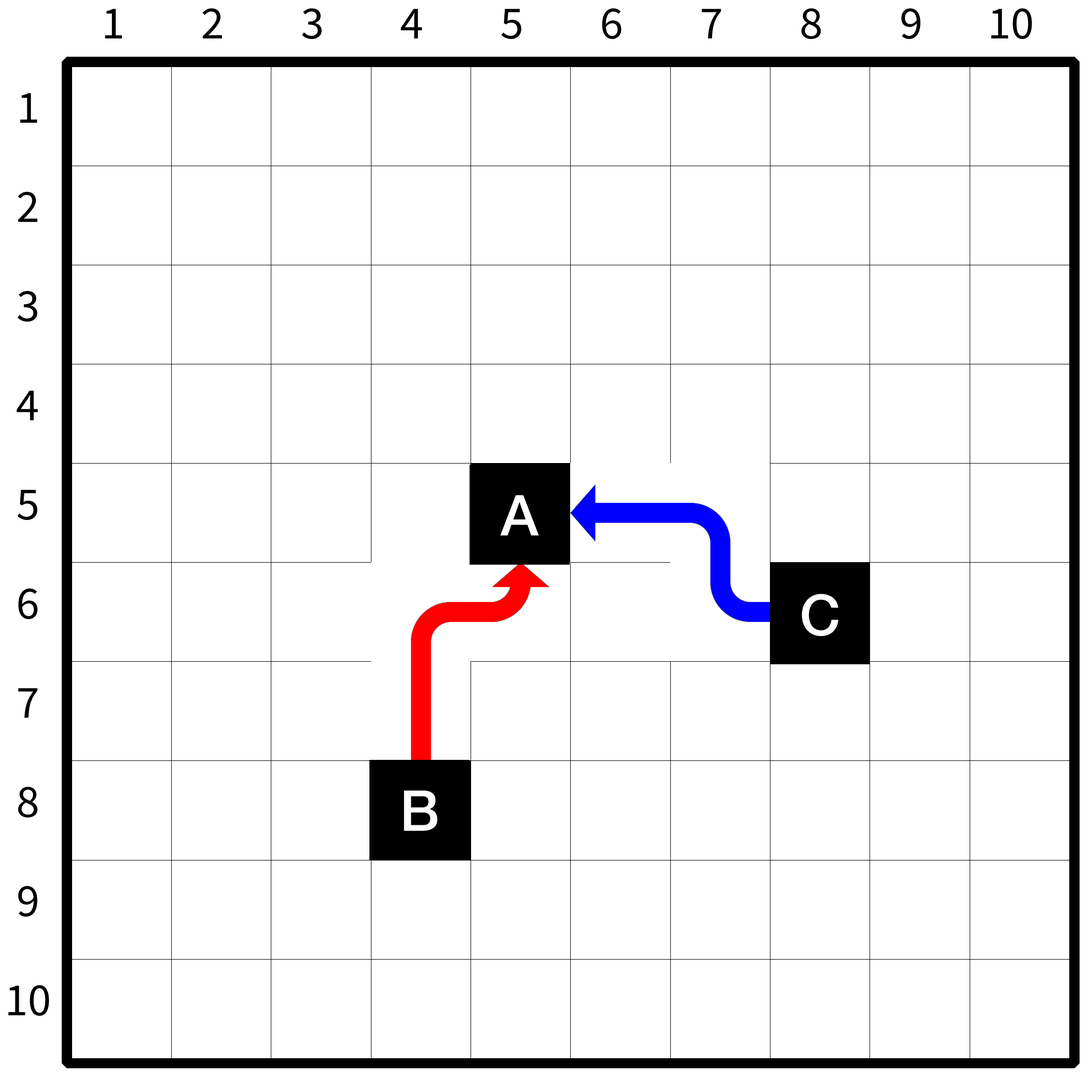
Core Adaptations

- Global Constraints
- Path Merging
- Dynamic Placeholder
- Dynamic Tile Weights

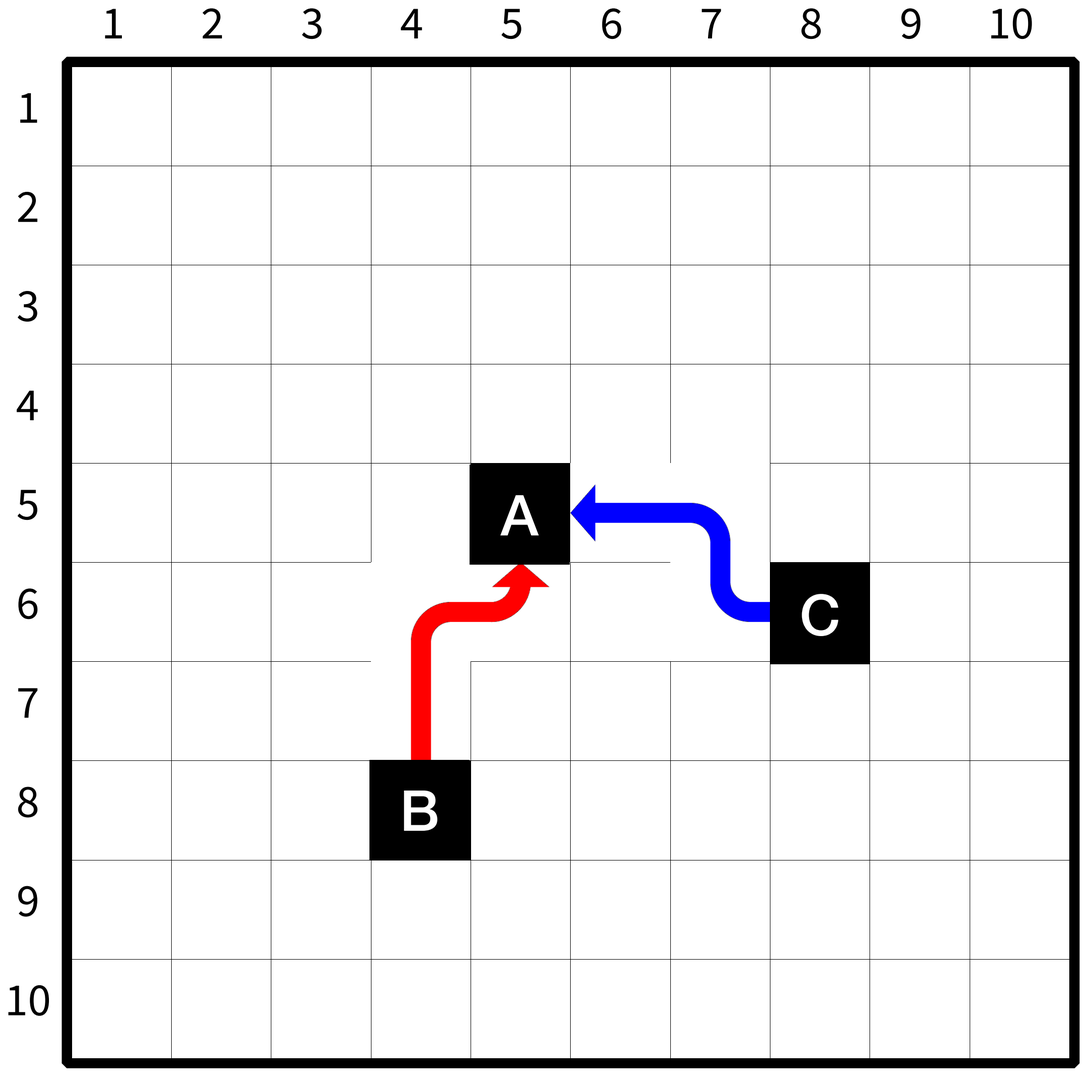
Core Adaptations

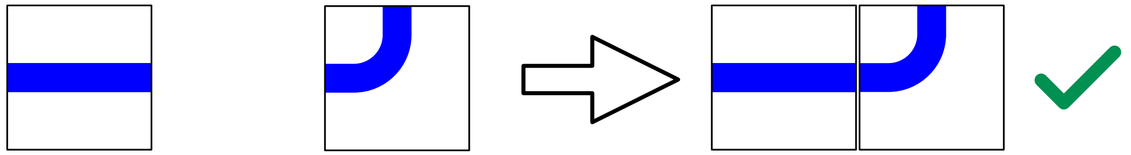
Global Constraints

Global Constraints

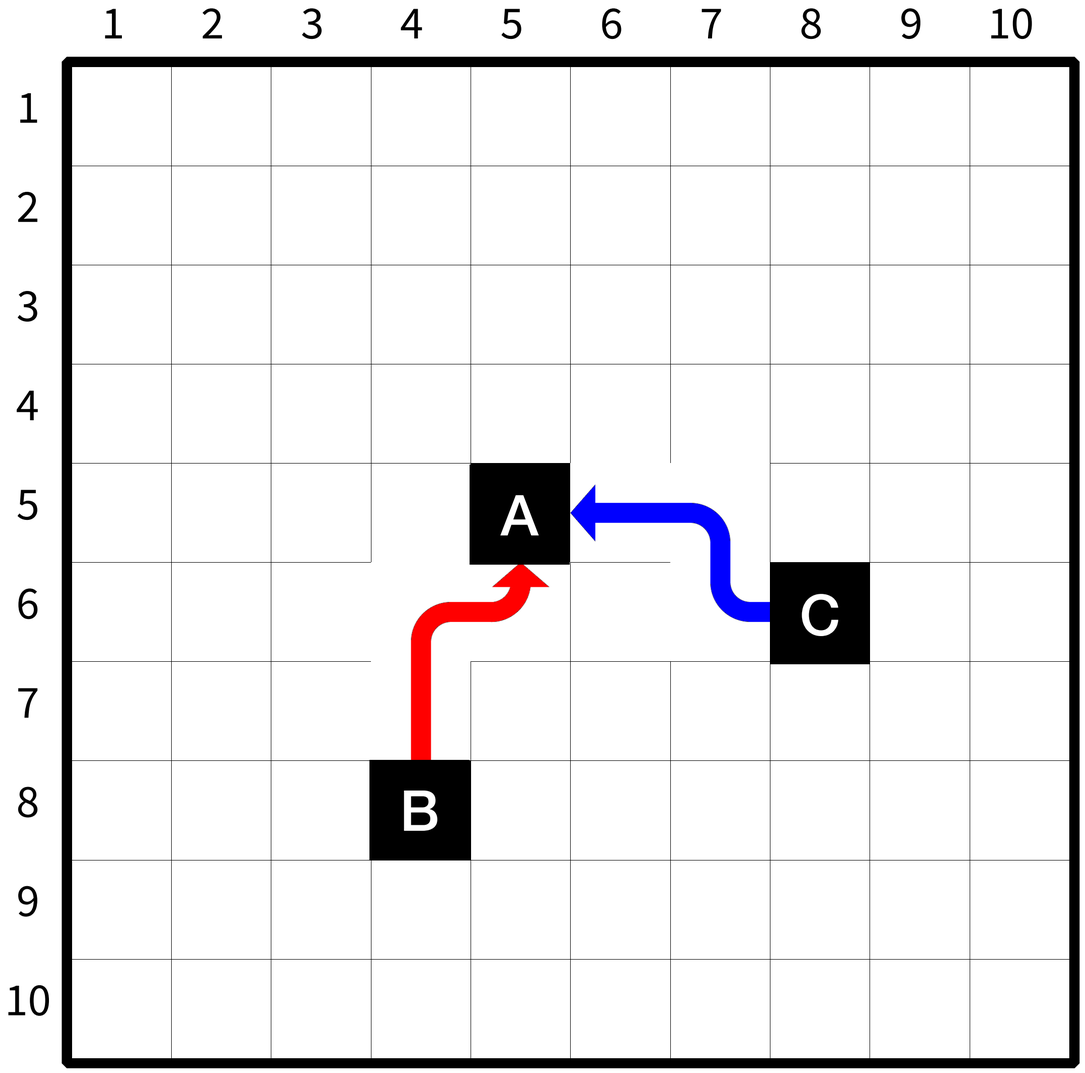


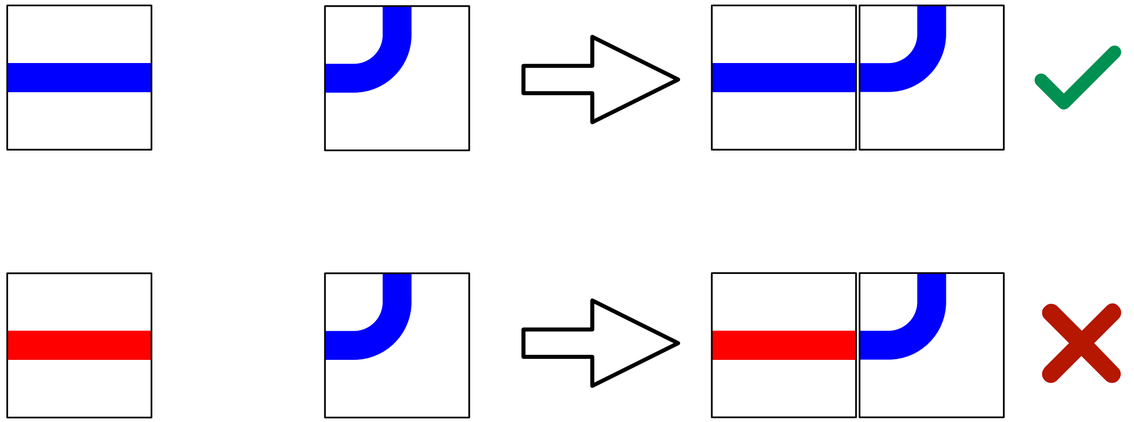
Global Constraints



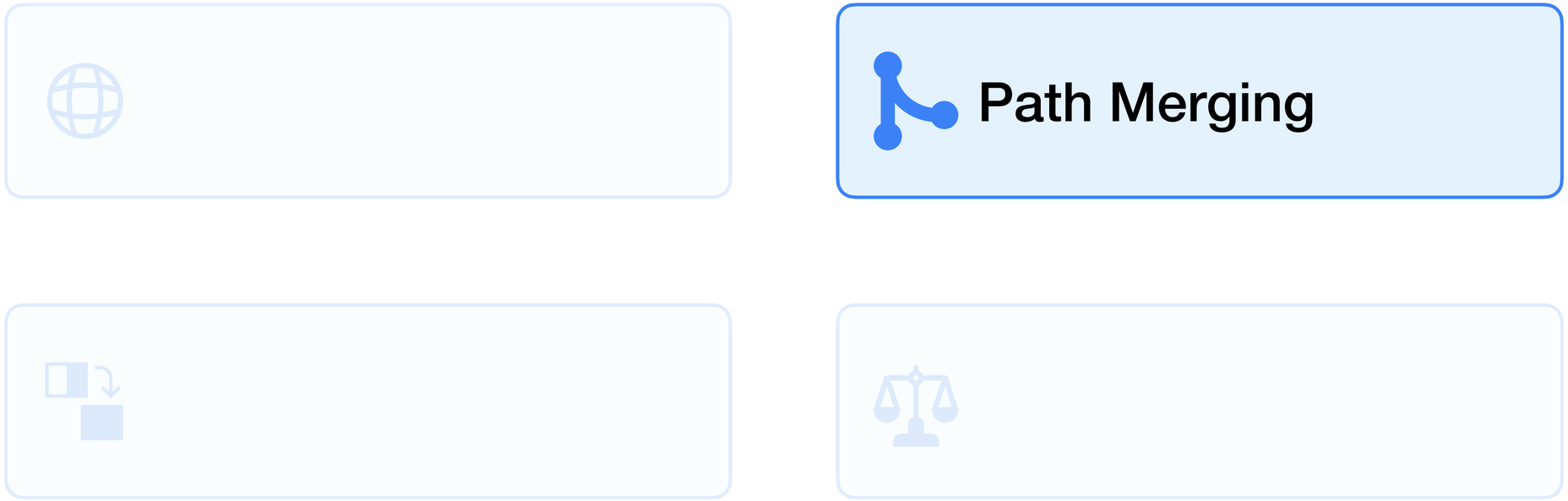


Global Constraints

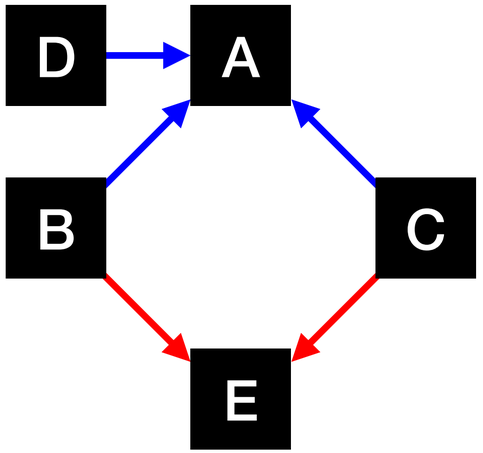




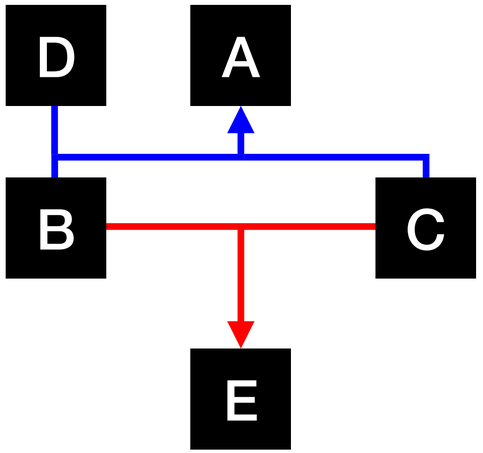
Core Adaptations



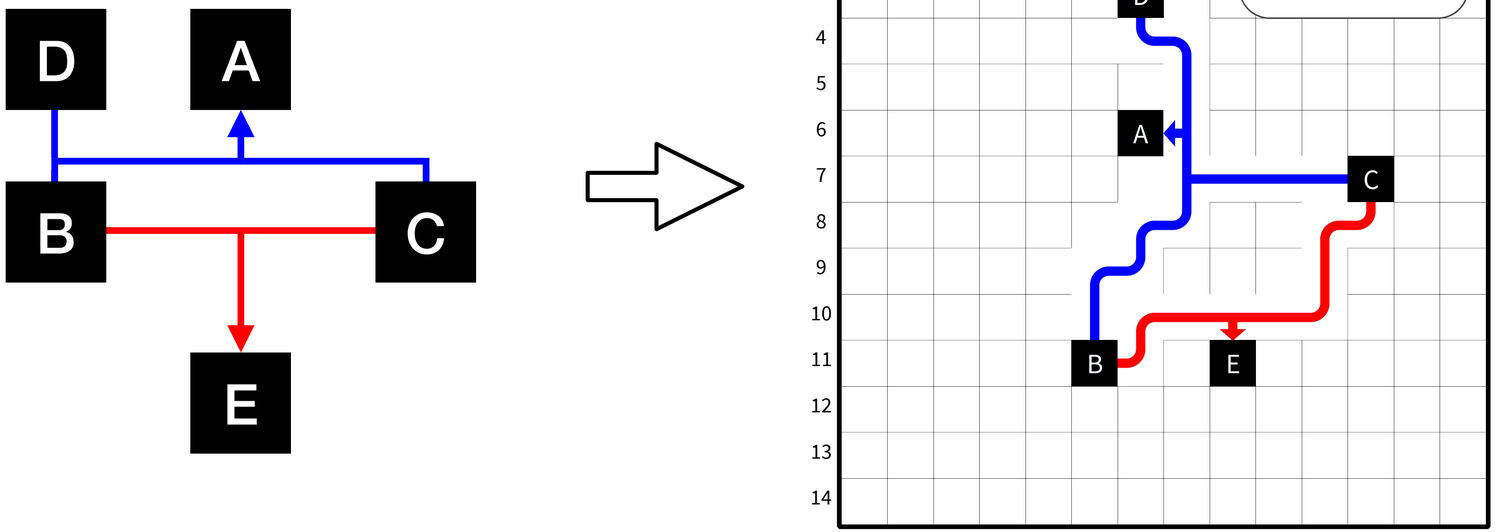
Path Merging



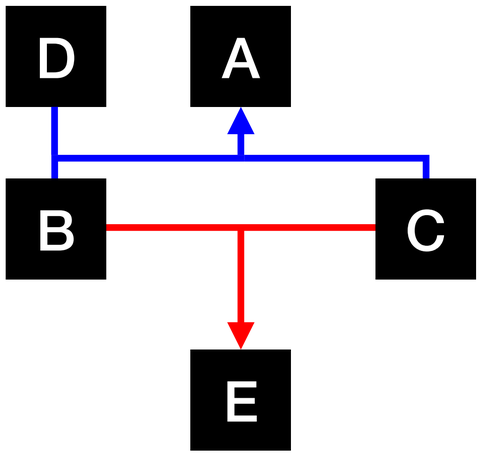
Path Merging

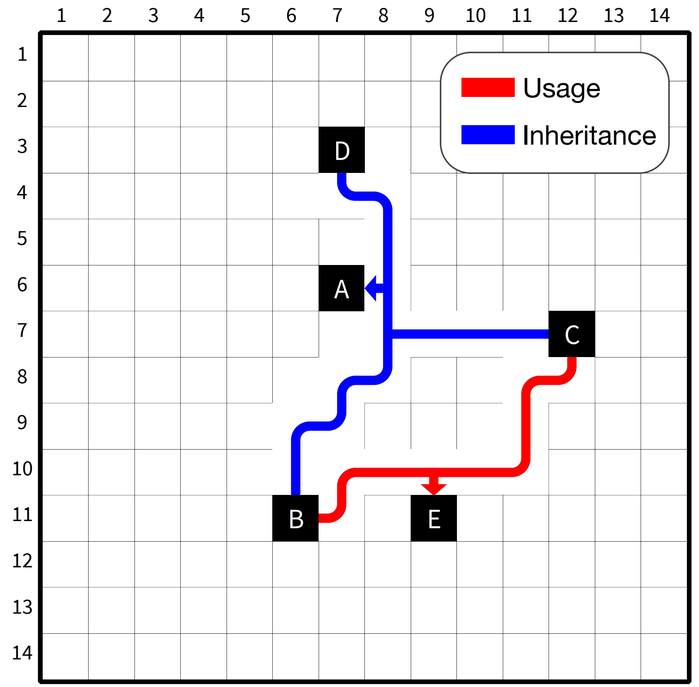


Path Merging



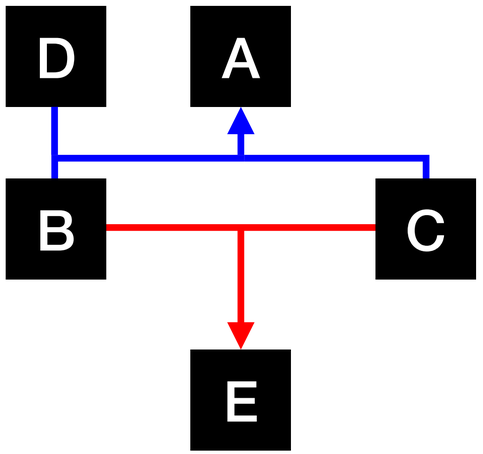
Path Merging



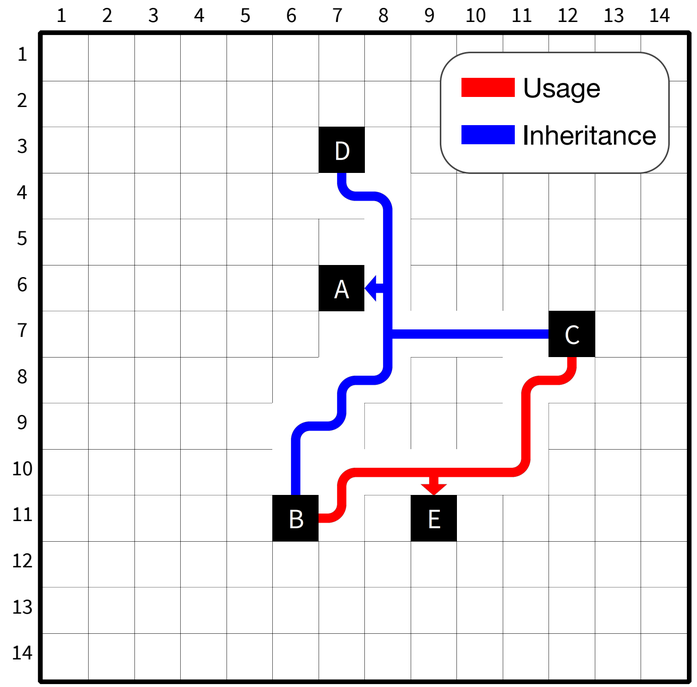


- Reduces Visual Clutter

Path Merging

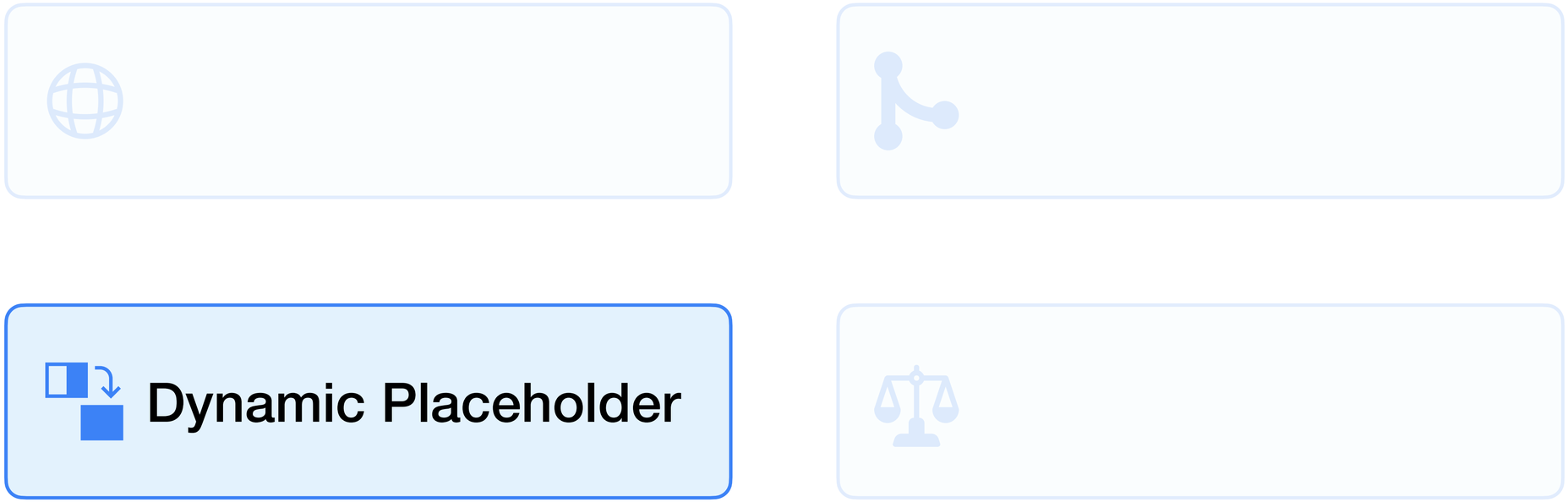




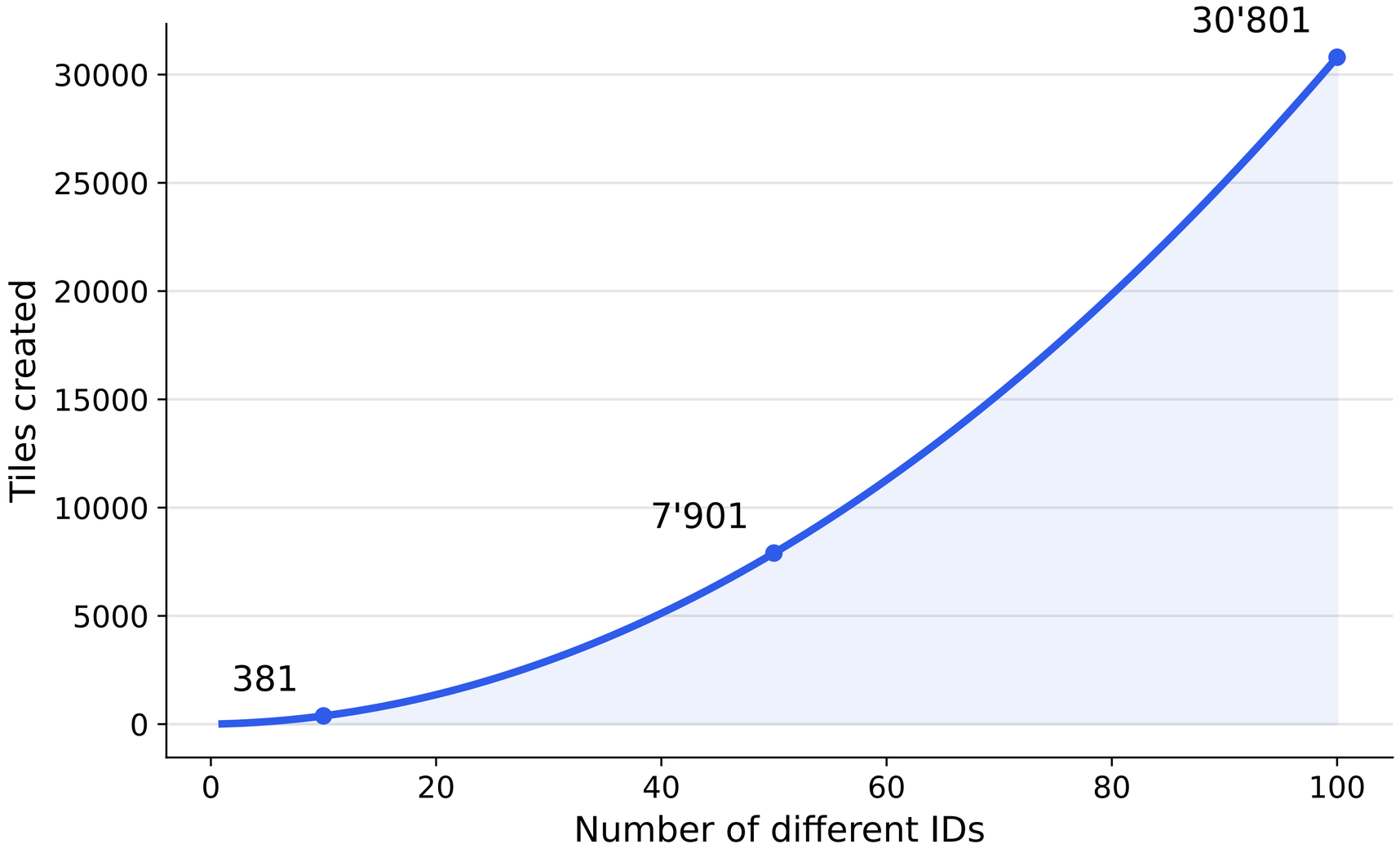


- Reduces Visual Clutter
- Facilitates solution finding

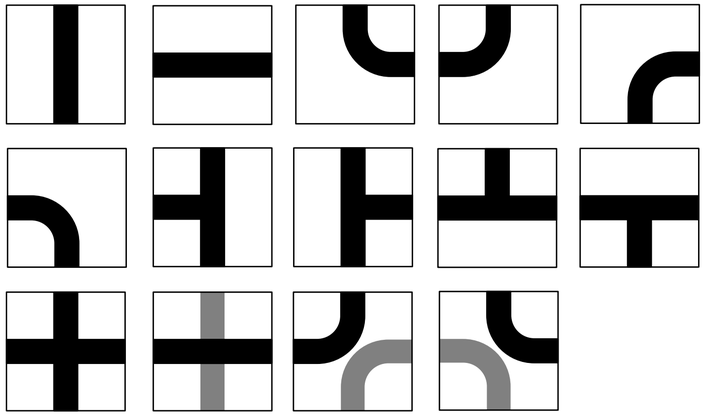
Core Adaptations



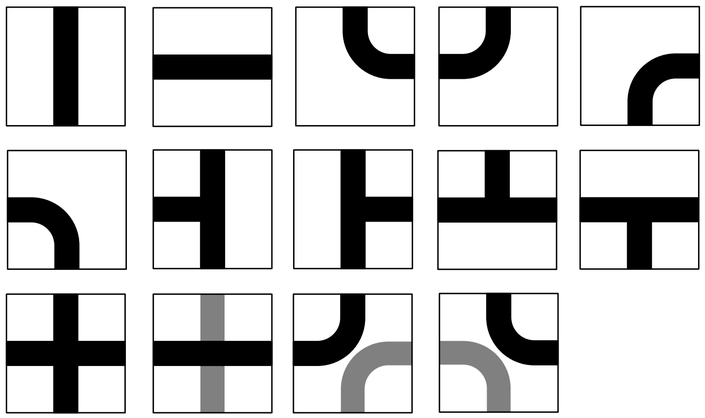
Exploding Number of Tiles

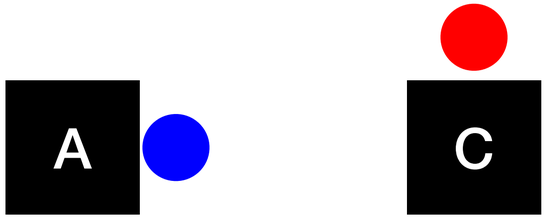


Dynamic Placeholder

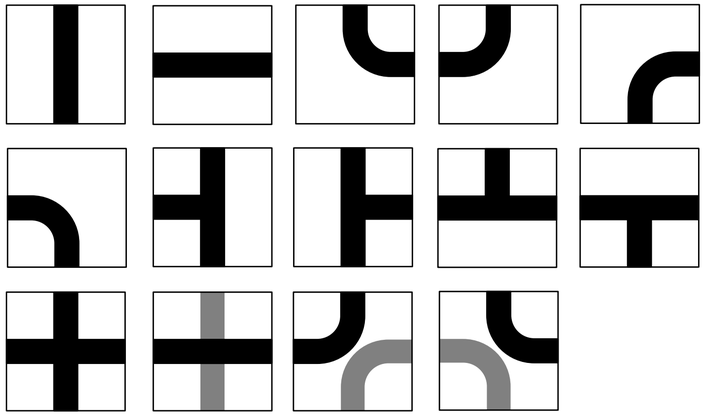


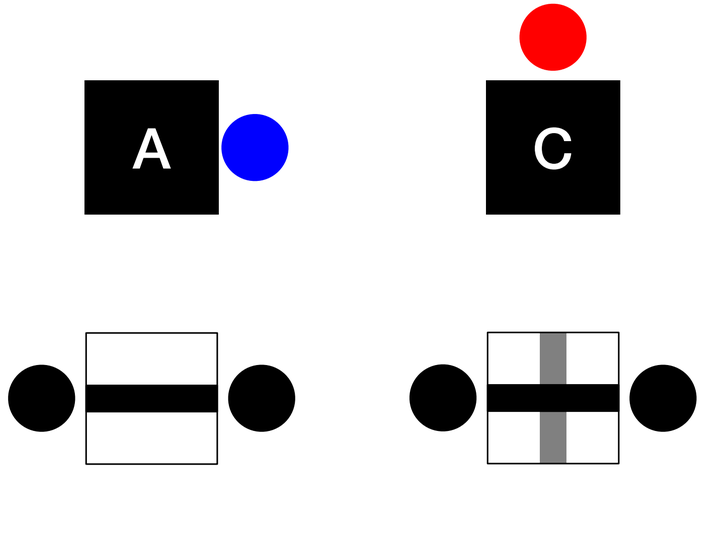
Dynamic Placeholder



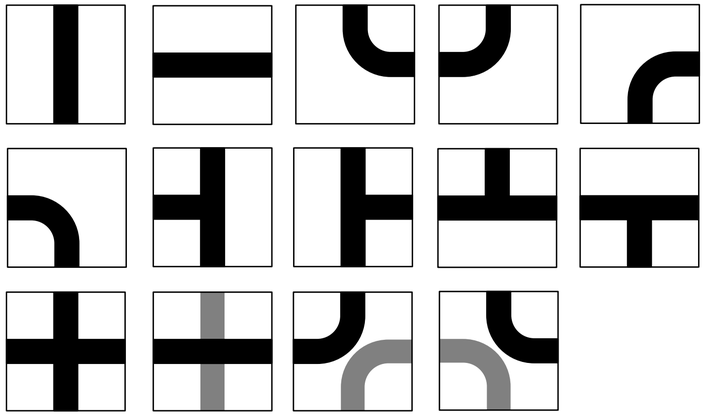


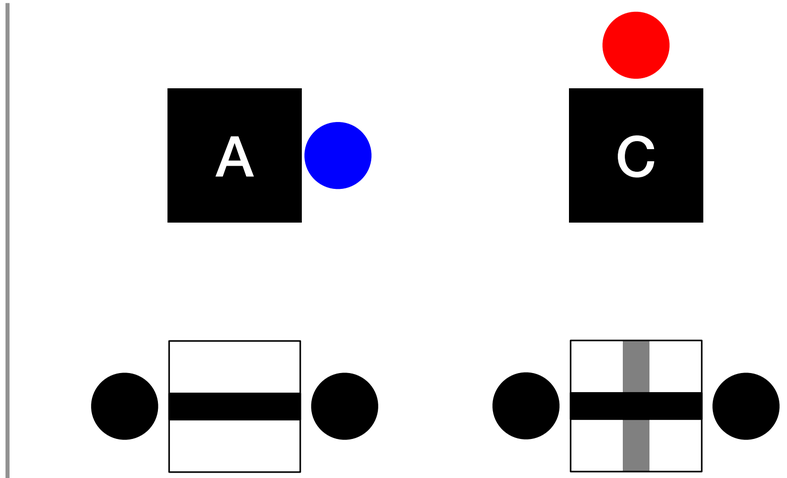
Dynamic Placeholder

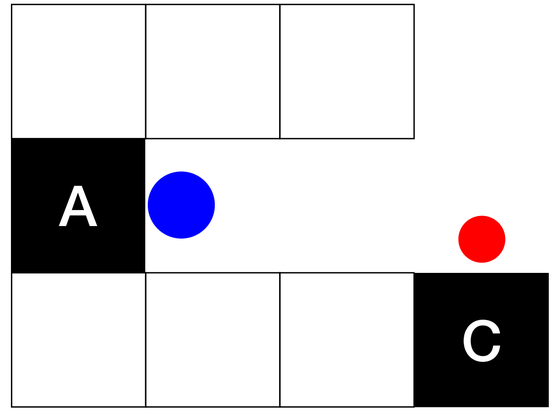




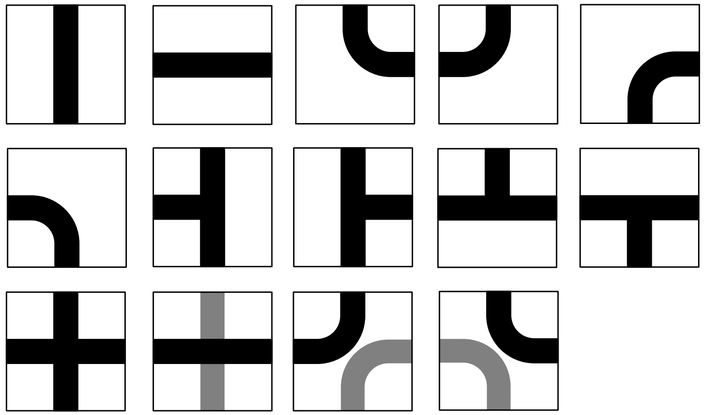
Dynamic Placeholder

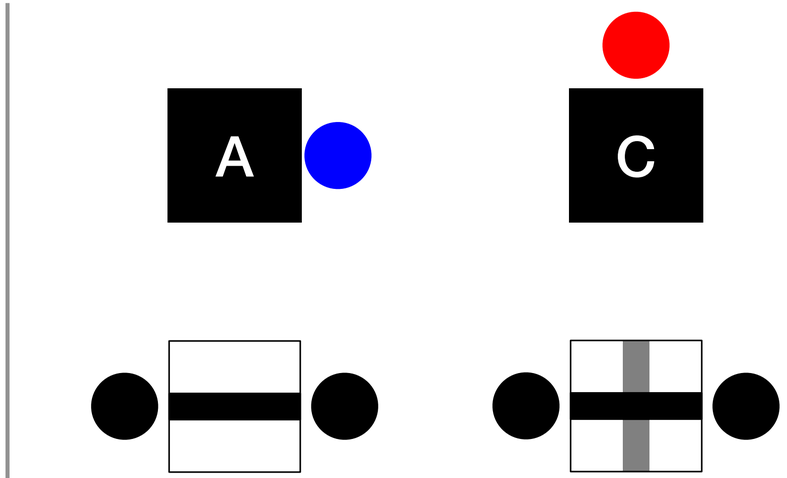


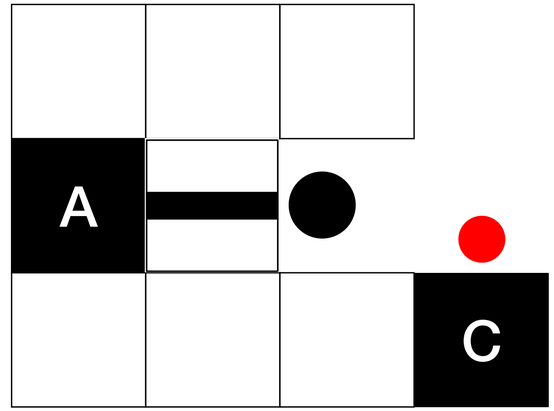




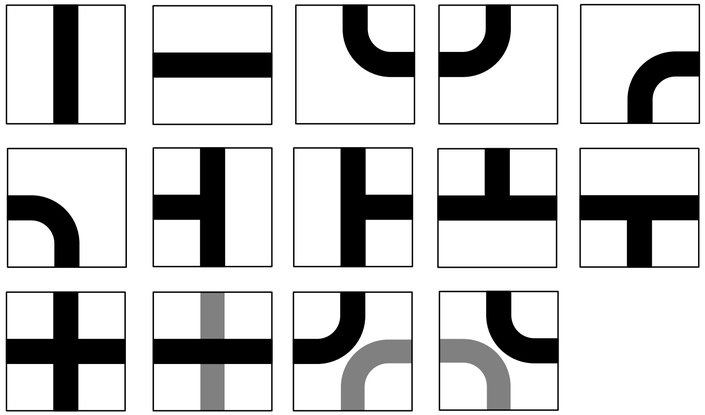
Dynamic Placeholder

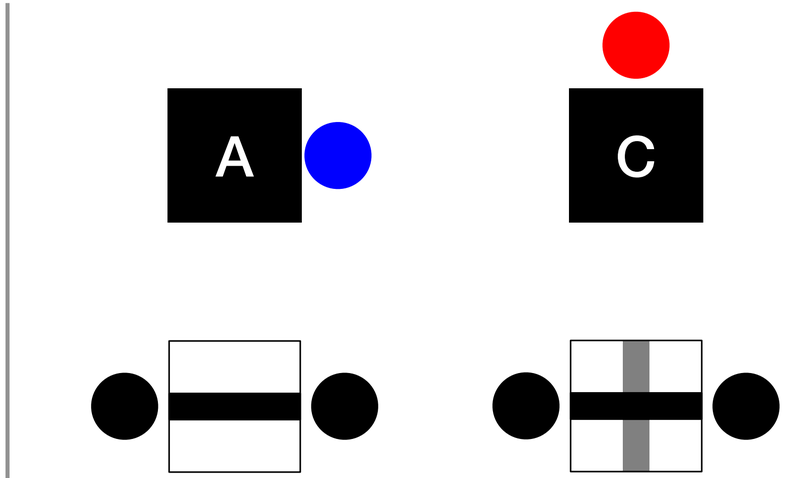


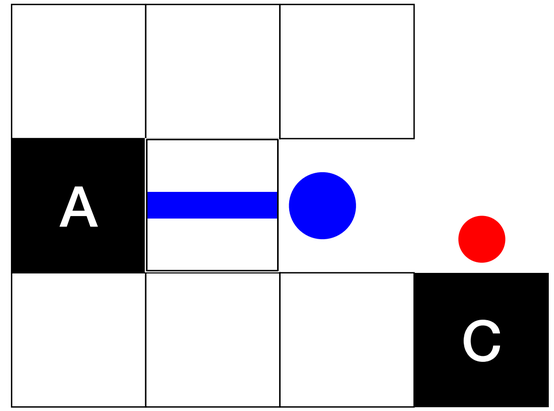




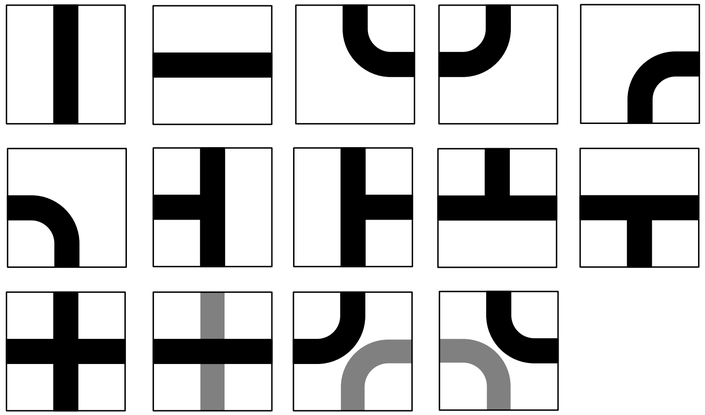
Dynamic Placeholder

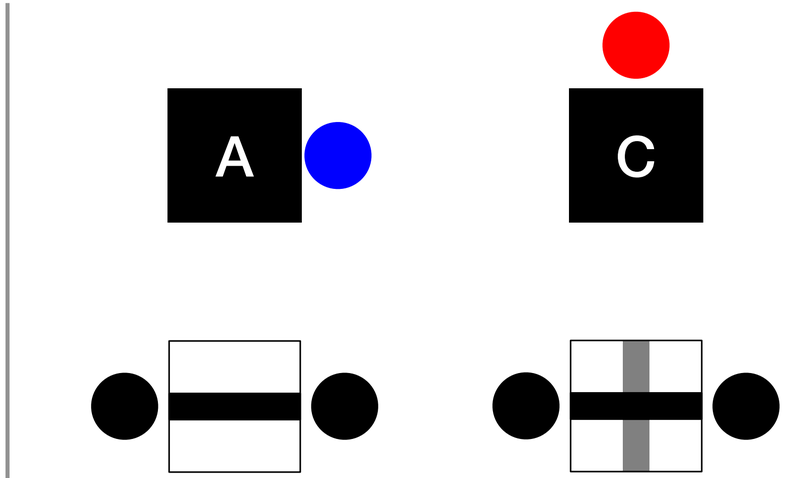


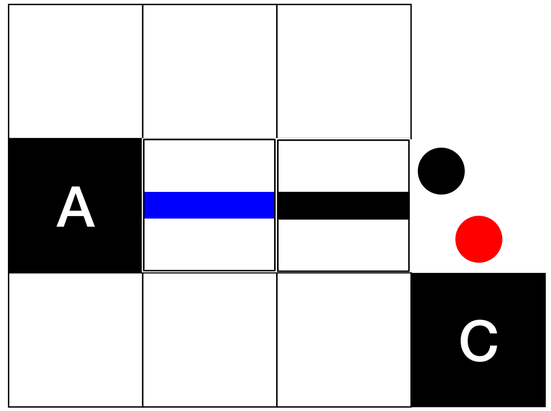




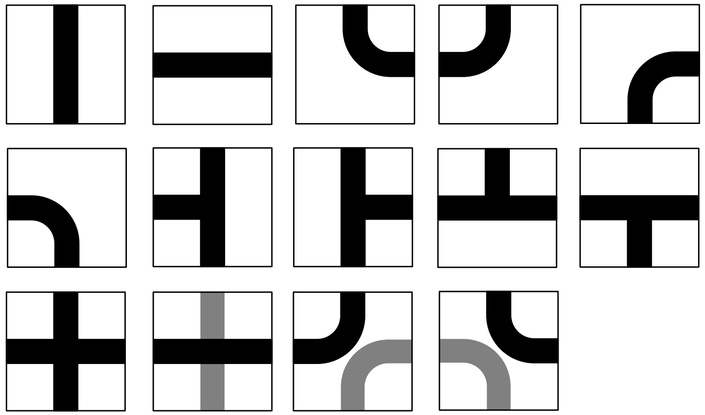
Dynamic Placeholder

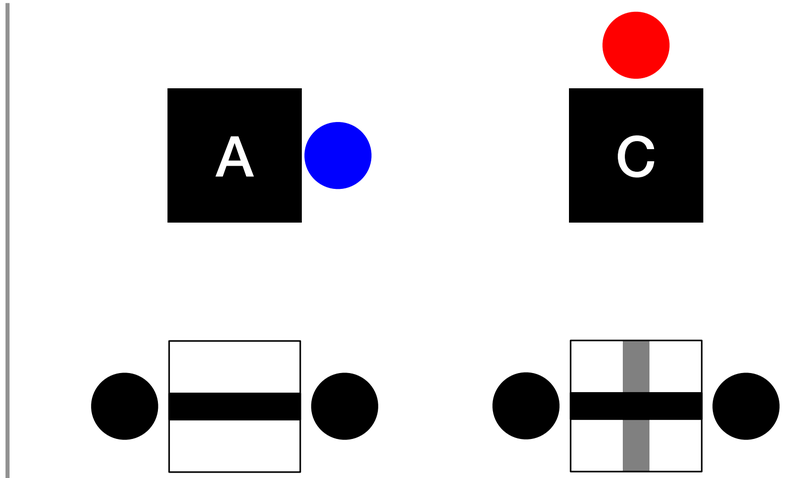


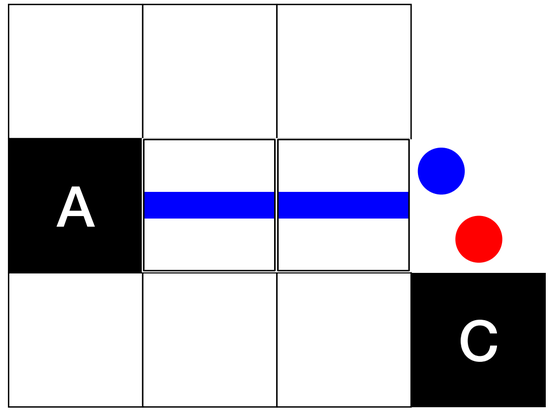




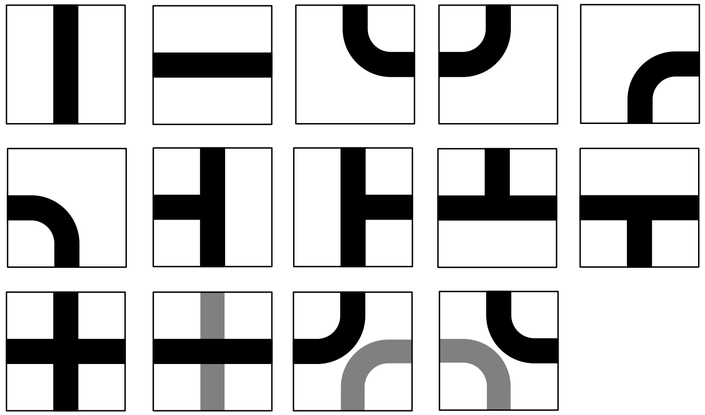
Dynamic Placeholder

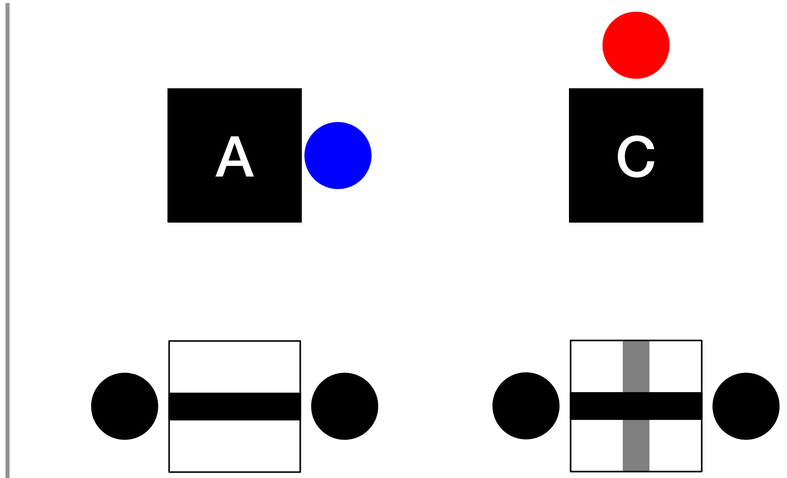


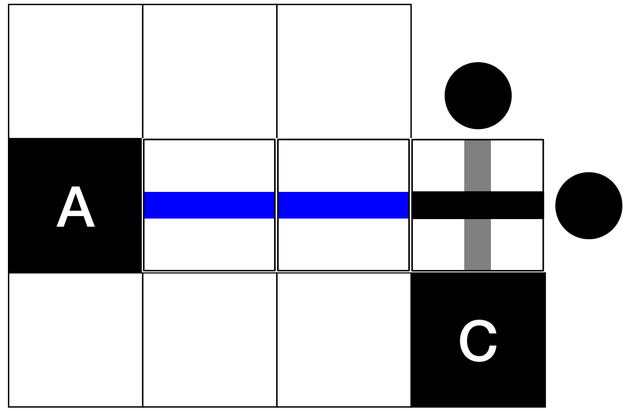




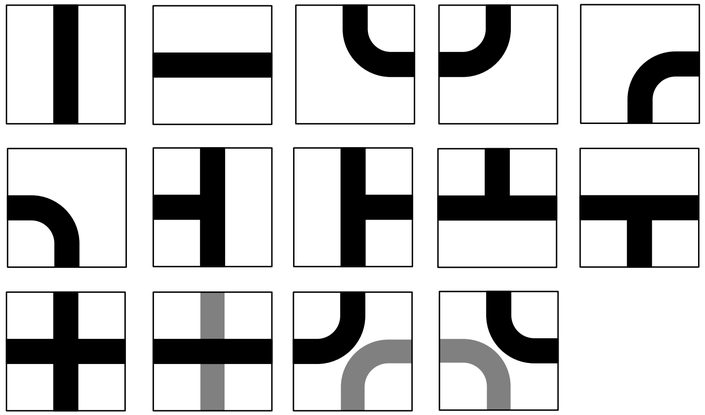
Dynamic Placeholder

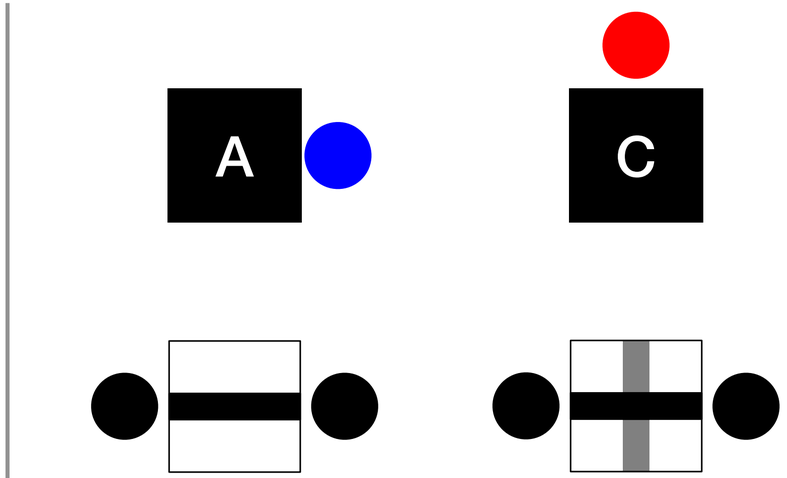


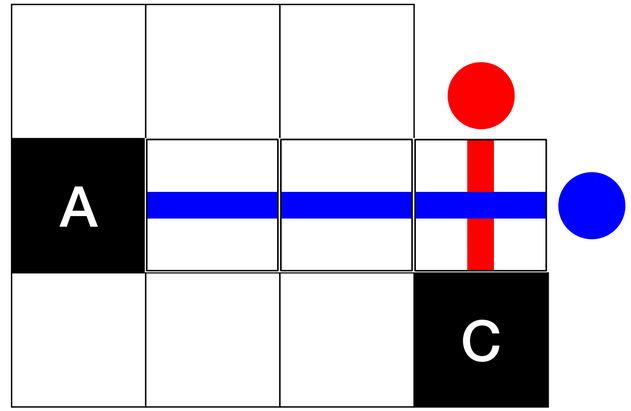




Dynamic Placeholder



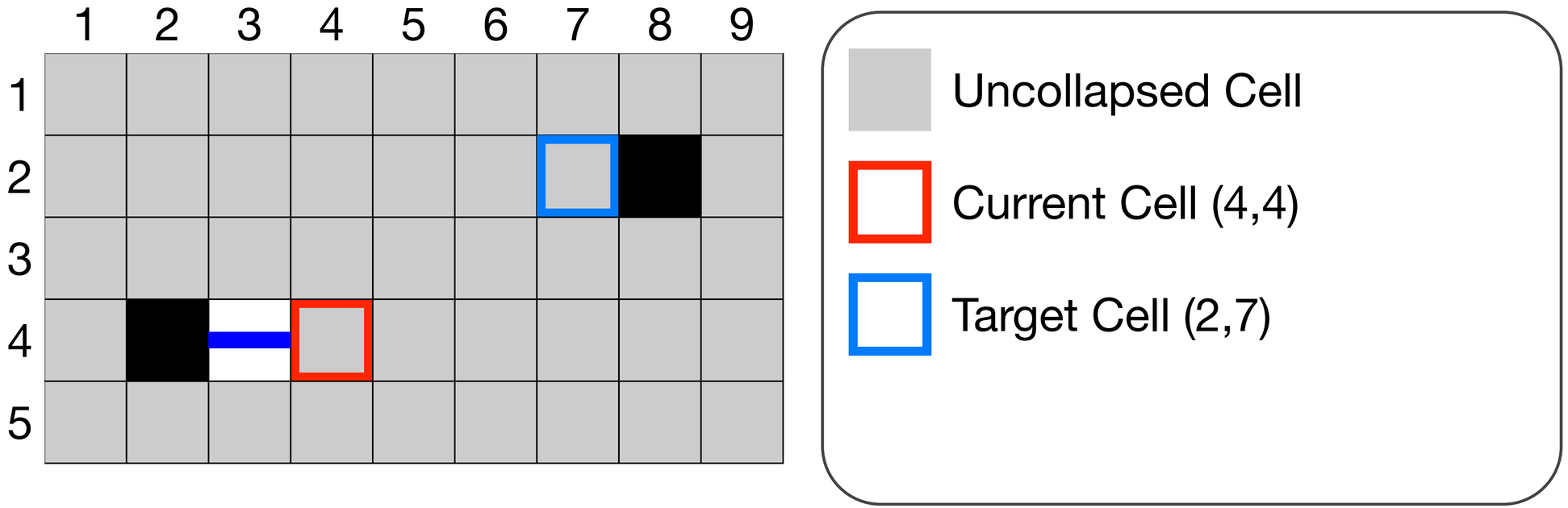




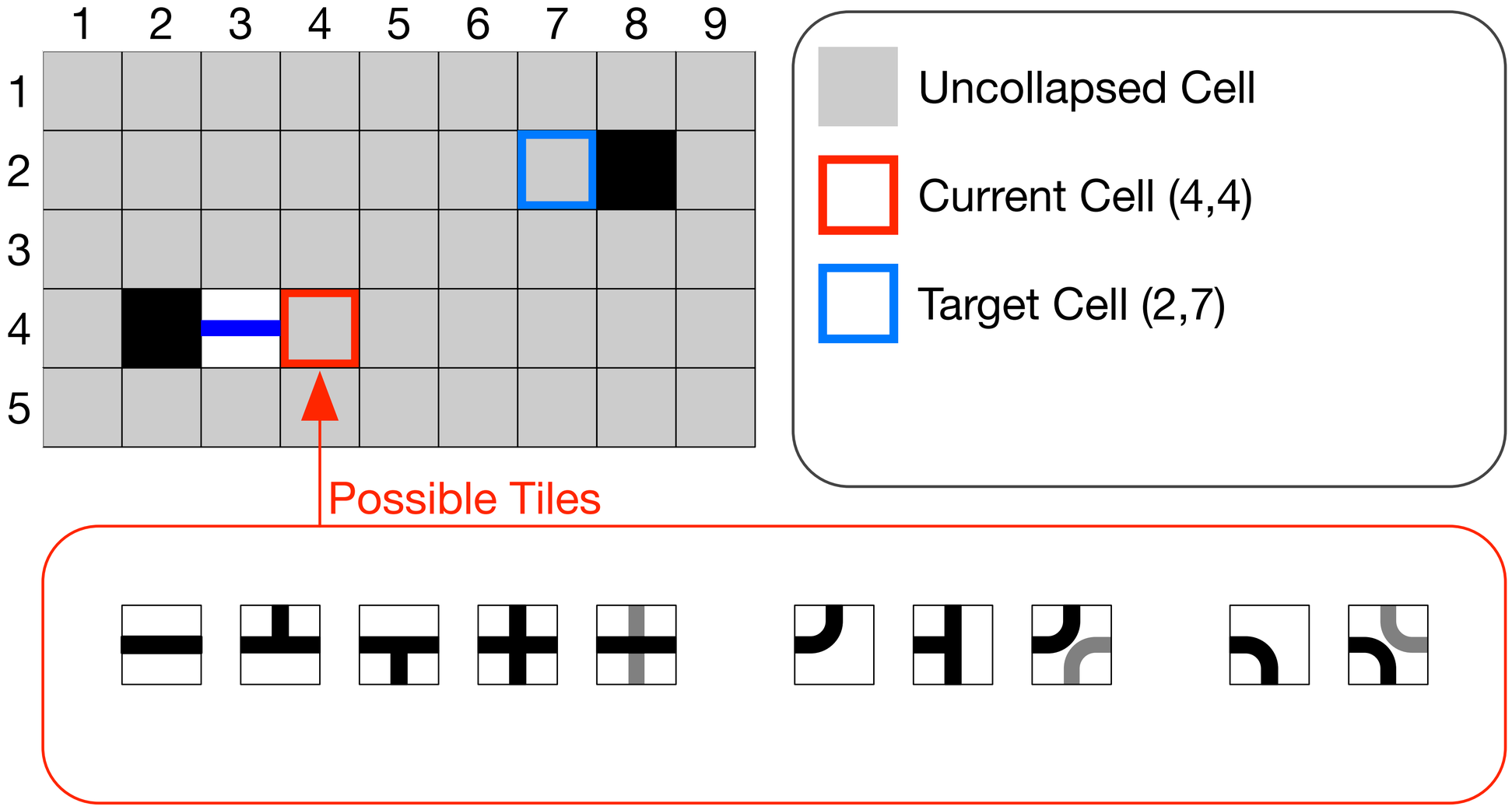
Core Adaptations

Dynamic Tile Weights

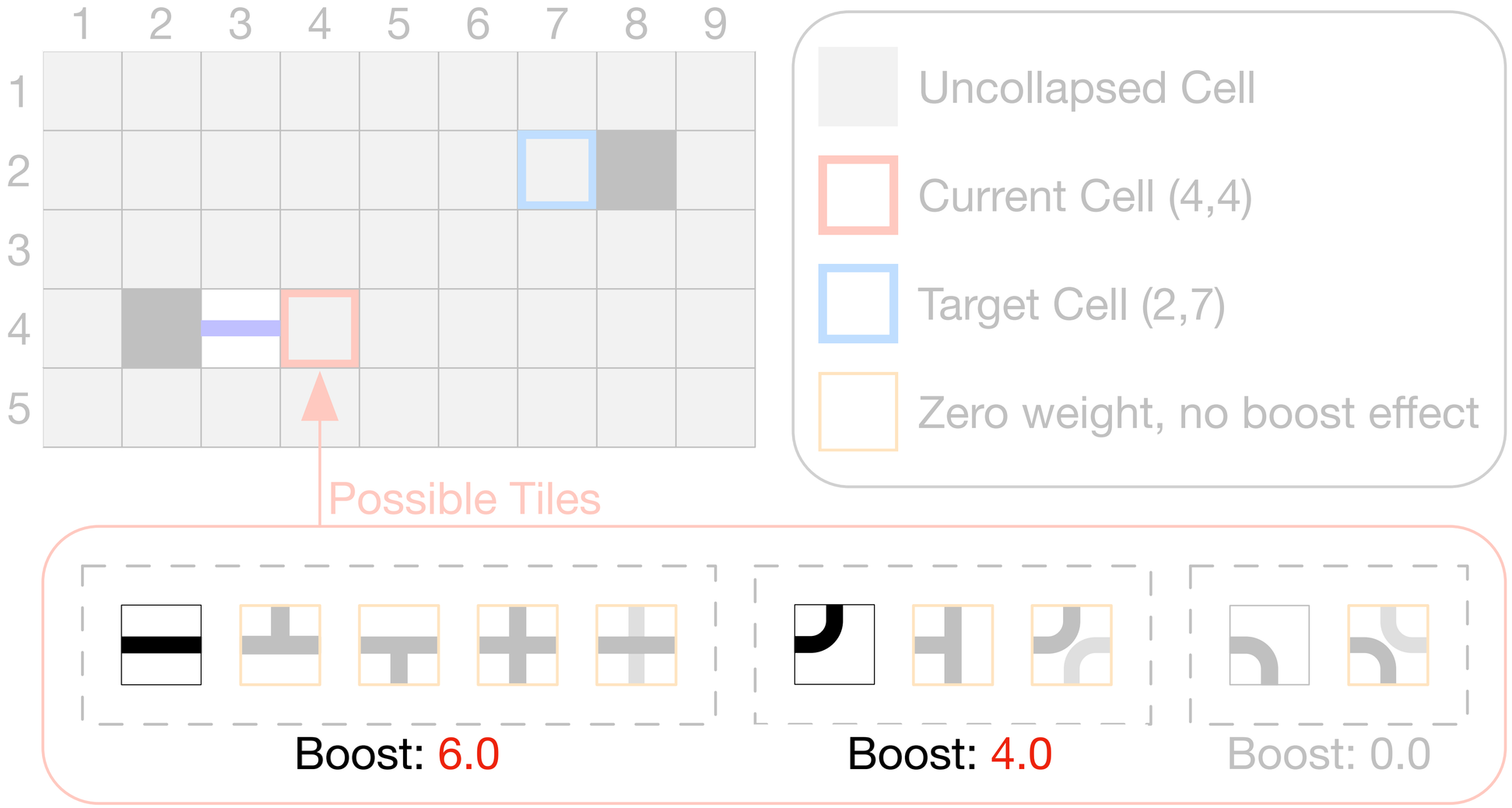
Dynamic Tile Weights



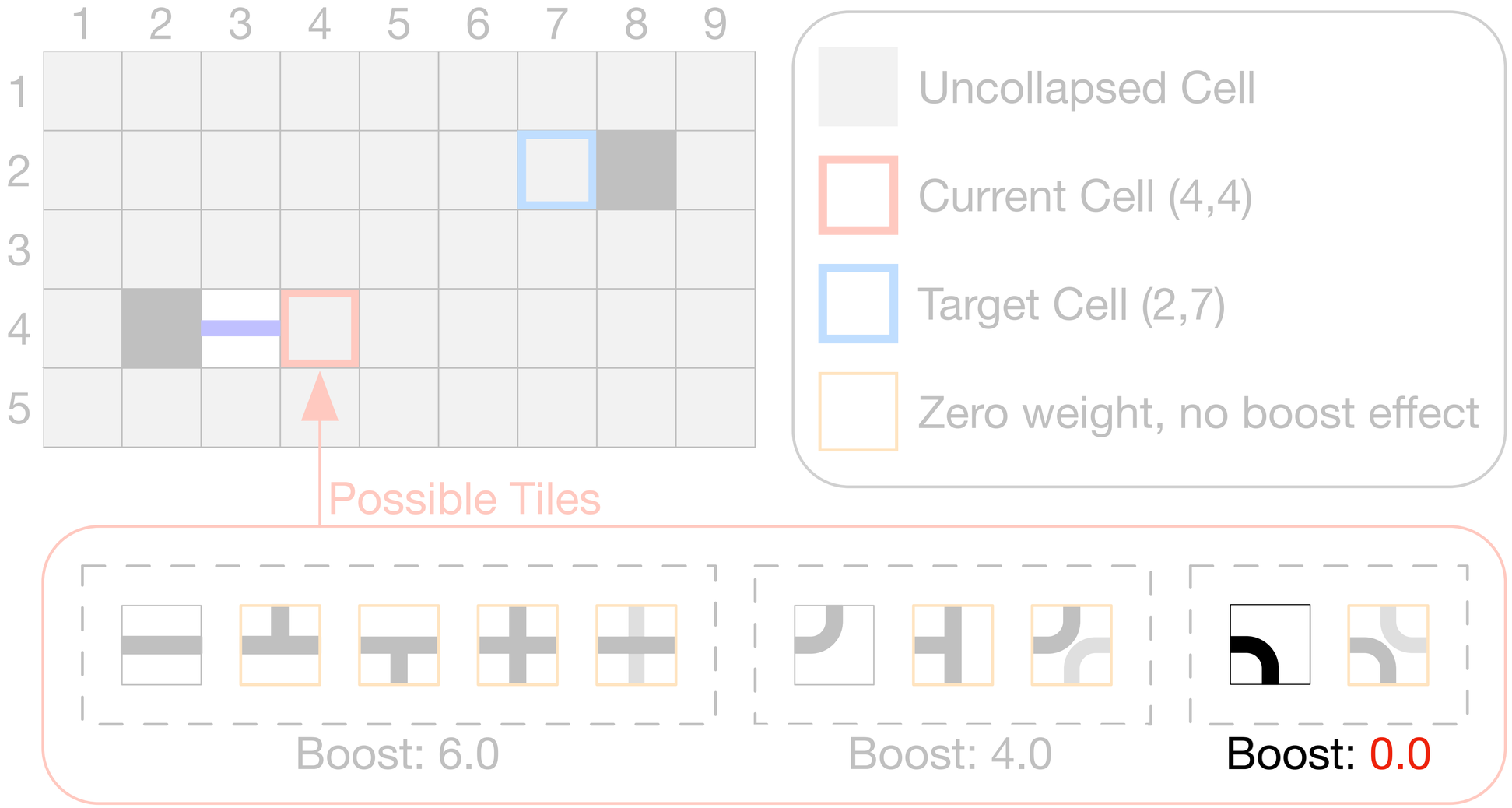
Dynamic Tile Weights



Dynamic Tile Weights



Dynamic Tile Weights



Core Adaptations

Additional Mechanism

Core Adaptations

Additional Mechanism

Grid Expansion

Core Adaptations

Additional Mechanism

Grid Expansion • Zero-weight Tiles

Core Adaptations

Additional Mechanism

Grid Expansion • Zero-weight Tiles • Maximum-Entropy Heuristic

Core Adaptations

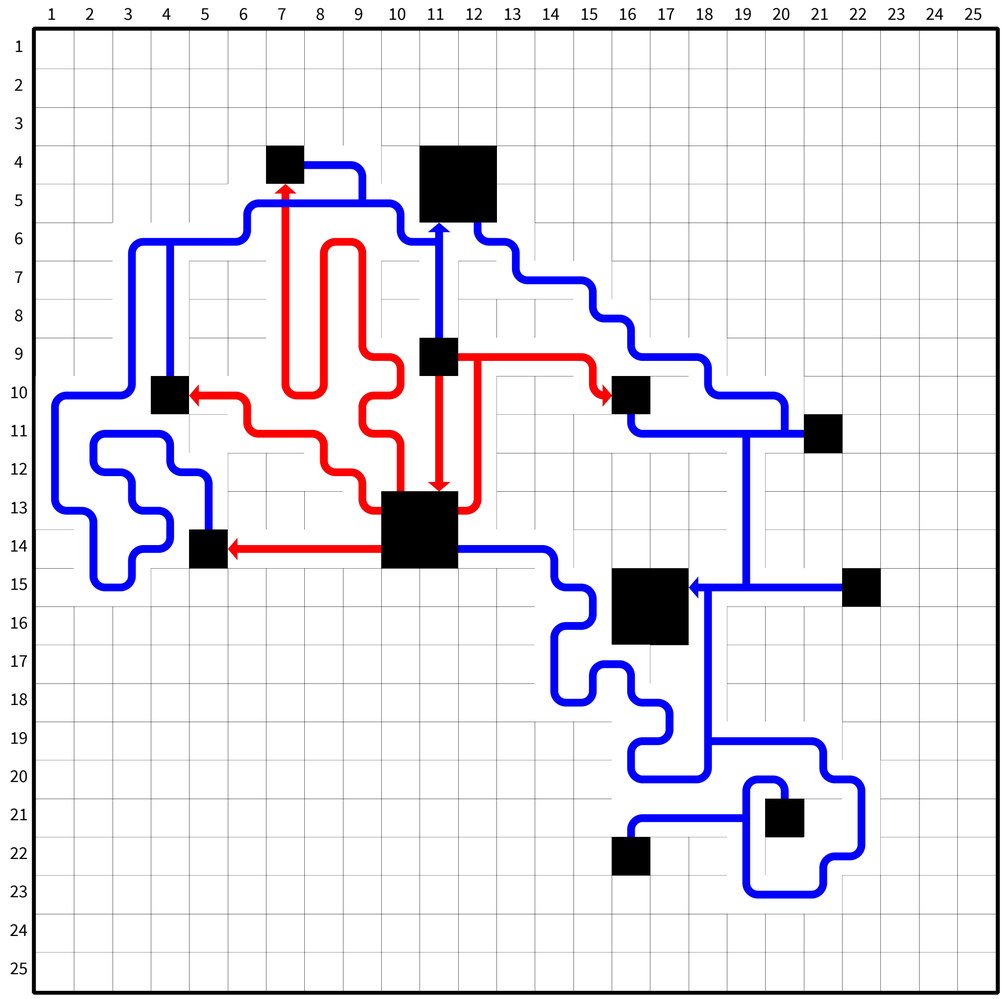
Additional Mechanism

Grid Expansion • Zero-weight Tiles • Maximum-Entropy Heuristic • Backtracking

Core Adaptations

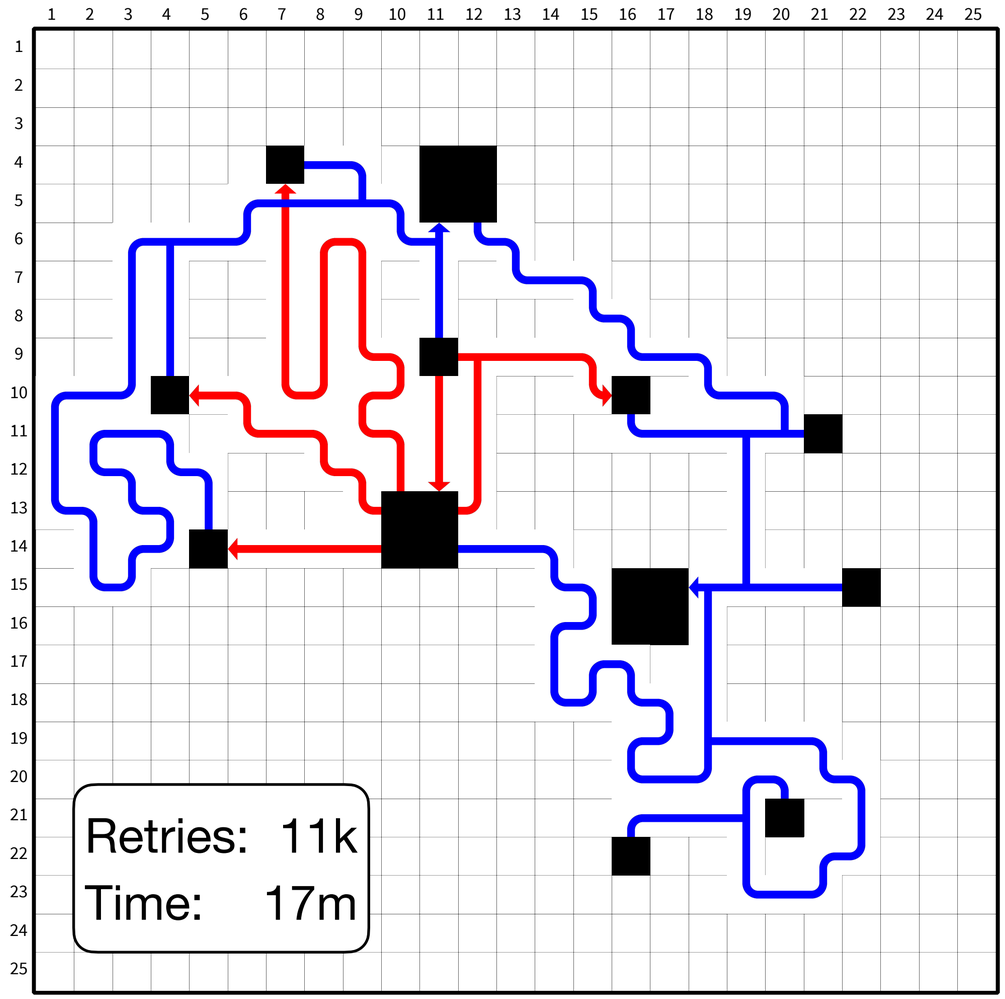
Additional Mechanism

Grid Expansion • Zero-weight Tiles • Maximum-Entropy Heuristic • Backtracking • Parallel Execution



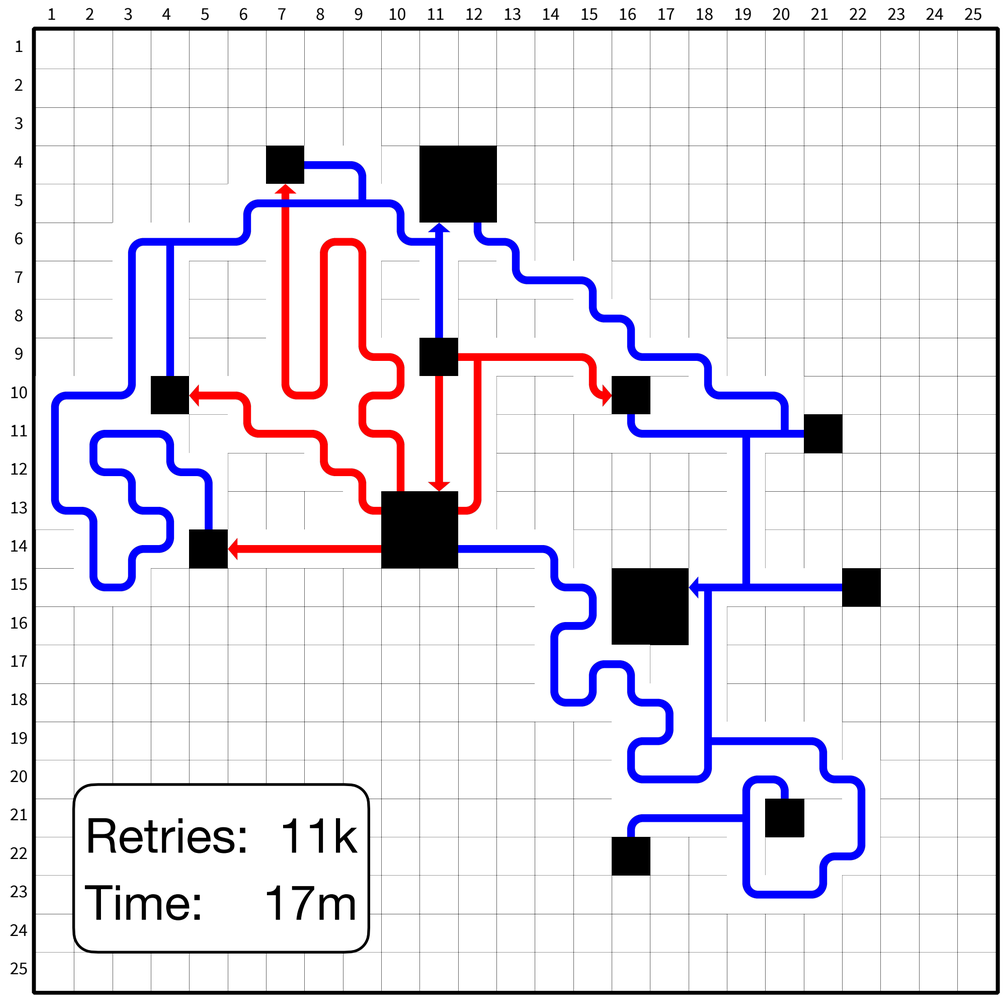
WFC pure

Class Inheritance Usage

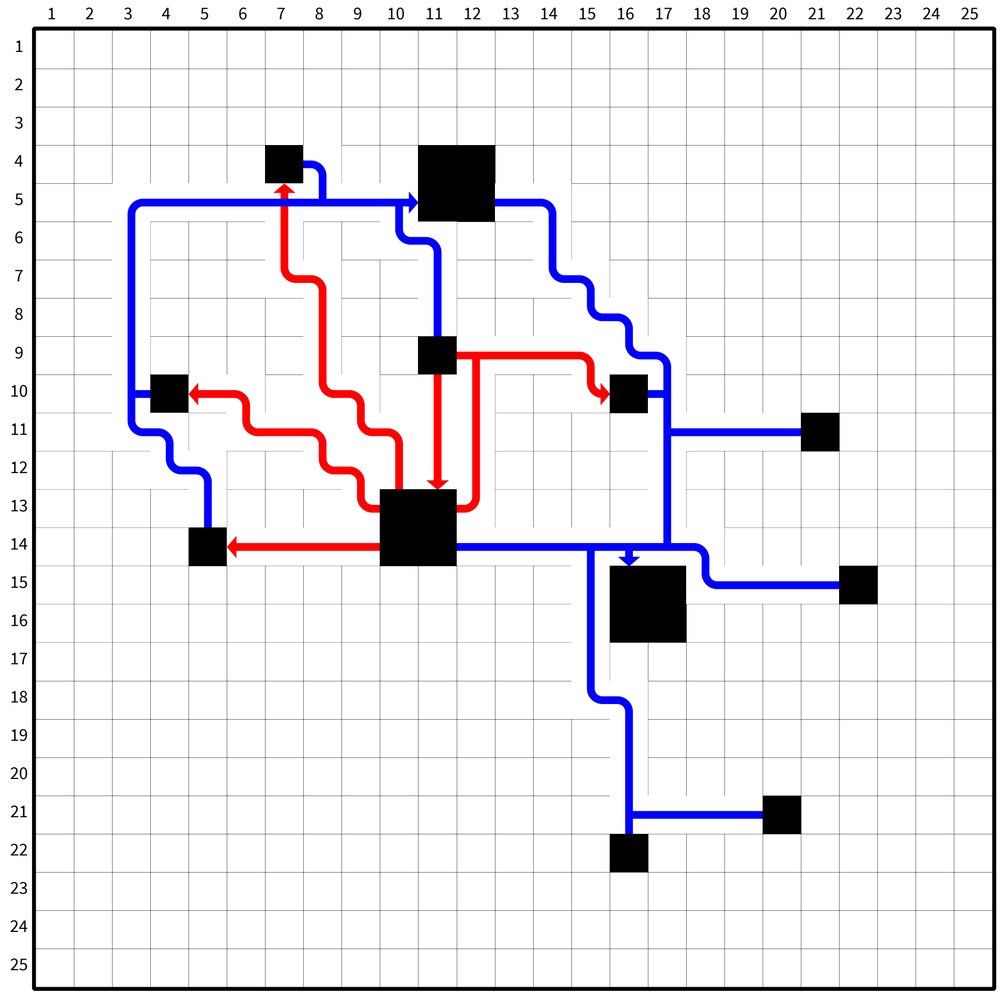


WFC pure

Class Inheritance Usage

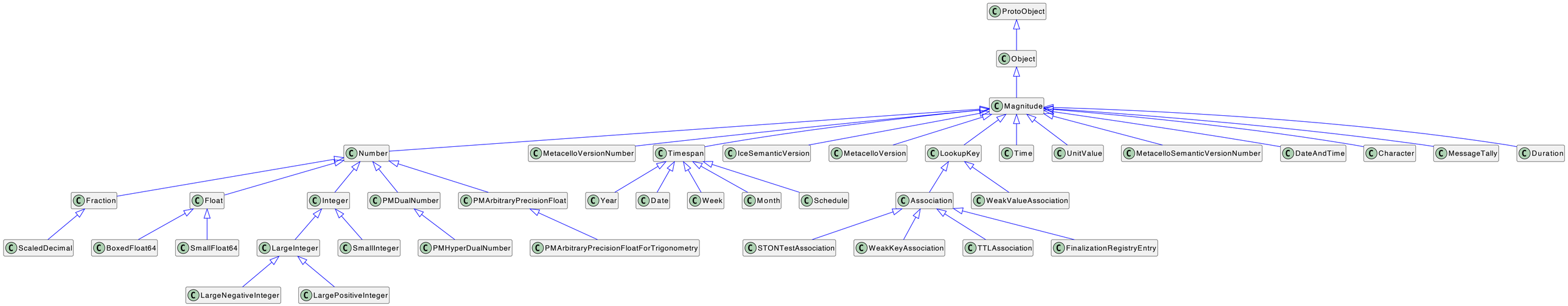


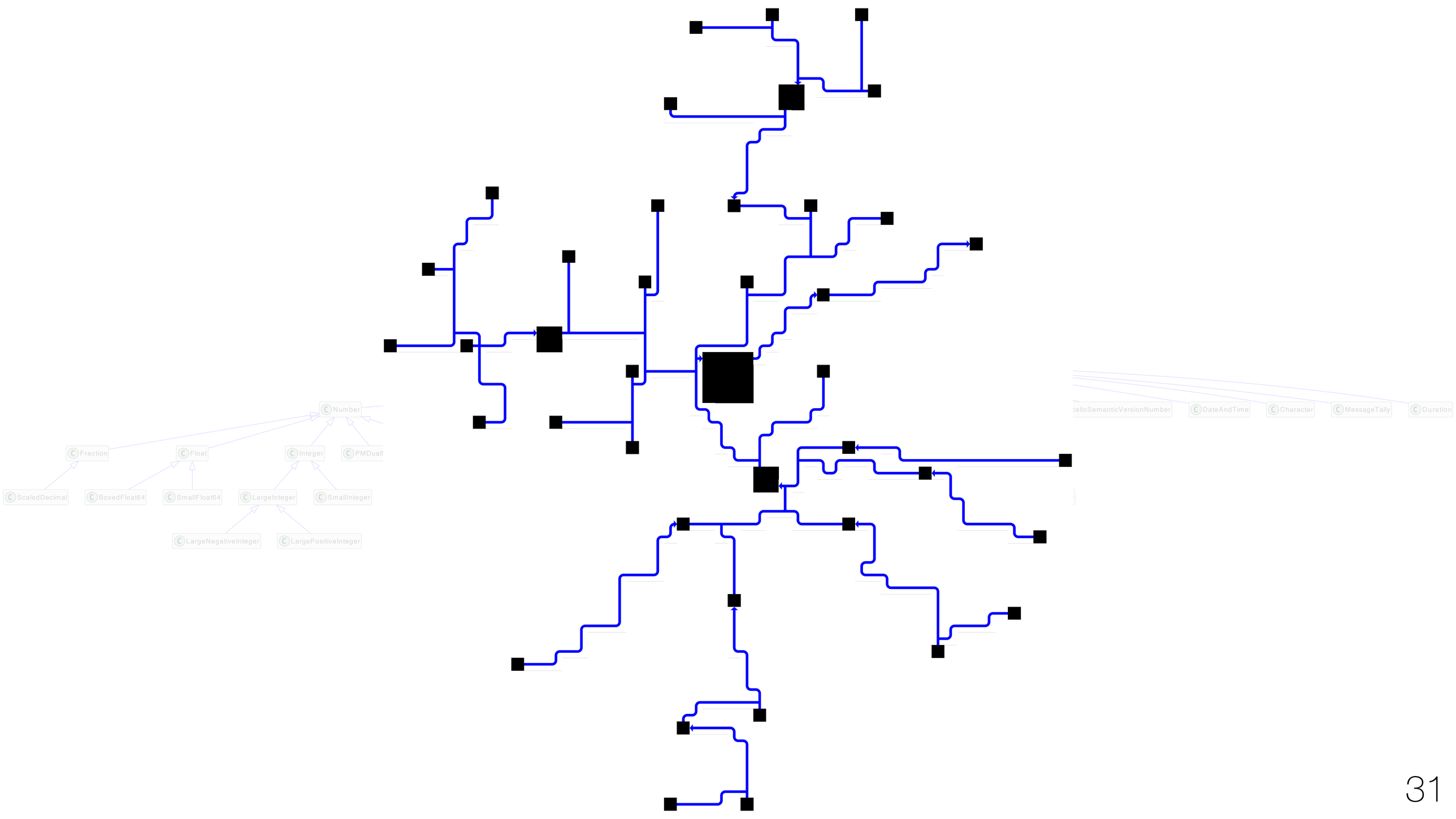
WFC pure



Optional path optimization

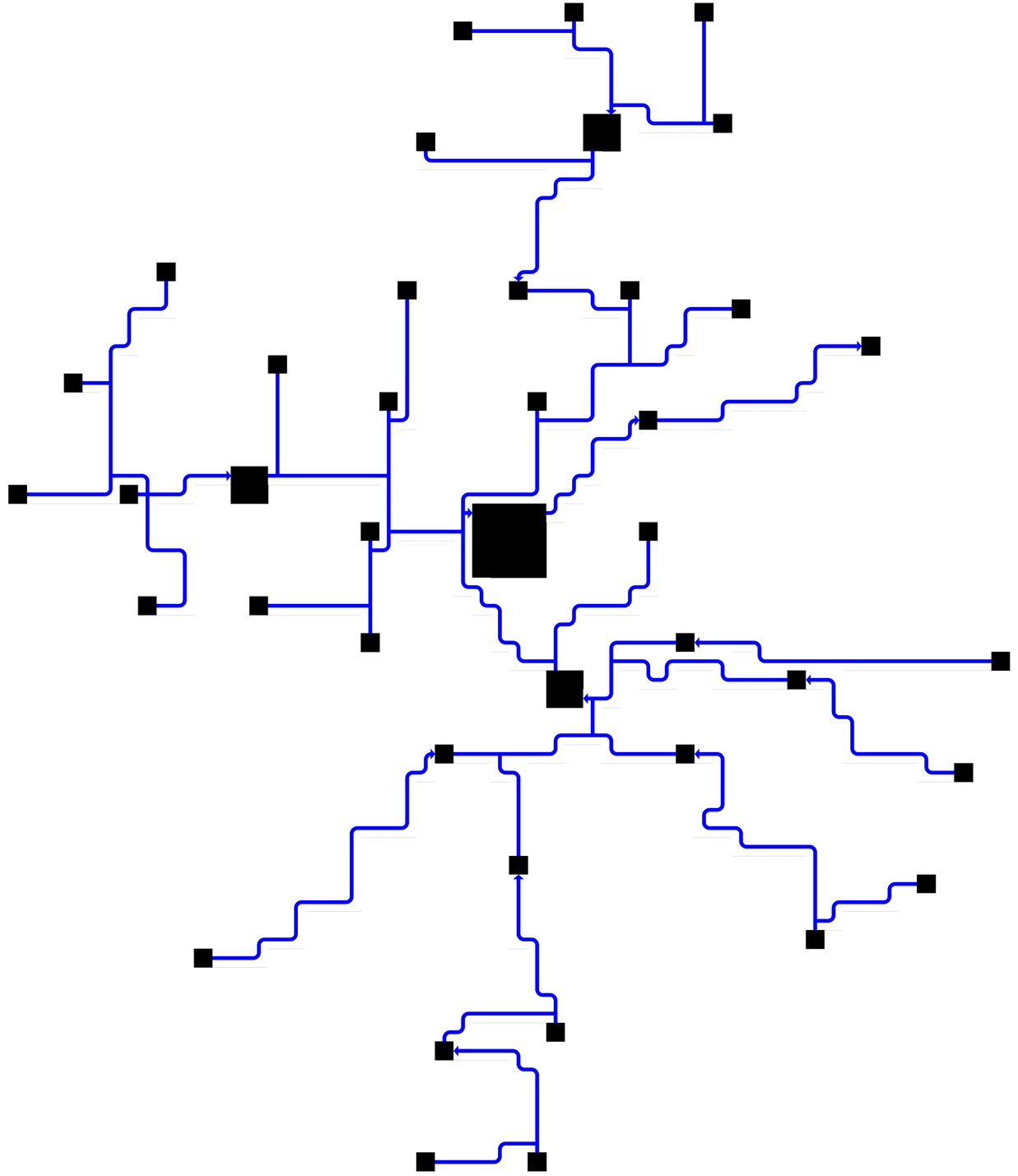
Class Inheritance Usage


ProtoObject
Object
Magnitude
Number
MetacelloVersionNumber
Timespan
IceSemanticVersion
MetacelloVersion
LookupKey
Time
UnitValue
MetacelloSemanticVersionNumber
DateAndTime
Character
MessageTally
Duration
Fraction
Float
Integer
PMDualNumber
PMArbitraryPrecisionFloat
Year
Date
Week
Month
Schedule
Association
WeakValueAssociation
ScaledDecimal
BoxedFloat64
SmallFloat64
LargeInteger
SmallInteger
PMHyperDualNumber
PMArbitraryPrecisionFloatForTrigonometry
STONTestAssociation
WeakKeyAssociation
TTLAssociation
FinalizationRegistryEntry
LargeNegativeInteger
LargePositiveInteger


Number
Fraction
Float
Integer
PMDuall
ScaledDecimal
BoxedFloat64
SmallFloat64
LargeInteger
SmallInteger
LargeNegativeInteger
LargePositiveInteger
elloSemanticVersionNumber
DateAndTime
Character
MessageTally
Duration

Strengths

Compact Layout

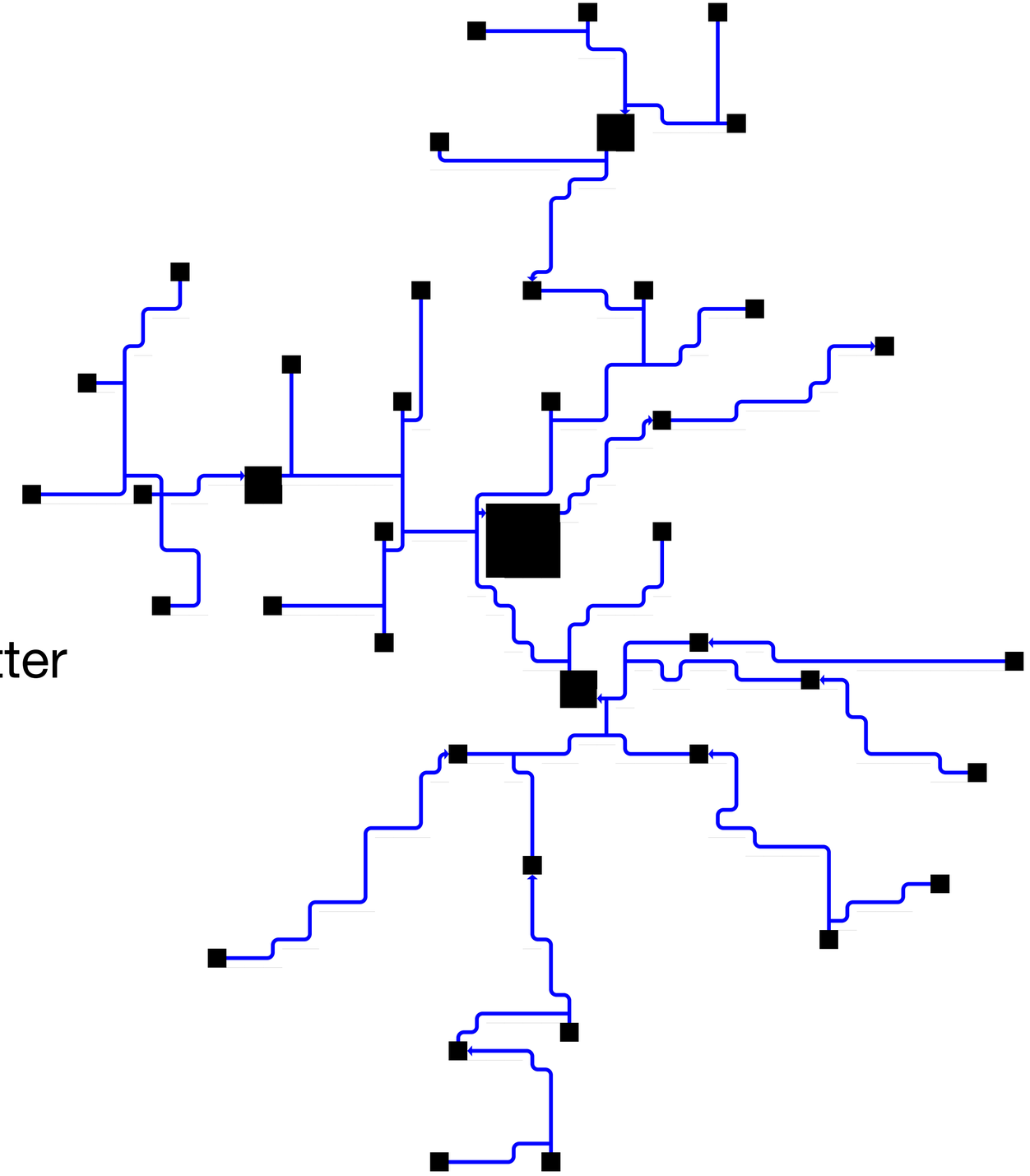


Weaknesses

Strengths

Compact Layout

Reduce Visual Clutter



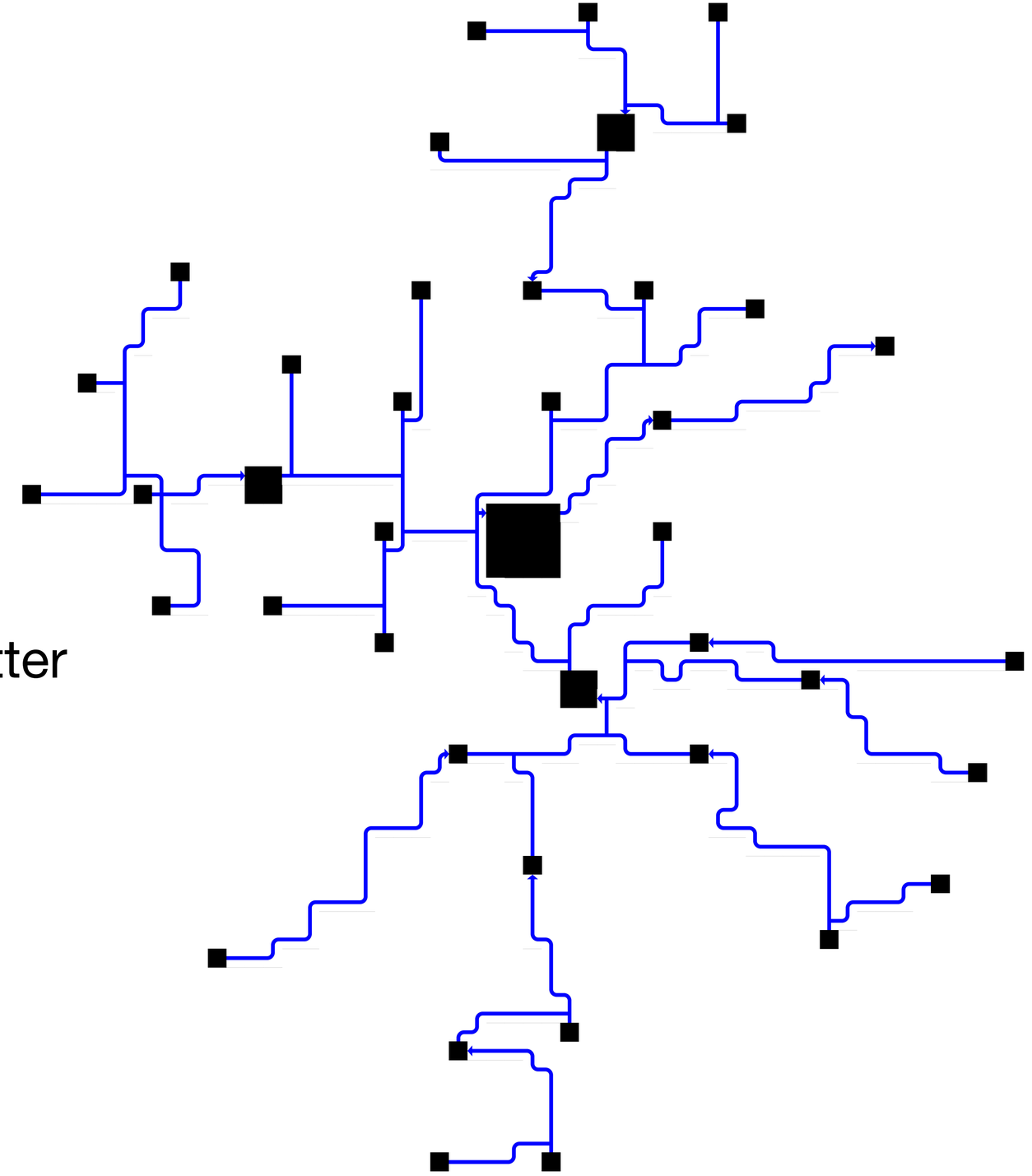
Weaknesses

Strengths

Compact Layout

Reduce Visual Clutter

Promising Results



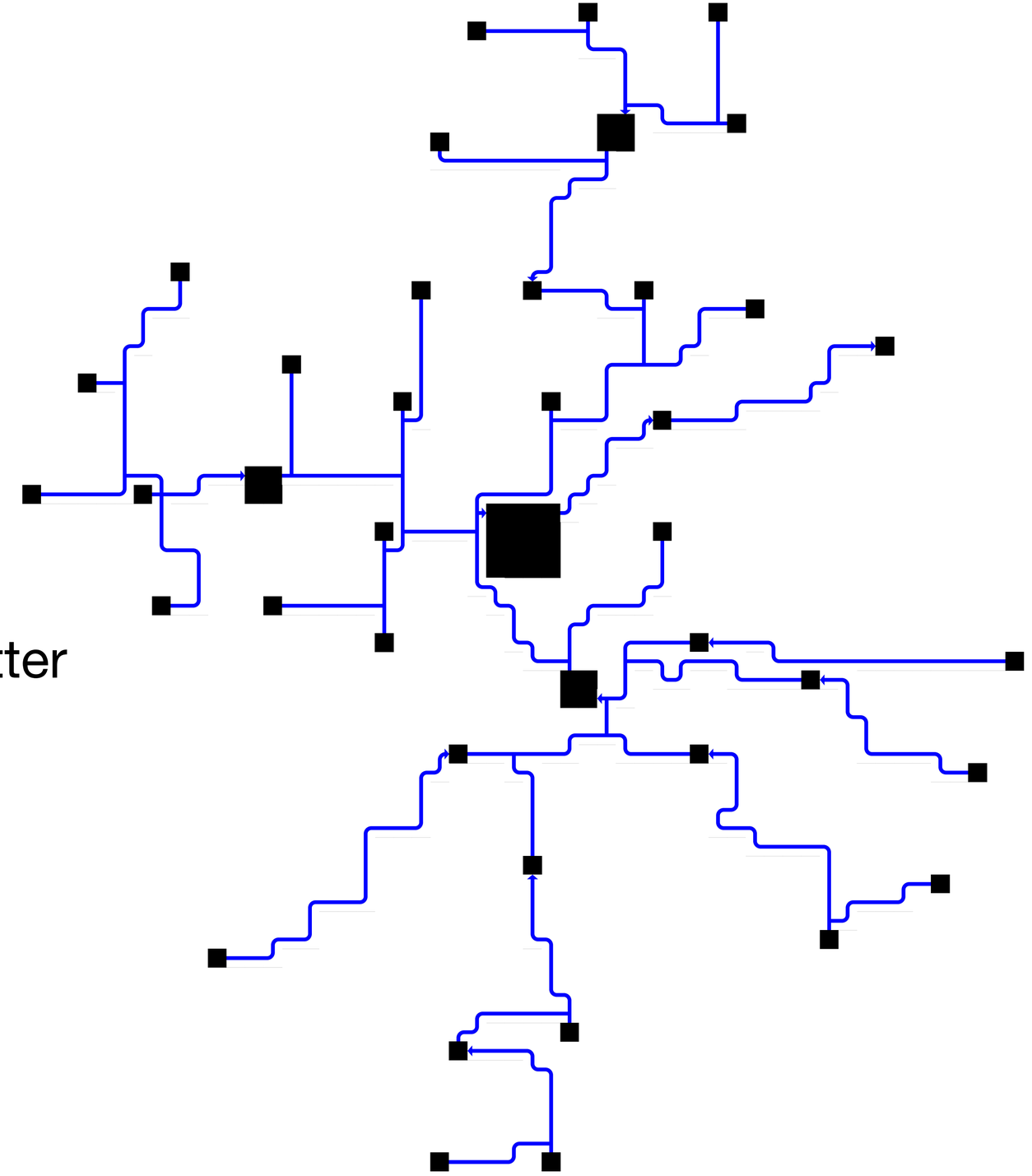
Weaknesses

Strengths

Compact Layout

Reduce Visual Clutter

Promising Results



Weaknesses

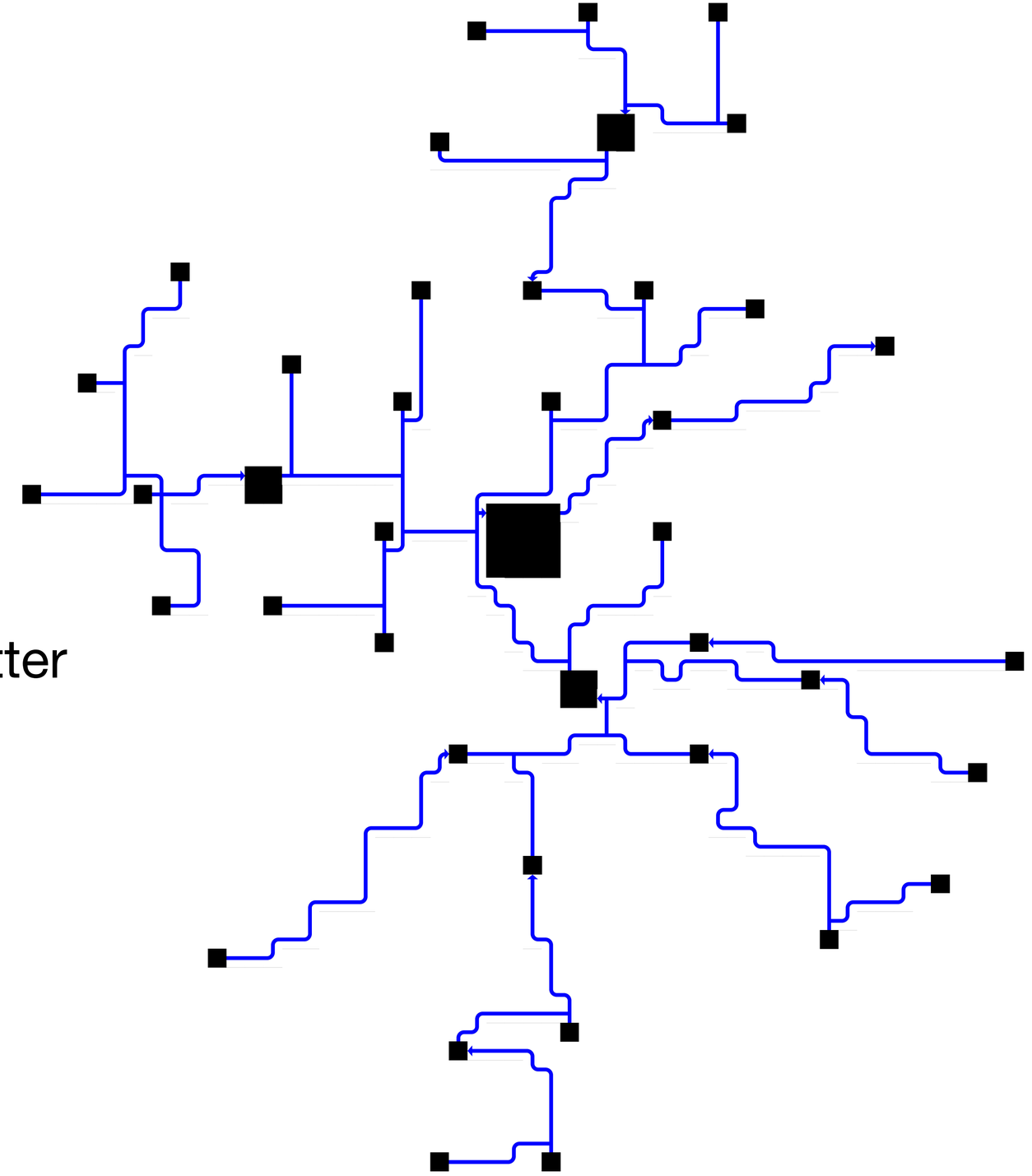
No evaluation

Strengths

Compact Layout

Reduce Visual Clutter

Promising Results



Weaknesses

No evaluation

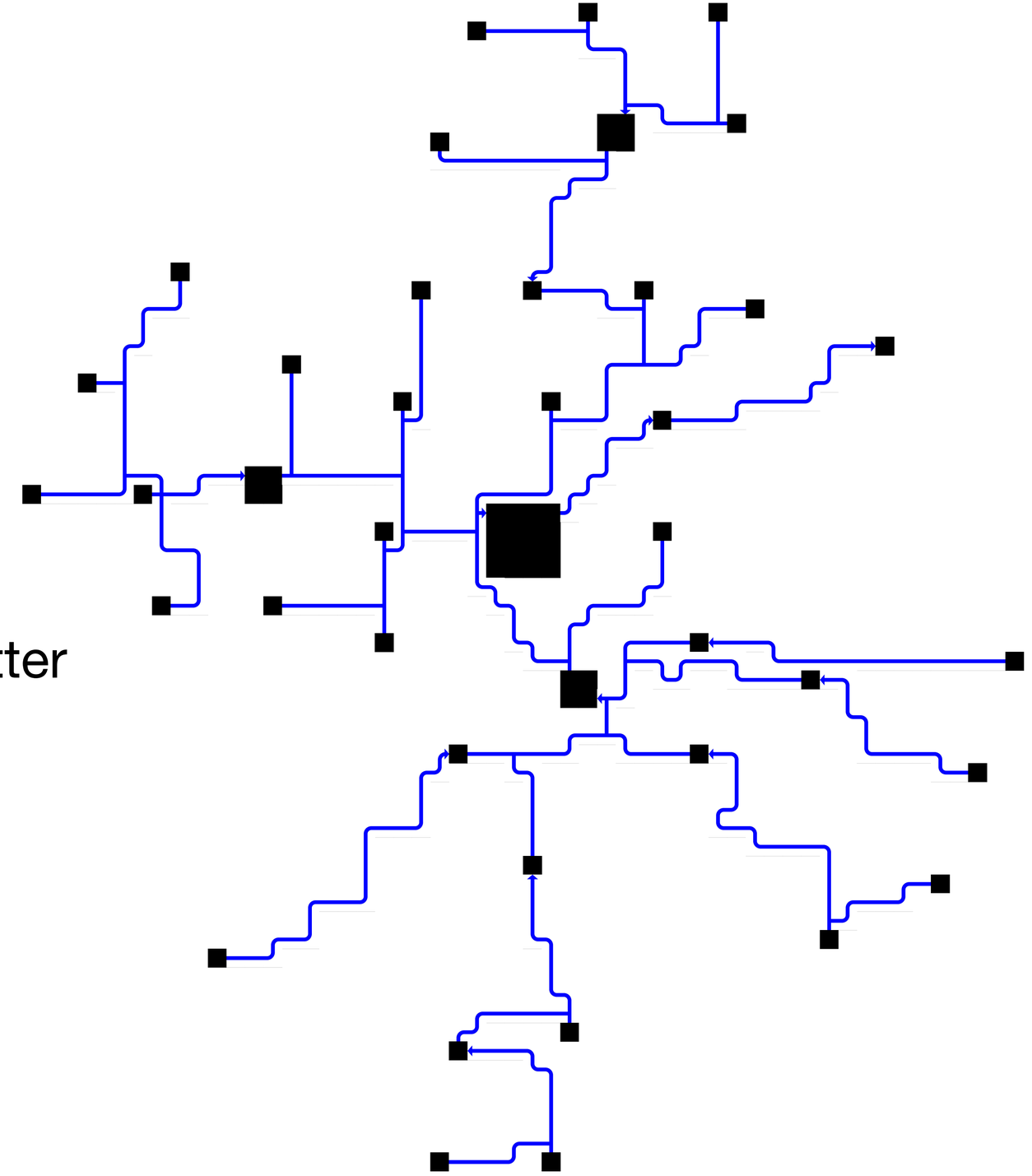
Wasted Time

Strengths

Compact Layout

Reduce Visual Clutter

Promising Results



Weaknesses

No evaluation

Wasted Time

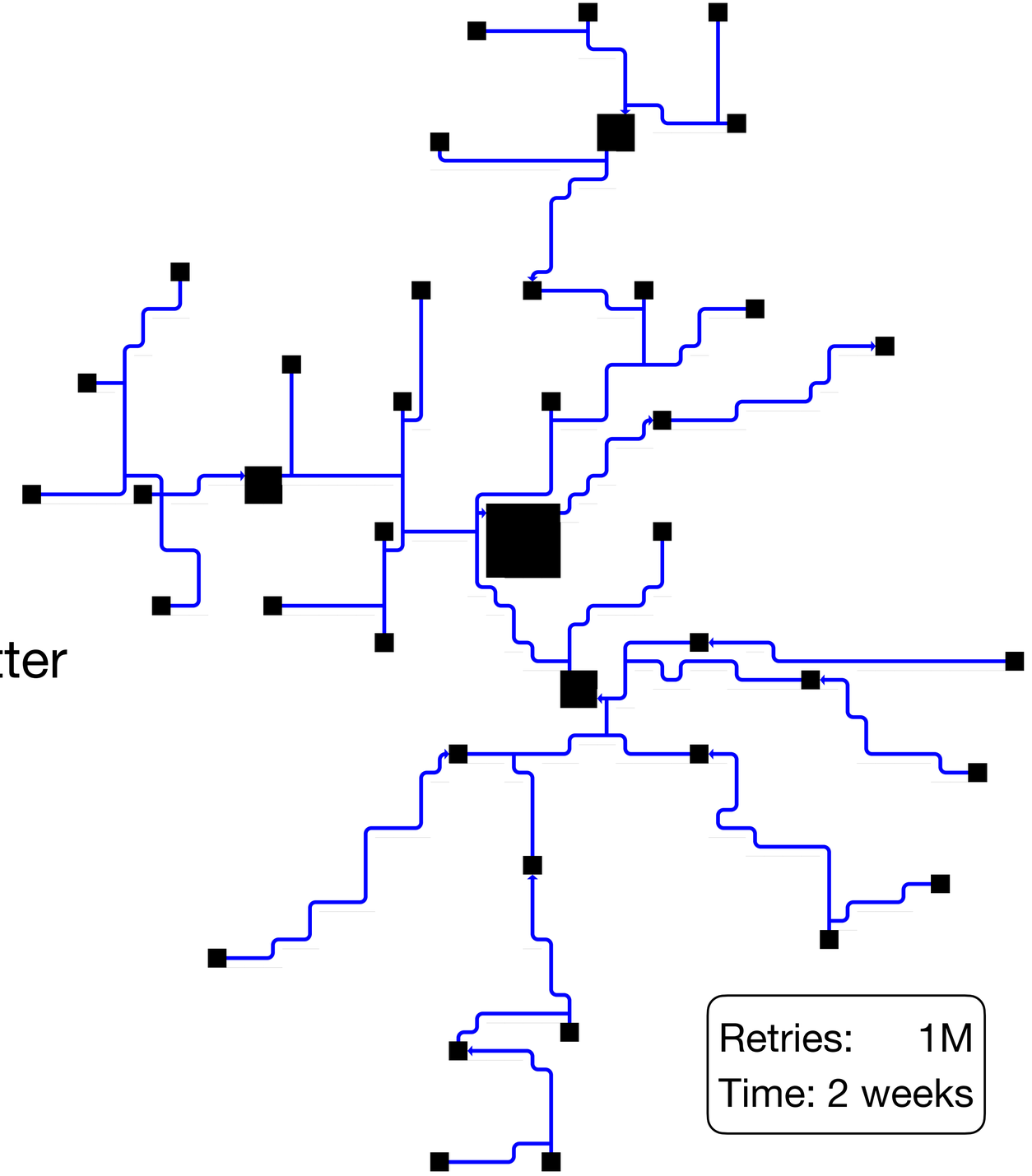
Scalability

Strengths

Compact Layout

Reduce Visual Clutter

Promising Results



Weaknesses

No evaluation

Wasted Time

Scalability

Contributions

- Combining MDS with WFC

Contributions



Combining MDS with WFC



Global Constraints

Contributions


Combining MDS with WFC


Global Constraints


Tool Implementation

Contributions

 Combining MDS with WFC

 Global Constraints

 Tool Implementation

 Many Fine-tunings

Contributions

 Combining MDS with WFC

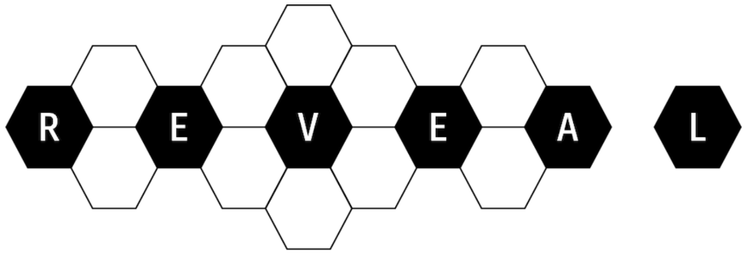
 Global Constraints

 Tool Implementation

 Many Fine-tunings

 IEEE VISSOFT Publication



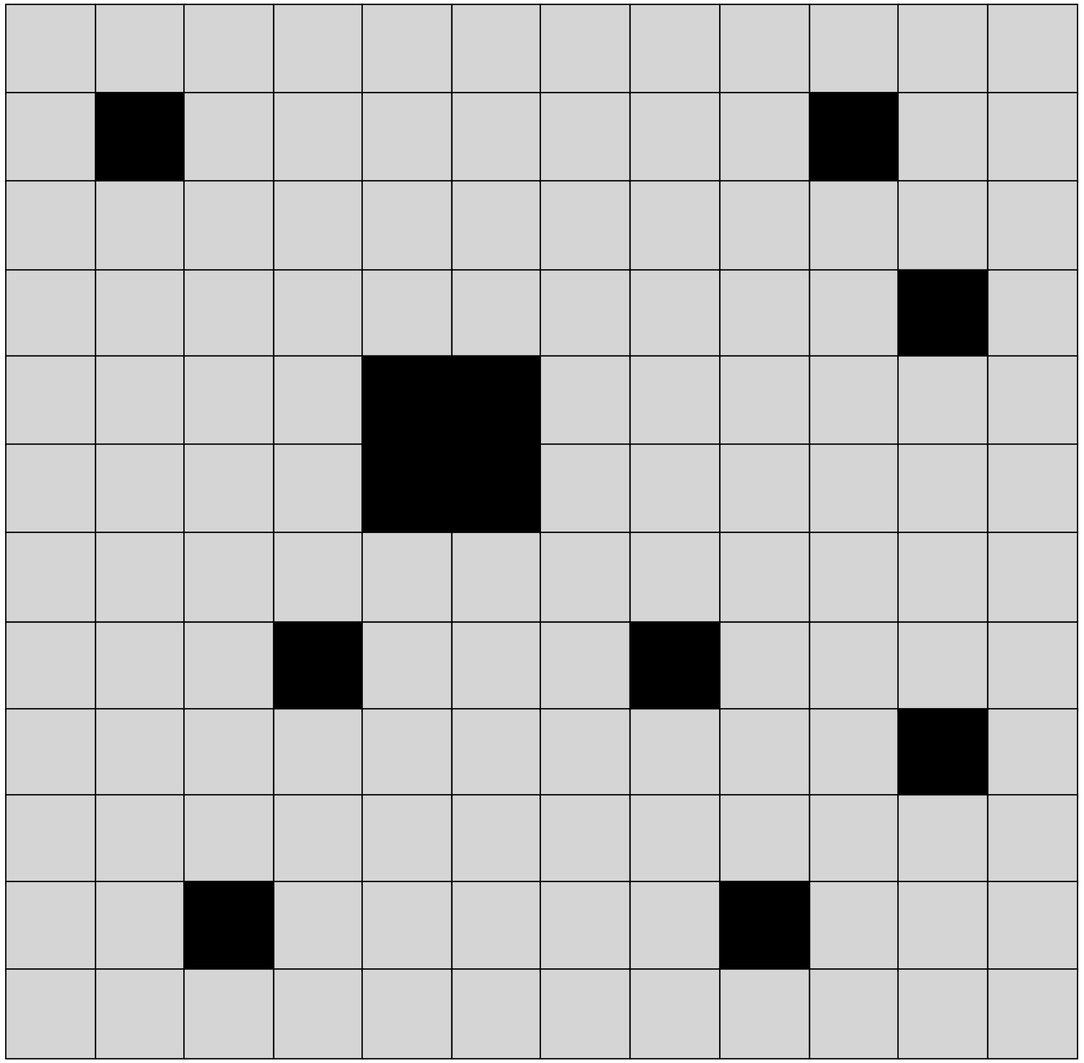


Thanks for your attention!

Simone Lesinigo

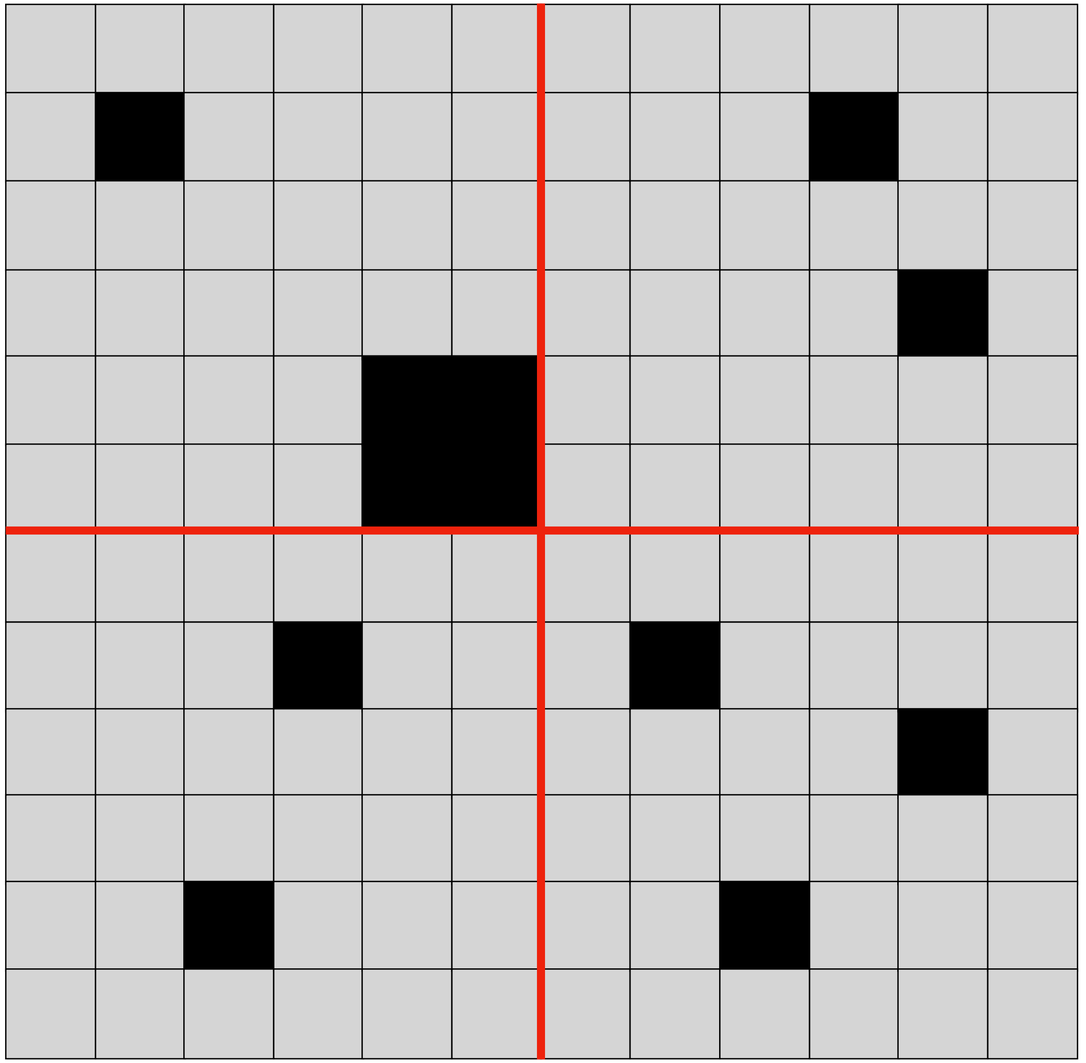
Future Work

- Nested WFC



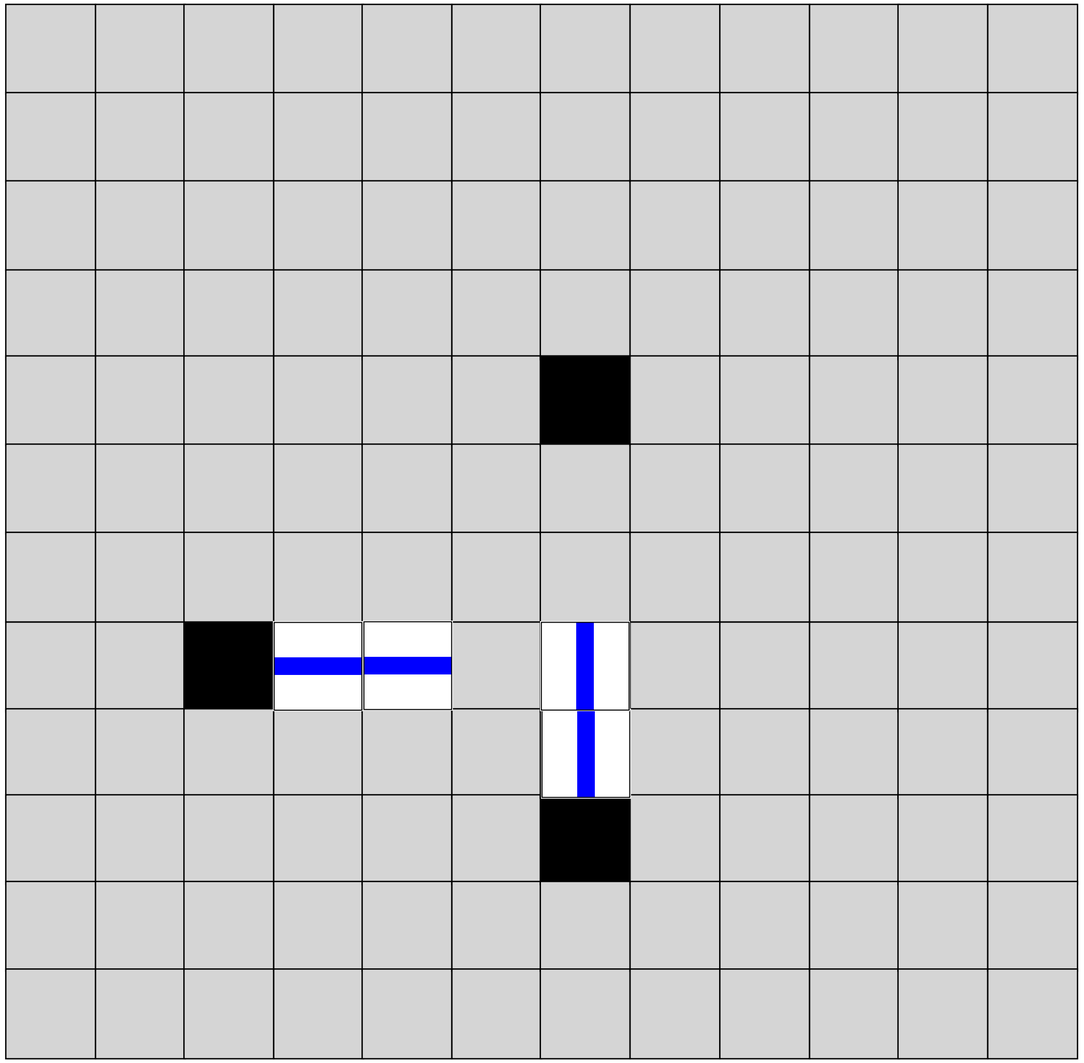
Future Work

- Nested WFC



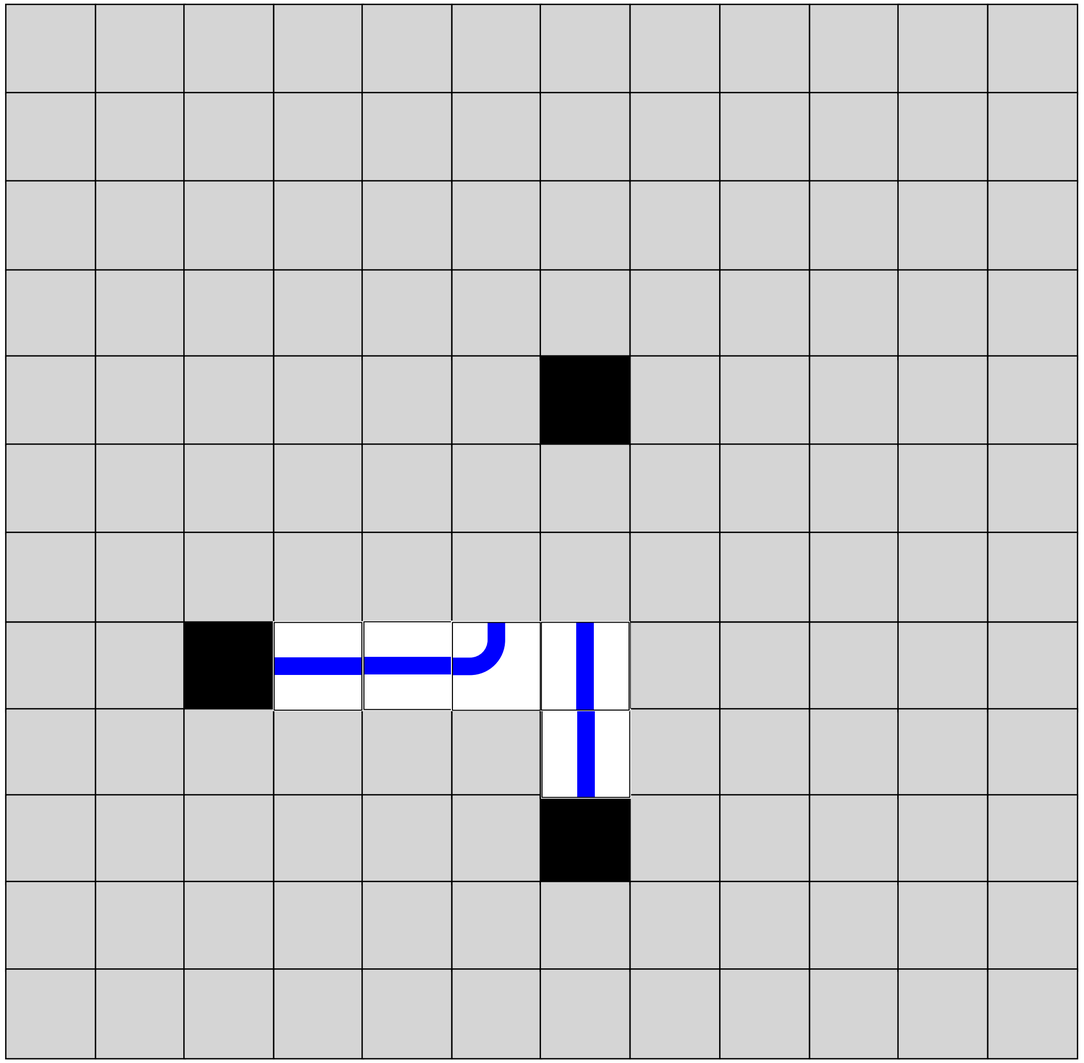
Future Work

- Nested WFC
- Collapsed tiles update



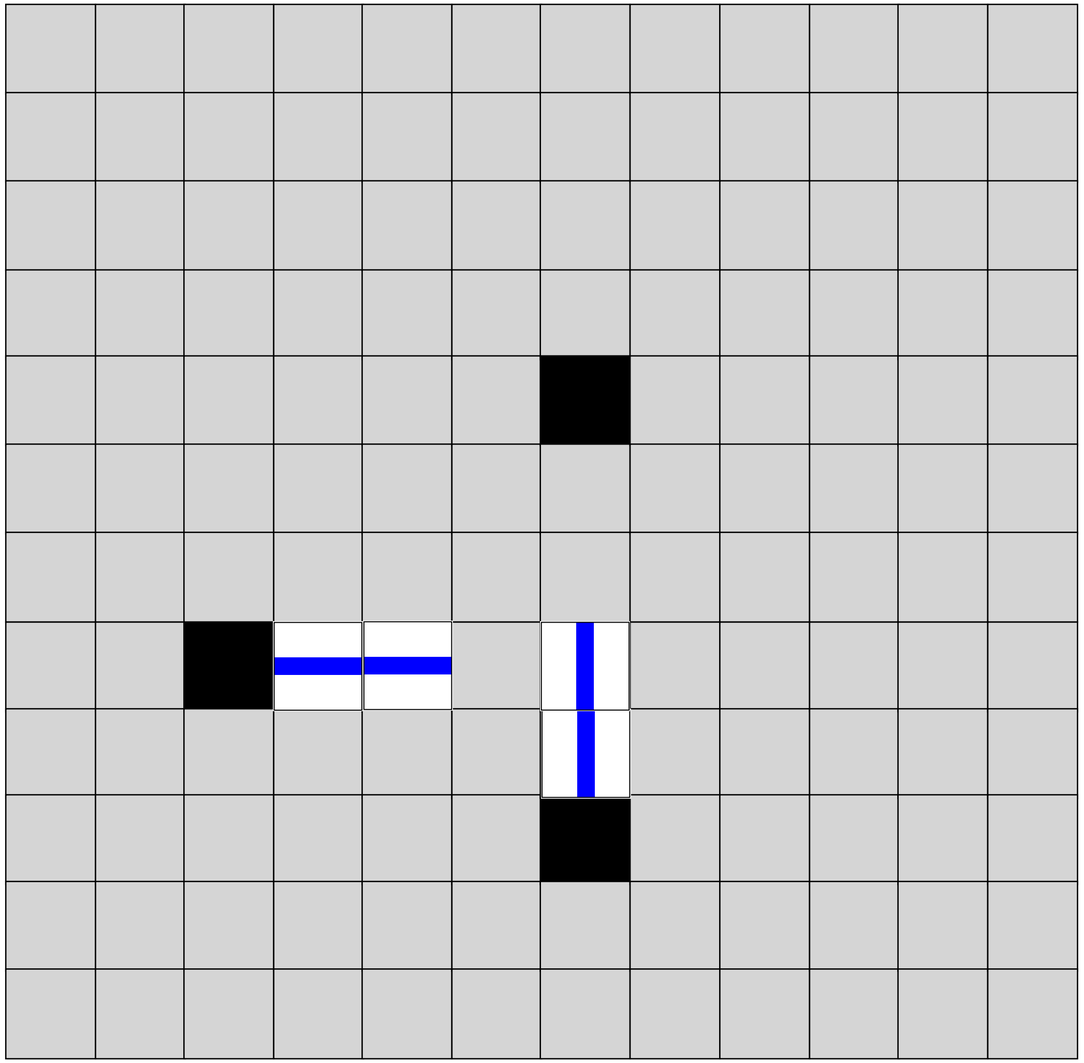
Future Work

- Nested WFC
- Collapsed tiles update



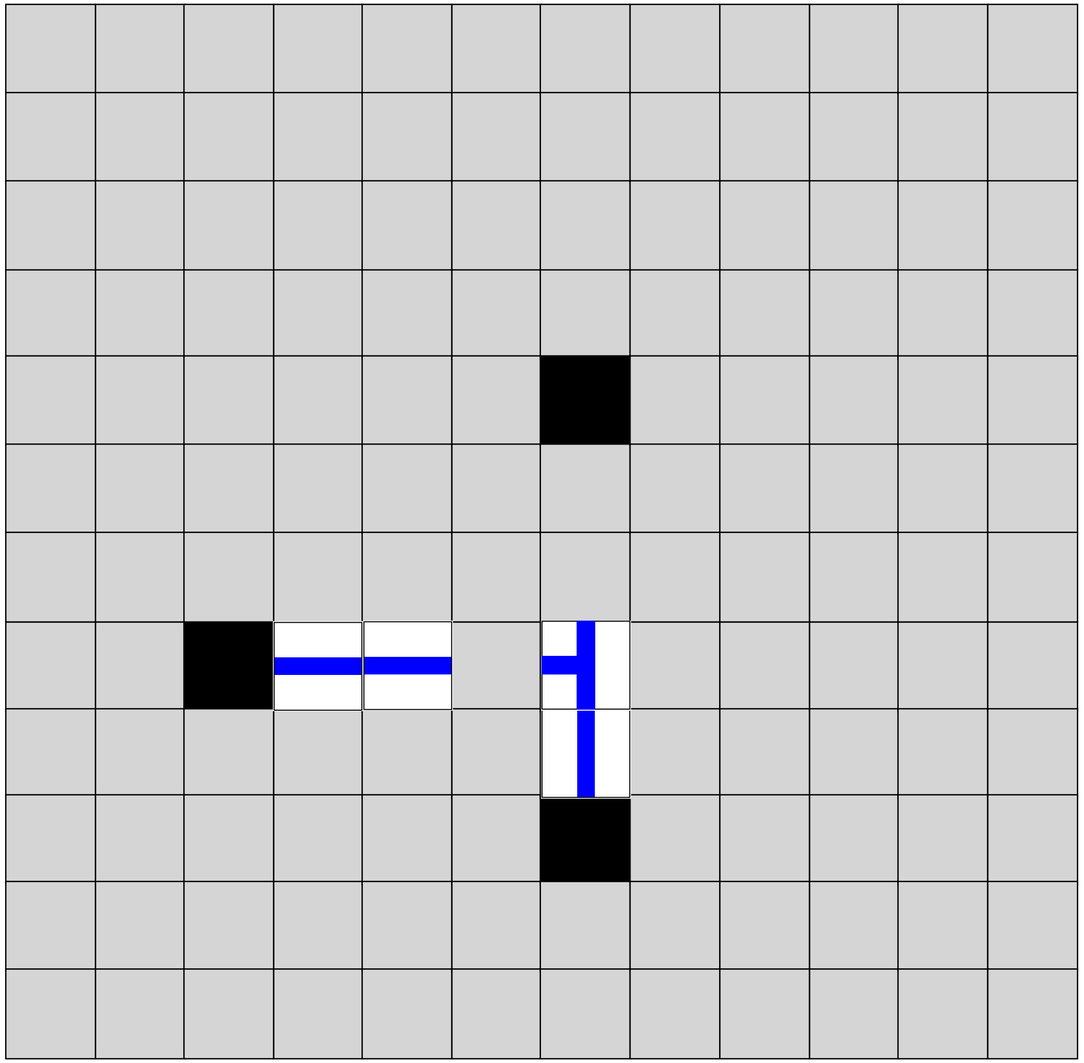
Future Work

- Nested WFC
- Collapsed tiles update



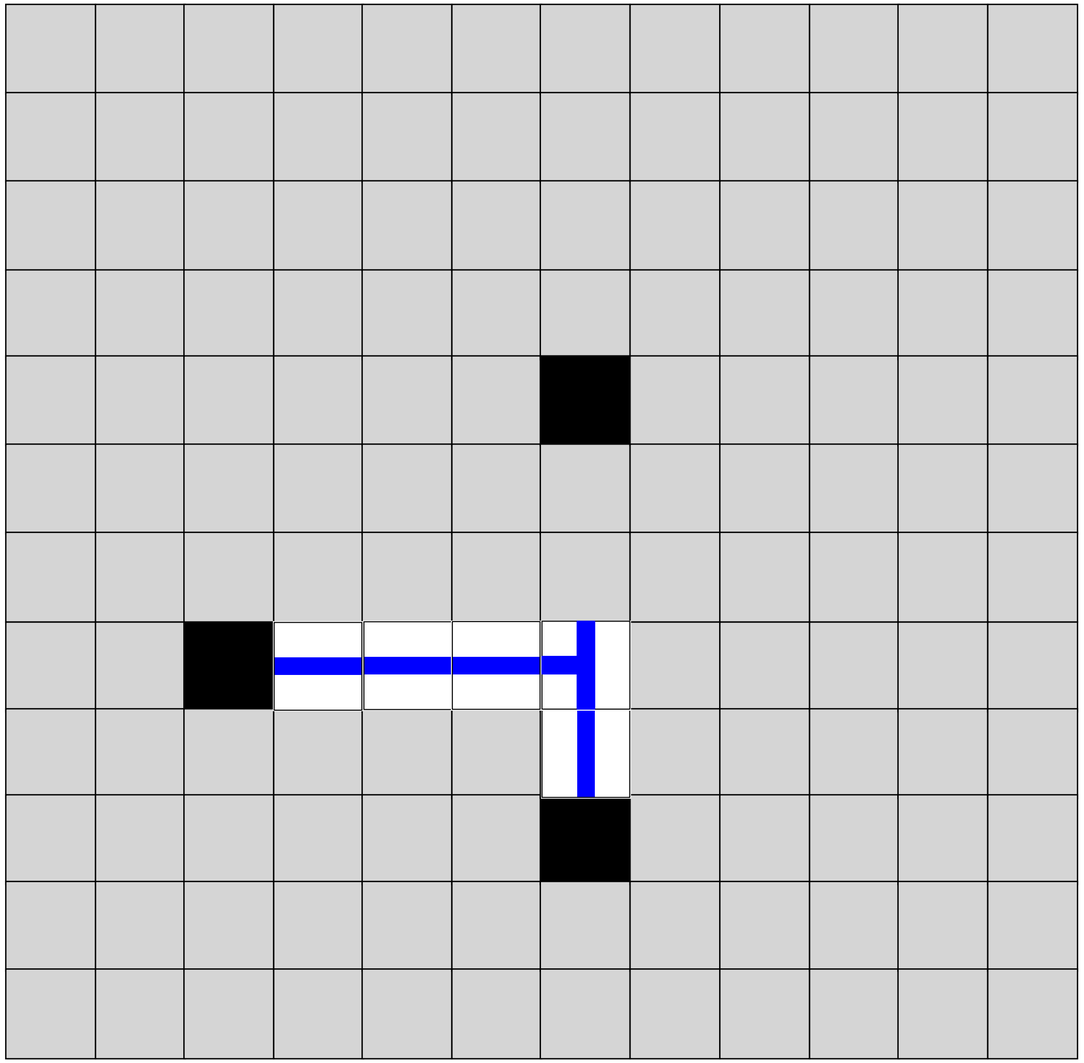
Future Work

- Nested WFC
- Collapsed tiles update



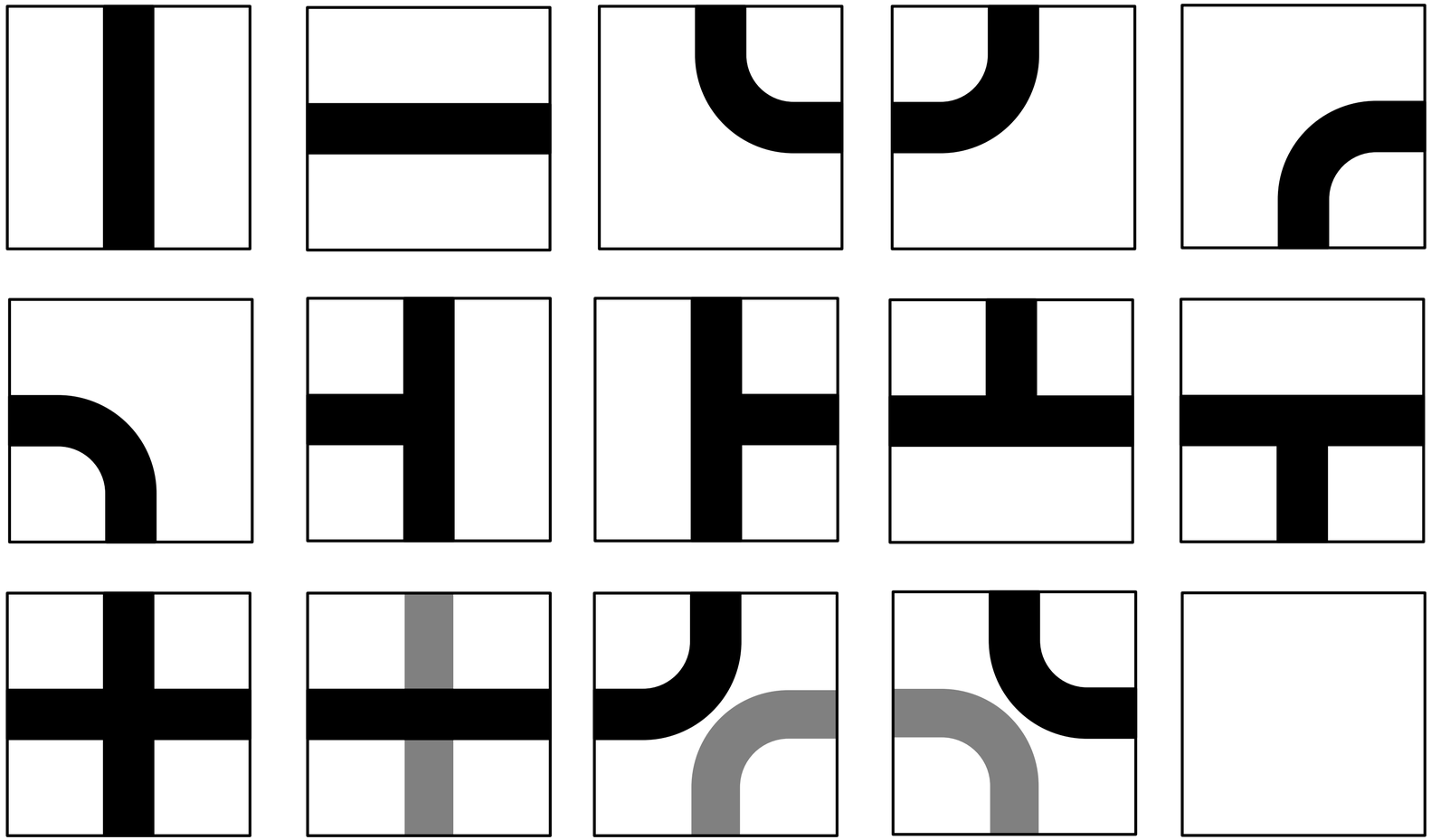
Future Work

- Nested WFC
- Collapsed tiles update

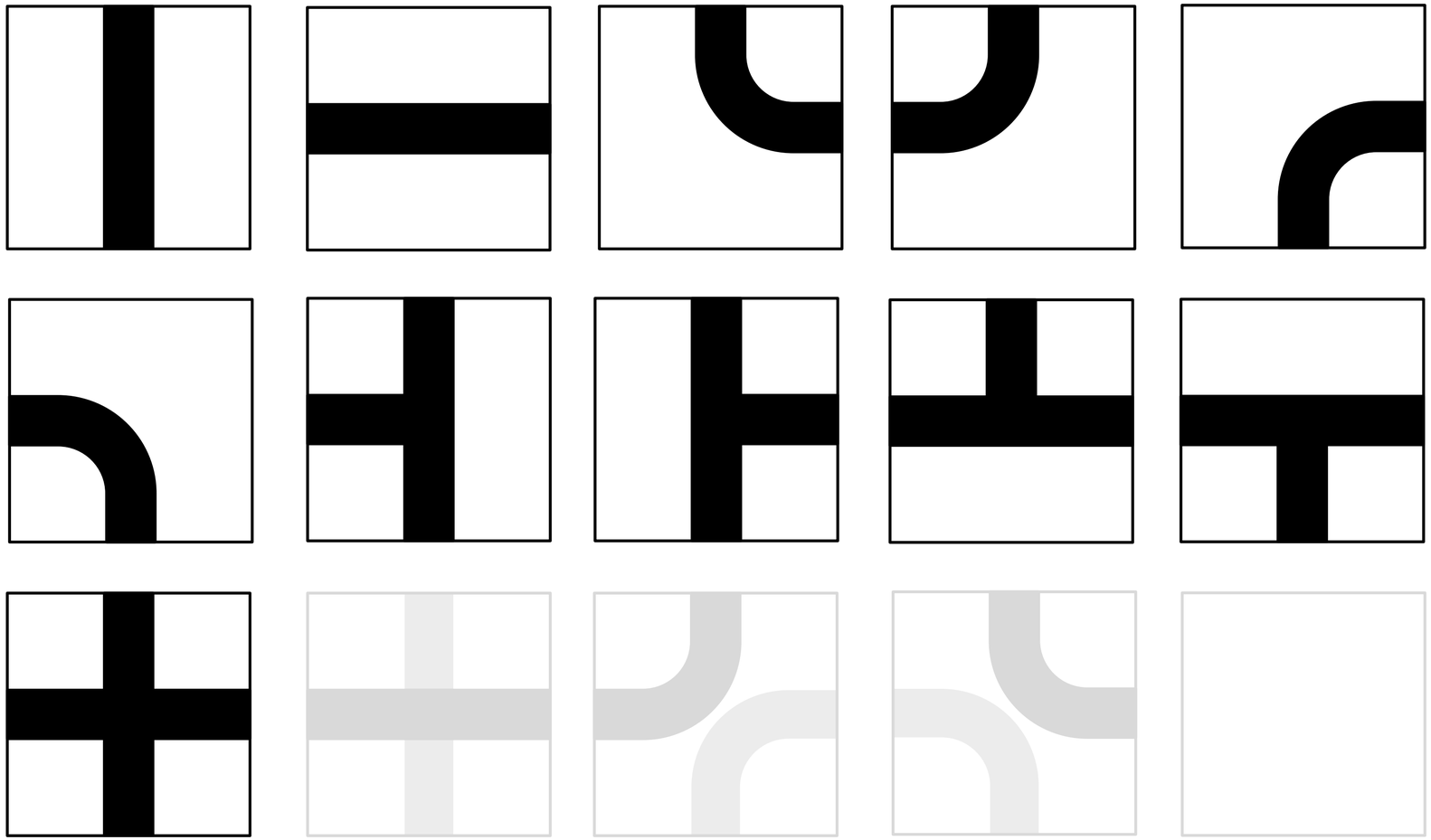


BACKUP SLIDES

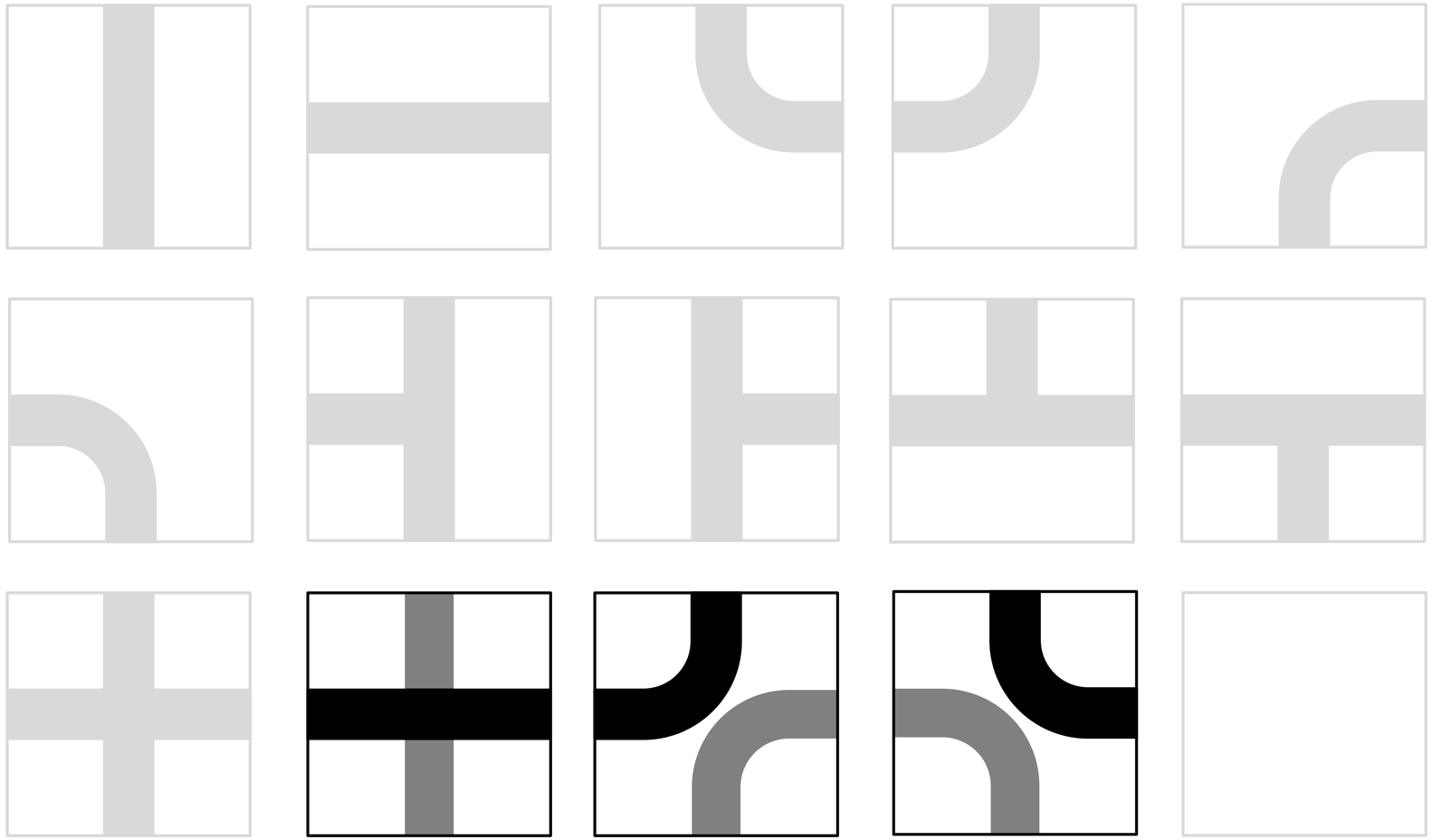
Possible Tiles



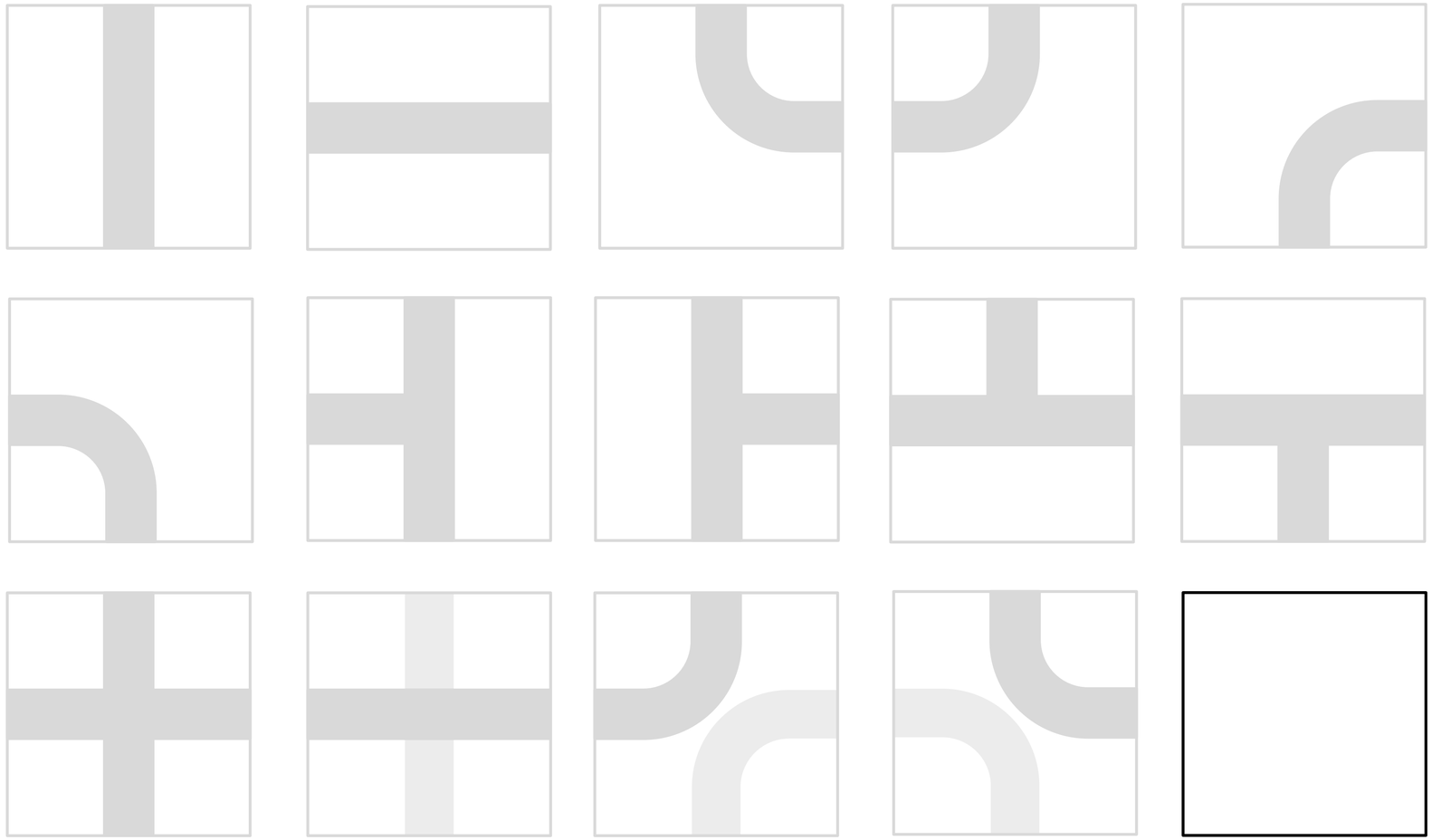
Possible Tiles



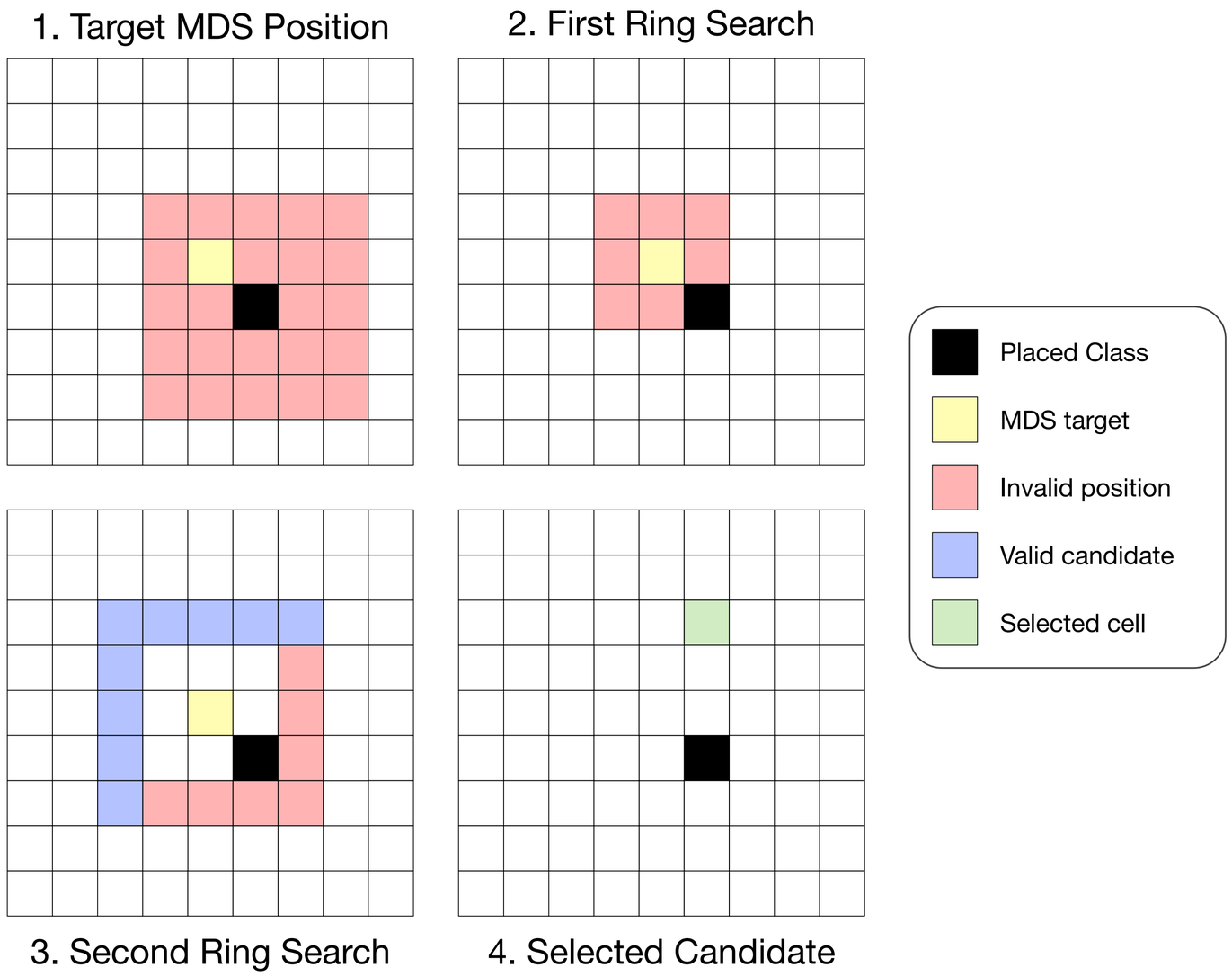
Possible Tiles



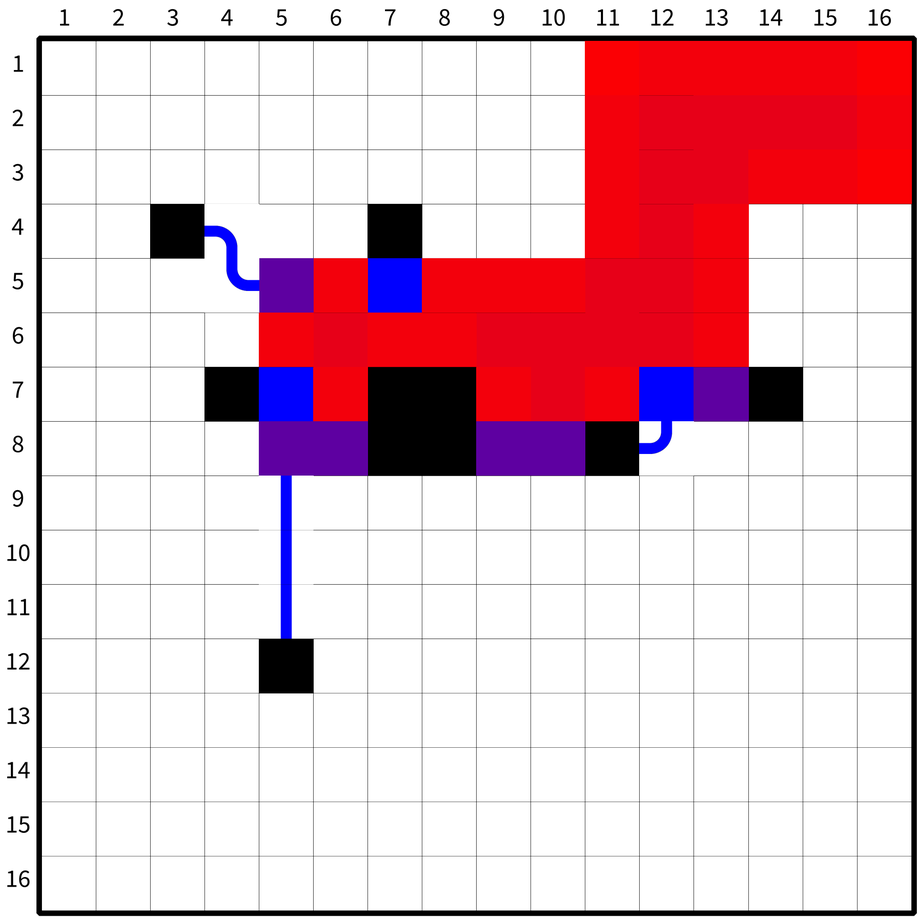
Possible Tiles



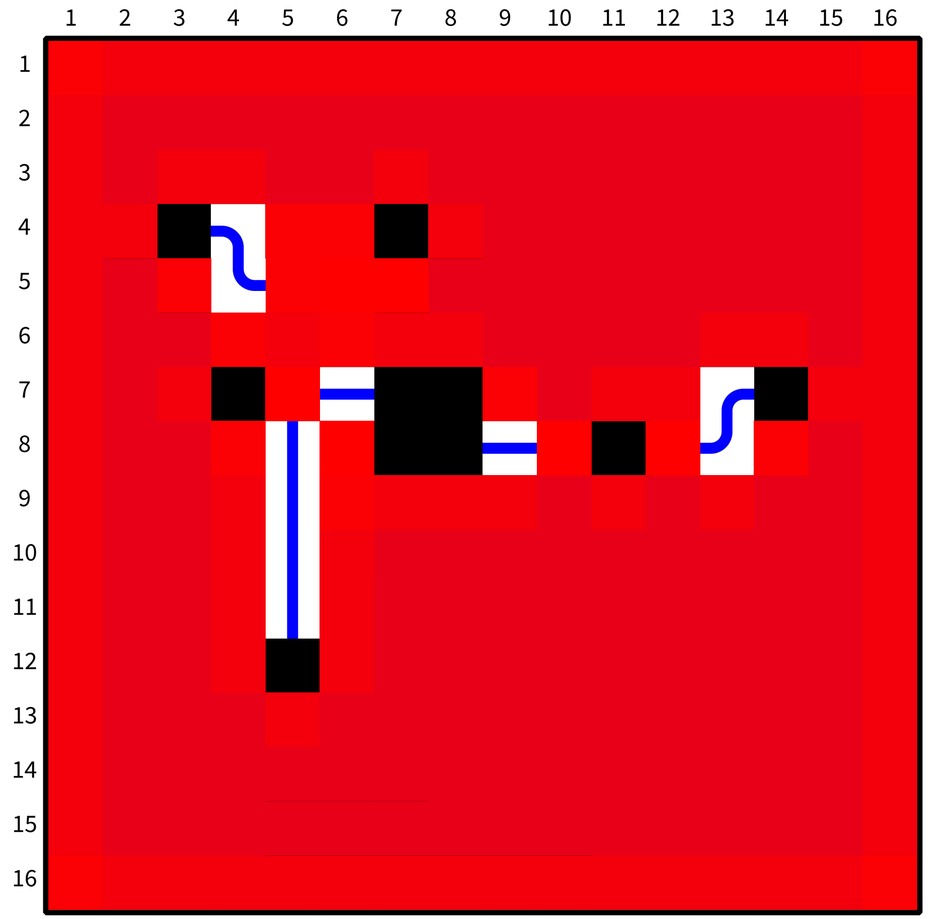
Class Placement



Maximum entropy heuristic

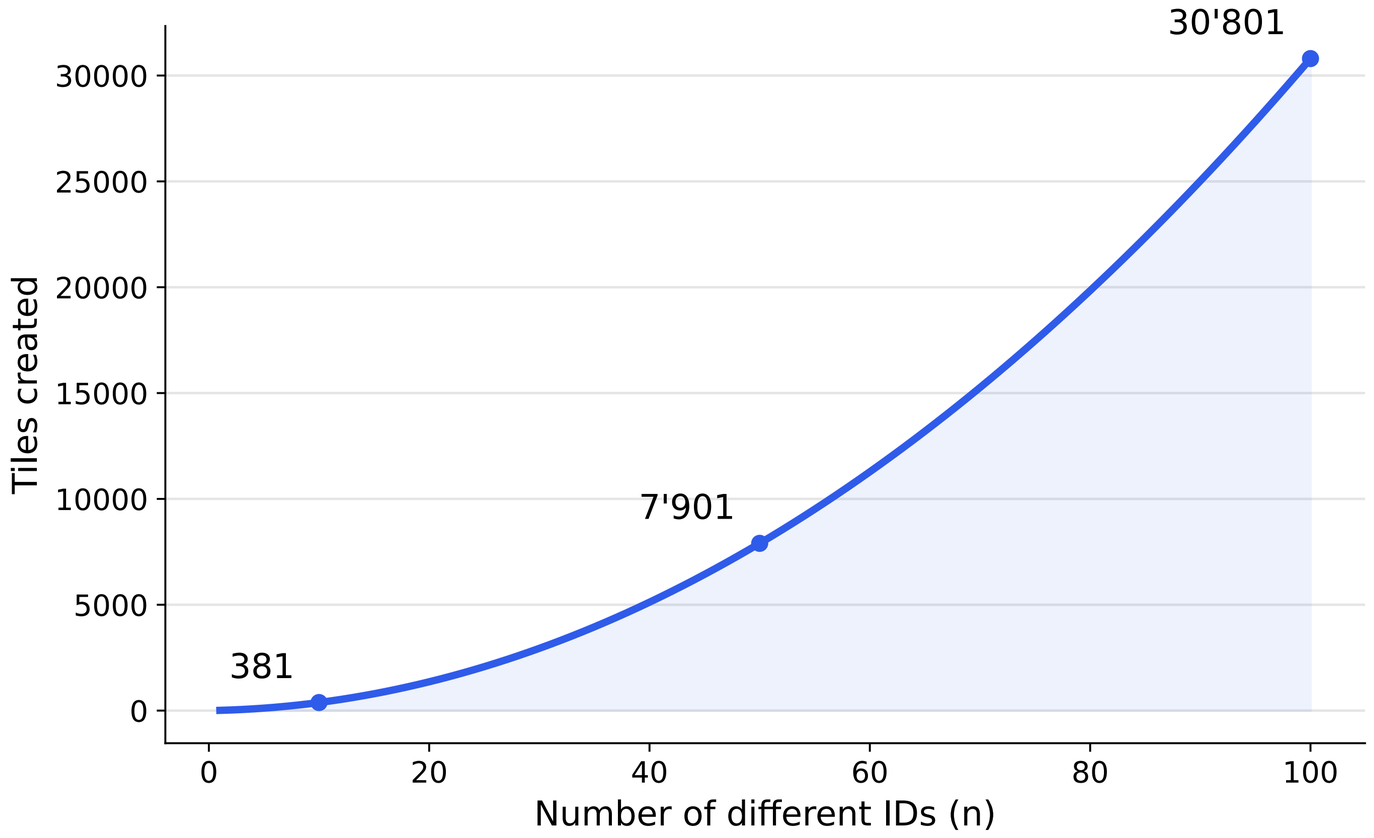


min-heap



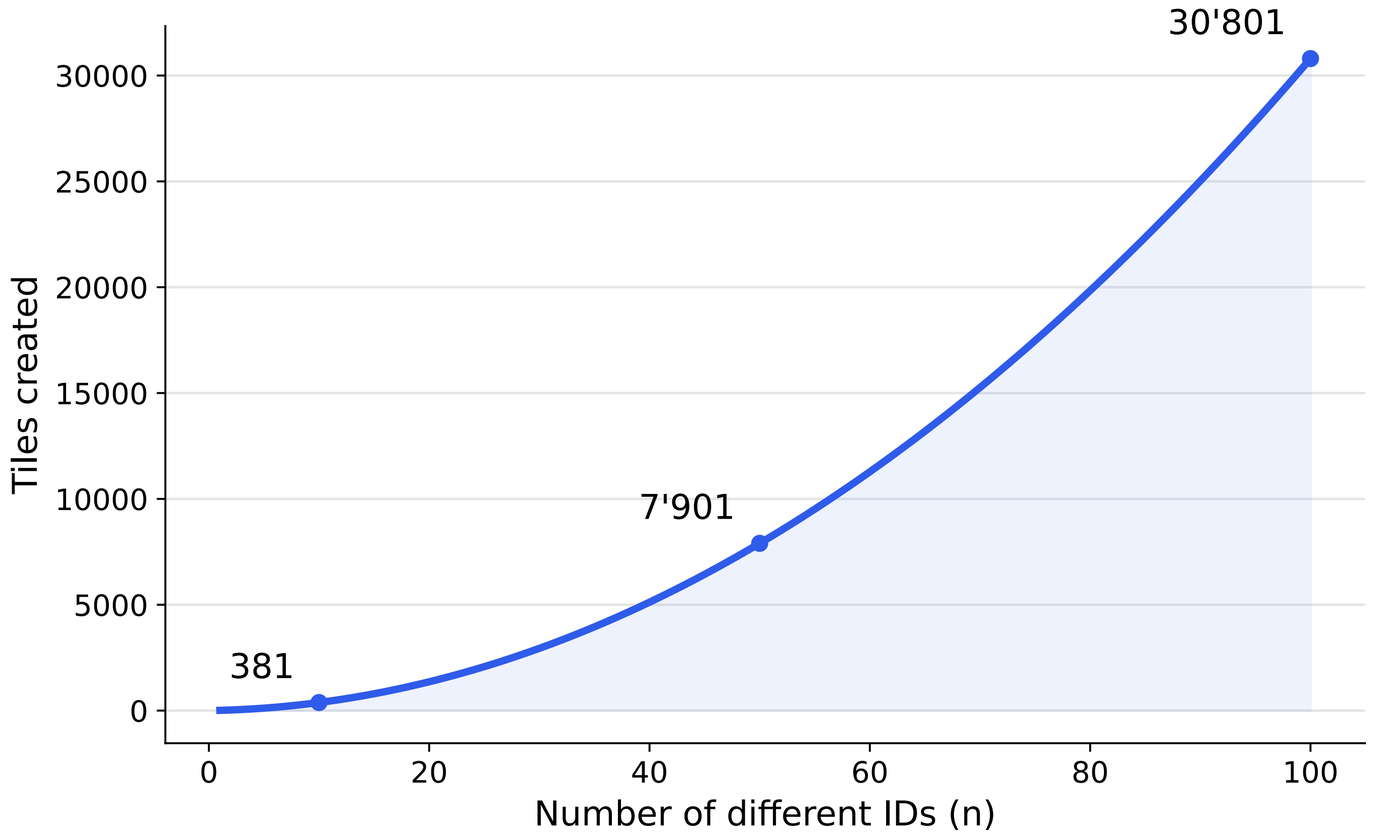
max-heap

Exploding number of tiles



TOT = 1 + 11n + 3n(n-1) =
= $3n^2 + 8n + 1$

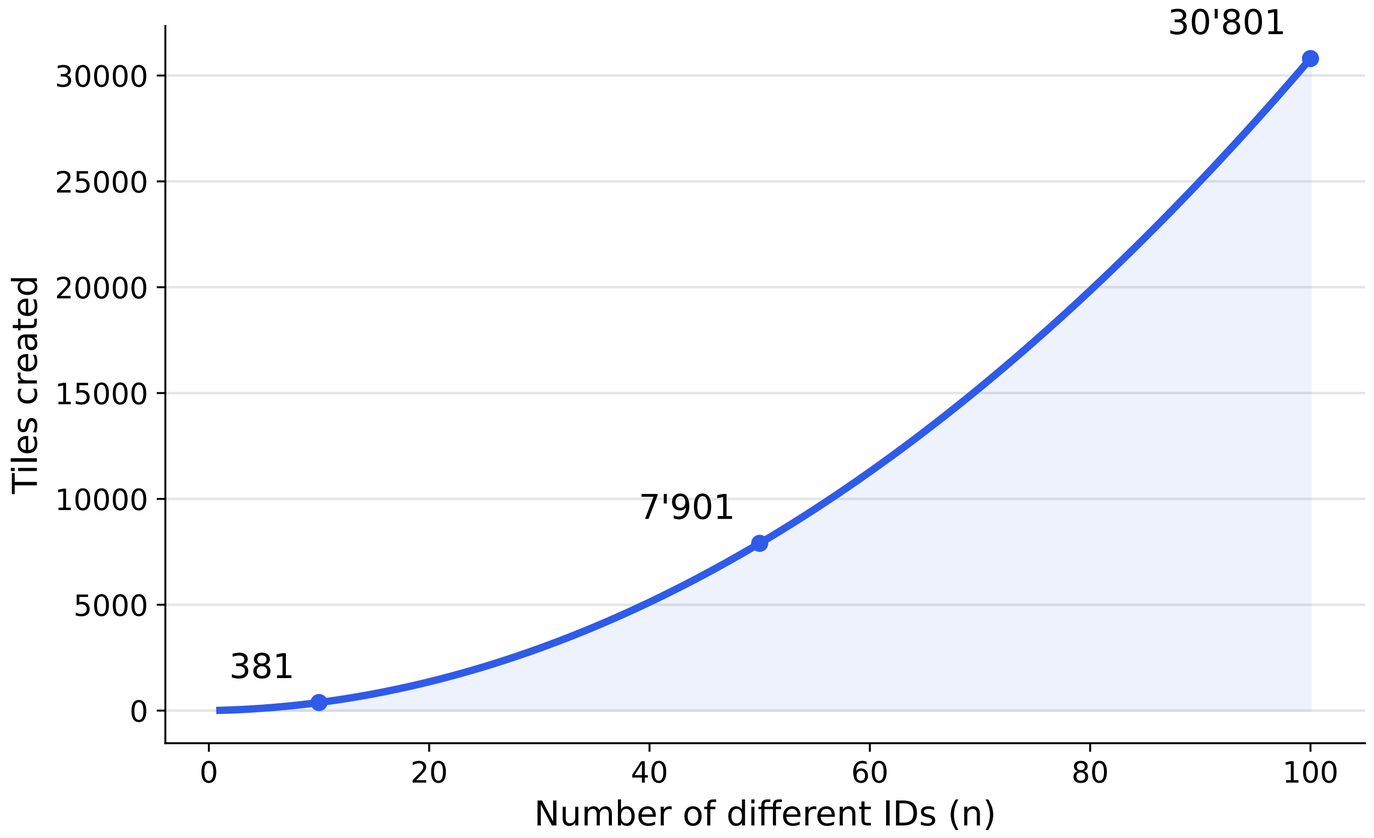
Exploding number of tiles



$TOT = 1 + 11n + 3n(n-1) =$
$= 3n^2 + 8n + 1$

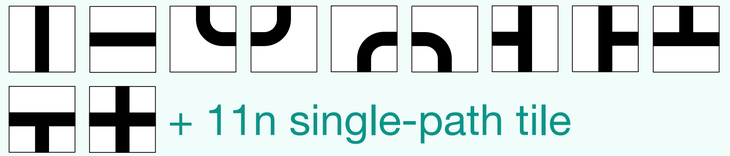
+ 1 empty tile

Exploding number of tiles



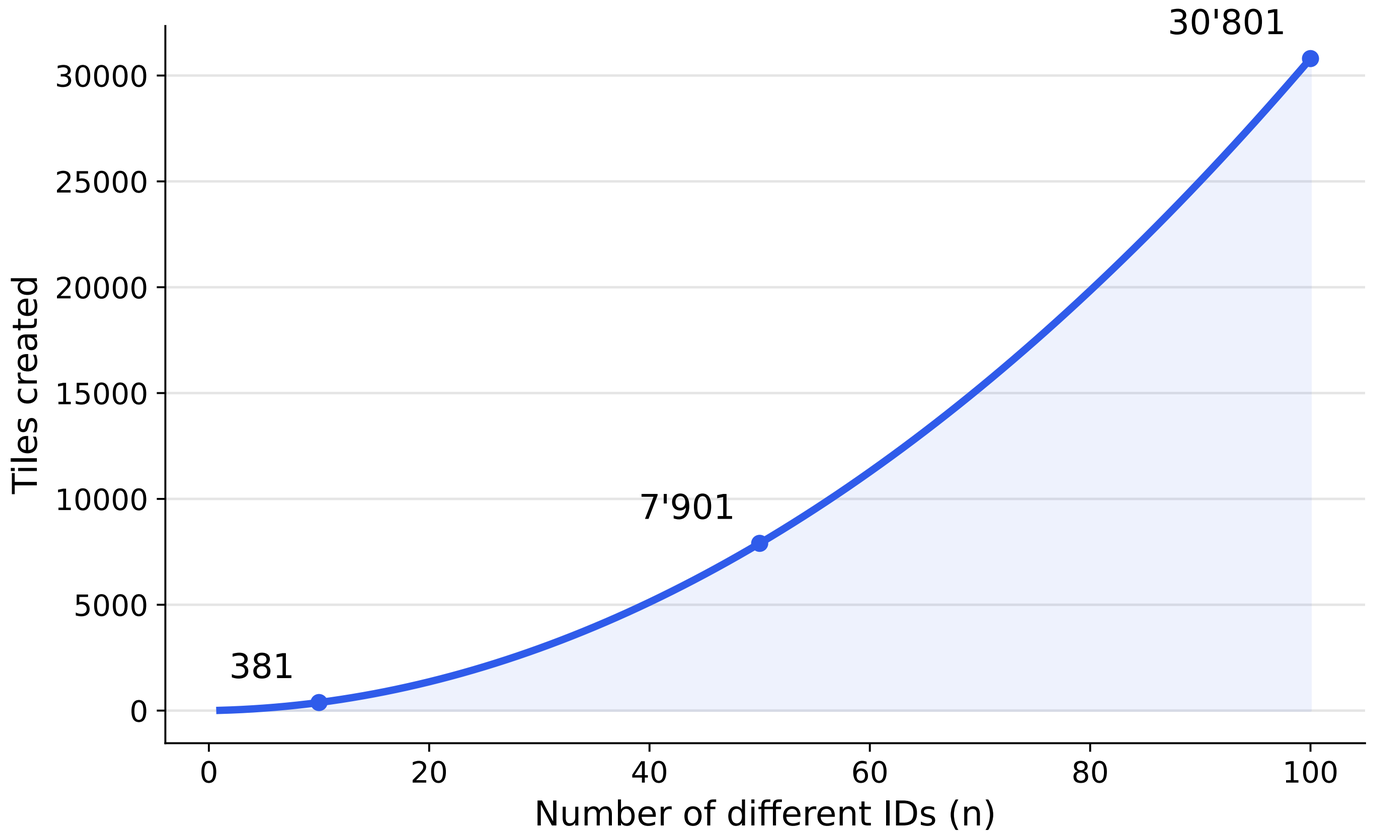
$$TOT = 1 + 11n + 3n(n-1) =$$
$$= 3n^2 + 8n + 1$$

+ 1 empty tile



+ 11n single-path tile

Exploding number of tiles



$TOT = 1 + 11n + 3n(n-1) =$
$= 3n^2 + 8n + 1$

+ 1 empty tile

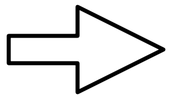
+ 11n single-path tile

+ 3n(n-1)

3 two-path tiles × other IDs

Distance Matrix

	Object	ProtoObject	MCReader	MCStReader	MCVersionReader	MCMczReader	MCFileTreeAbstractReader	MCFileTreeStReader	MCFileTreeStSnapshotReader	MCFileTreeStCypressReader	IceLibgitFiletreeReader	IceLibgitFiletreeLogReader	TonelReader	IceLibgitTonelReader	IceLibgitTonelLogReader
Object	0	1.0	1.0	Inf	Inf	Inf	Inf	Inf	Inf	Inf	Inf	Inf	Inf	Inf	Inf
ProtoObject	1.0	0	Inf	Inf	Inf	Inf	Inf	Inf	Inf	Inf	Inf	Inf	Inf	Inf	Inf
MCReader	1.0	Inf	0	1.0	1.0	Inf	Inf	Inf	Inf	Inf	Inf	Inf	Inf	Inf	Inf
MCStReader	Inf	Inf	1.0	0	Inf	Inf	Inf	Inf	Inf	Inf	Inf	Inf	Inf	Inf	Inf
MCVersionReader	Inf	Inf	1.0	Inf	0	1.0	Inf	Inf	Inf	Inf	Inf	Inf	1.0	Inf	Inf
MCMczReader	Inf	Inf	Inf	Inf	1.0	0	1.0	Inf	Inf	Inf	Inf	Inf	Inf	Inf	Inf
MCFileTreeAbstractReader	Inf	Inf	Inf	Inf	Inf	1.0	0	1.0	1.0	Inf	Inf	Inf	Inf	Inf	Inf
MCFileTreeStReader	Inf	Inf	Inf	Inf	Inf	Inf	1.0	0	Inf	Inf	Inf	Inf	Inf	Inf	Inf
MCFileTreeStSnapshotReader	Inf	Inf	Inf	Inf	Inf	Inf	1.0	Inf	0	1.0	Inf	Inf	Inf	Inf	Inf
MCFileTreeStCypressReader	Inf	Inf	Inf	Inf	Inf	Inf	Inf	Inf	1.0	0	1.0	Inf	Inf	Inf	Inf
IceLibgitFiletreeReader	Inf	Inf	Inf	Inf	Inf	Inf	Inf	Inf	Inf	1.0	0	1.0	Inf	Inf	Inf
IceLibgitFiletreeLogReader	Inf	Inf	Inf	Inf	Inf	Inf	Inf	Inf	Inf	Inf	1.0	0	Inf	Inf	Inf
TonelReader	Inf	Inf	Inf	Inf	1.0	Inf	Inf	Inf	Inf	Inf	Inf	Inf	0	1.0	Inf
IceLibgitTonelReader	Inf	Inf	Inf	Inf	Inf	Inf	Inf	Inf	Inf	Inf	Inf	Inf	1.0	0	1.0
IceLibgitTonelLogReader	Inf	Inf	Inf	Inf	Inf	Inf	Inf	Inf	Inf	Inf	Inf	Inf	Inf	1.0	0



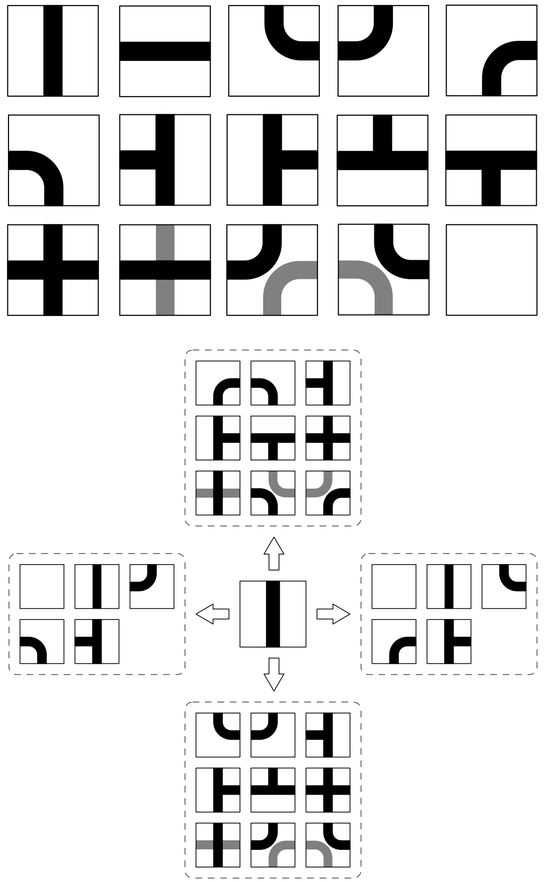
	Object	ProtoObject	MCReader	MCStReader	MCVersionReader	MCMczReader	MCFileTreeAbstractReader	MCFileTreeStReader	MCFileTreeStSnapshotReader	MCFileTreeStCypressReader	IceLibgitFiletreeReader	IceLibgitFiletreeLogReader	TonelReader	IceLibgitTonelReader	IceLibgitTonelLogReader
Object	0	1.0	1.0	2.0	2.0	3.0	4.0	5.0	5.0	6.0	7.0	8.0	3.0	4.0	5.0
ProtoObject	1.0	0	2.0	3.0	3.0	4.0	5.0	6.0	6.0	7.0	8.0	9.0	4.0	5.0	6.0
MCReader	1.0	2.0	0	1.0	1.0	2.0	3.0	4.0	4.0	5.0	6.0	7.0	2.0	3.0	4.0
MCStReader	2.0	3.0	1.0	0	2.0	3.0	4.0	5.0	5.0	6.0	7.0	8.0	3.0	4.0	5.0
MCVersionReader	2.0	3.0	1.0	2.0	0	1.0	2.0	3.0	3.0	4.0	5.0	6.0	1.0	2.0	3.0
MCMczReader	3.0	4.0	2.0	3.0	1.0	0	1.0	2.0	2.0	3.0	4.0	5.0	2.0	3.0	4.0
MCFileTreeAbstractReader	4.0	5.0	3.0	4.0	2.0	1.0	0	1.0	1.0	2.0	3.0	4.0	3.0	4.0	5.0
MCFileTreeStReader	5.0	6.0	4.0	5.0	3.0	2.0	1.0	0	2.0	3.0	4.0	5.0	4.0	5.0	6.0
MCFileTreeStSnapshotReader	5.0	6.0	4.0	5.0	3.0	2.0	1.0	2.0	0	1.0	2.0	3.0	4.0	5.0	6.0
MCFileTreeStCypressReader	6.0	7.0	5.0	6.0	4.0	3.0	2.0	3.0	1.0	0	1.0	2.0	5.0	6.0	7.0
IceLibgitFiletreeReader	7.0	8.0	6.0	7.0	5.0	4.0	3.0	4.0	2.0	1.0	0	1.0	6.0	7.0	8.0
IceLibgitFiletreeLogReader	8.0	9.0	7.0	8.0	6.0	5.0	4.0	5.0	3.0	2.0	1.0	0	7.0	8.0	9.0
TonelReader	3.0	4.0	2.0	3.0	1.0	2.0	3.0	4.0	4.0	5.0	6.0	7.0	0	1.0	2.0
IceLibgitTonelReader	4.0	5.0	3.0	4.0	2.0	3.0	4.0	5.0	5.0	6.0	7.0	8.0	1.0	0	1.0
IceLibgitTonelLogReader	5.0	6.0	4.0	5.0	3.0	4.0	5.0	6.0	6.0	7.0	8.0	9.0	2.0	1.0	0

Backtracking strategies

- Fixed Number of Steps
- Percentage
- Many Possible Tiles
- Similar Tile Weights
- Restart

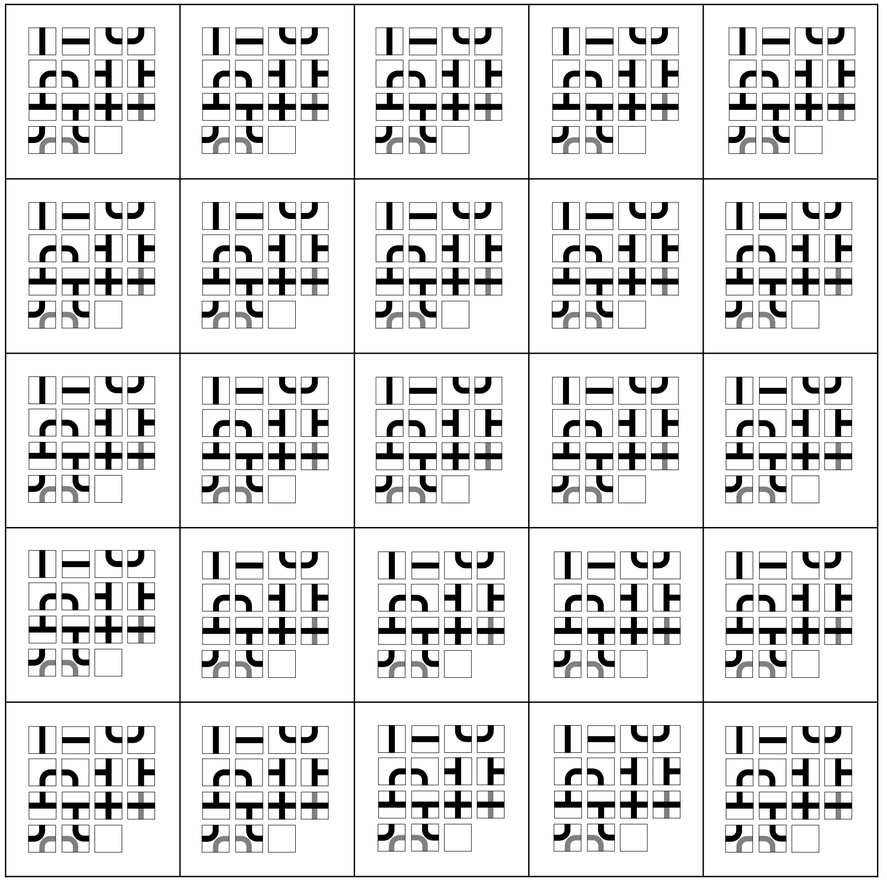
WFC pseudo-code

1. build the tileset and constraint tables



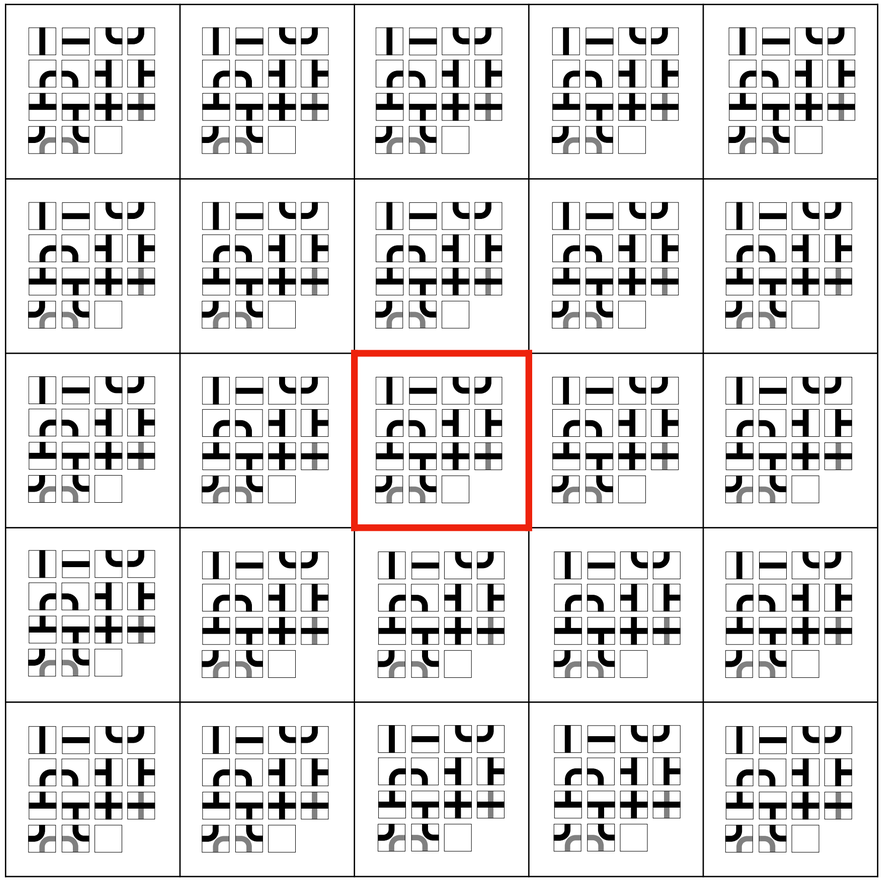
WFC pseudo-code

1. build the tileset and constraint tables
2. **repeat**



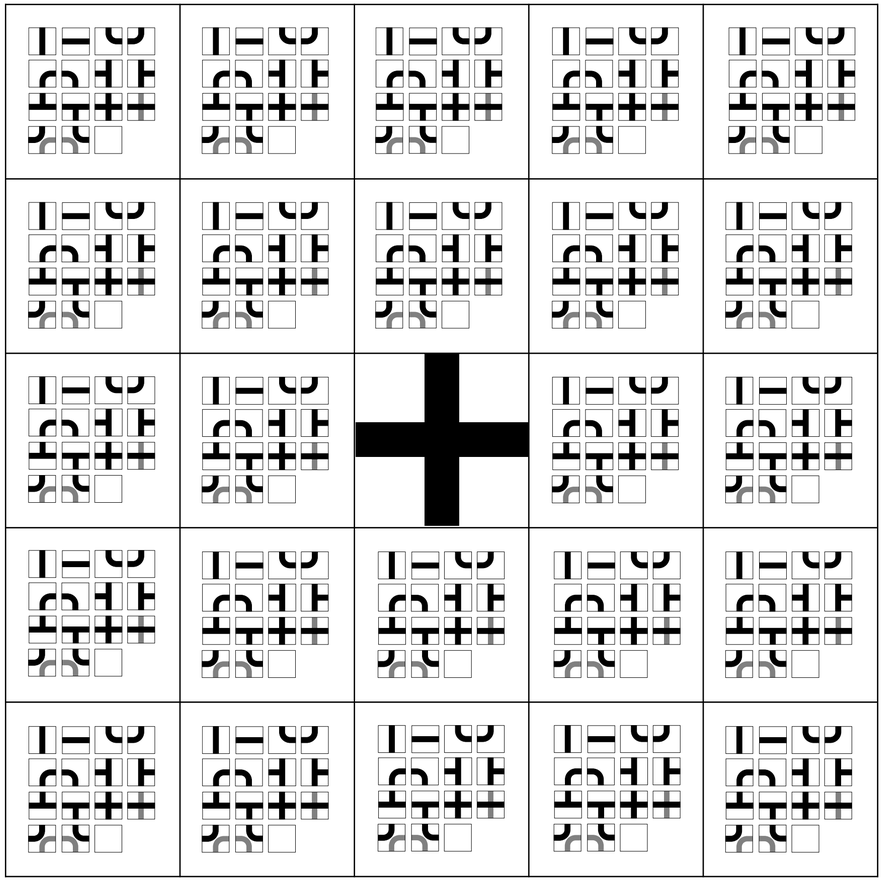
WFC pseudo-code

1. build the tileset and constraint tables
2. **repeat**
3. select the next cell to collapse



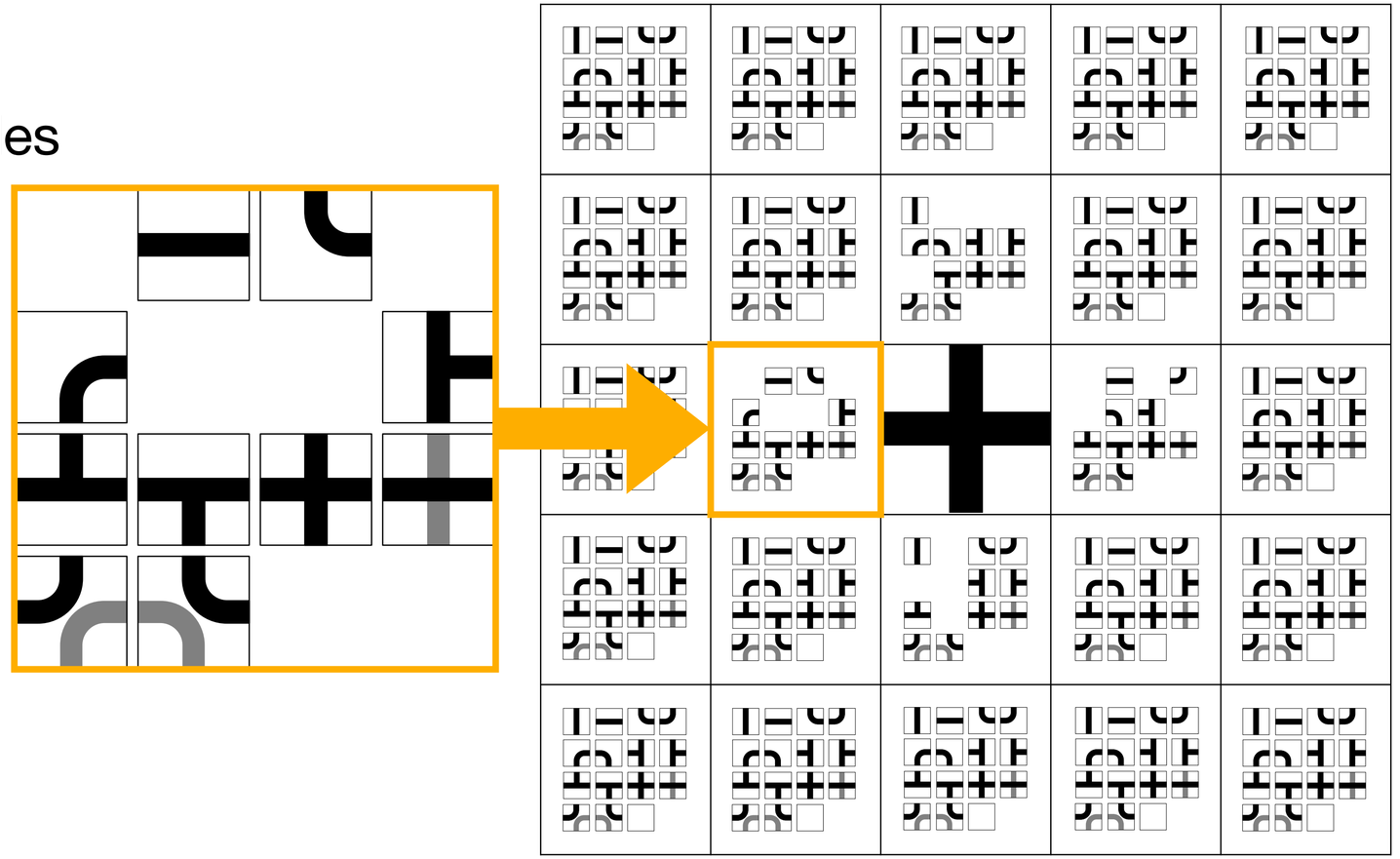
WFC pseudo-code

1. build the tileset and constraint tables
2. **repeat**
3. select the next cell to collapse
4. choose a tile for that cell



WFC pseudo-code

1. build the tileset and constraint tables
2. **repeat**
3. select the next cell to collapse
4. choose a tile for that cell
5. propagate the constraints



WFC pseudo-code

1. build the tileset and constraint tables
2. **repeat**
3. select the next cell to collapse
4. choose a tile for that cell
5. propagate the constraints
6. **until** all cells collapse or a conflict occurs

